

Diseño y Estructura de un CAS: Programación simbólica en LISP

Jose Daniel Fernandez Santos (@dfsantos1)

Parte I — Conociendo LISP a través de MyLISP

Implementaciones de referencia: MyLISP/QL (Prospero Pro Pascal · Sinclair QL) y MyLISP/Next (Z88DK C · ZX Spectrum Next)

Índice

Parte I — Conociendo LISP a través de MyLISP

- Capítulo 1 — ¿Qué es MYLISP?
- Capítulo 2 — Primeros pasos
- Capítulo 3 — El modelo de datos de LISP
- Capítulo 4 — Números y aritmética exacta
- Capítulo 5 — Trabajar con listas
- Capítulo 6 — Control del flujo
- Capítulo 7 — Definir funciones y clausuras léxicas
- Capítulo 8 — Cadenas, PRINT y LOAD
- Capítulo 9 — Galería de programas

Parte II — Cálculo simbólico: Construyendo un CAS

- Capítulo 10 — Representación y abstracción de datos
- Capítulo 11 — Sustitución (SUBST) y evaluación numérica
- Capítulo 12 — Derivación simbólica "cruda" (DERIV)
- Capítulo 13 — Simplificación mediante "Constructores Inteligentes"
- Capítulo 14 — Orden canónico y términos semejantes
- Capítulo 15 — Simplificación de árbol completo (SIMP)
- Capítulo 16 — Simplificador de productos (SIMPPROD)
- Capítulo 17 — El orquestador: DERIVA y DERIVAEN
- Capítulo 18 — Orden canónico estable (EXPLESS?)
- Capítulo 19 — Impresión infija (PRINTMAT)
- Capítulo 20 — Resumen de nuestro CAS: qué construimos y qué no

Apéndices

- Apéndice A — Referencia rápida (MyLISP)
- Apéndice B — Arquitectura y limitaciones (MyLISP)
- Apéndice C — Mensajes de error (MyLISP)
- Apéndice D — Referencia rápida del CAS: todas las funciones

- Apéndice E — Código fuente completo de `CAS.LISP`

\newpage

Capítulo 1 — ¿Qué es MYLISP?

1.1 Una breve historia de LISP

La historia de LISP es, en el fondo, la historia de cómo una notación matemática en papel se convirtió casi por accidente en un lenguaje de programación —y de cómo esa traducción generó un debate sobre el diseño de lenguajes que sigue vivo hoy.

1.1.1 El origen: del cálculo lambda a la máquina (1958)

En 1958, John McCarthy, matemático del Instituto Tecnológico de Massachusetts (MIT), publicó un artículo titulado *"Recursive Functions of Symbolic Expressions and Their Computation by Machine"*. Su propósito no era crear un lenguaje ejecutable: buscaba una **notación matemática** basada en el *cálculo lambda* de Alonzo Church para razonar sobre algoritmos de inteligencia artificial. Definió las S-expressions (expresiones simbólicas) y una función universal `eval` capaz de interpretar esas expresiones.

El salto a la informática real lo dio **Steve Russell**, uno de los estudiantes de McCarthy. Russell leyó el artículo y se dio cuenta de que podía implementar `eval` en código máquina para un IBM 704. McCarthy, según cuenta la leyenda, le respondió que `eval` era matemática teórica, no código real. Russell lo programó de todos modos. Así nació el primer intérprete LISP.

Lo que hizo a LISP radicalmente diferente de los lenguajes de la época (FORTRAN, COBOL, orientados al cálculo numérico o al proceso de datos) fue la acumulación de ideas que nadie había puesto juntas antes: **recolección de basura automática**, **tipado dinámico**, **funciones como valores de primera clase** y, sobre todo, la **homoiconicidad**: el código y los datos tienen exactamente la misma estructura. Un programa LISP puede crear, analizar y ejecutar otros programas LISP porque ambos son listas de símbolos.

```
(+ 2 3)           ; esto es a la vez código ejecutable...
(QUOTE (+ 2 3))   ; ...y un dato: la lista de tres elementos +, 2 y 3
```

A lo largo de los años sesenta y setenta, LISP se convirtió en el instrumento con el que los investigadores del MIT construyeron los primeros programas capaces de demostrar teoremas matemáticos, jugar al ajedrez, traducir idiomas y resolver integrales simbólicas. El Sistema de Álgebra Computacional **Macsyma** —quizás el proyecto de software más ambicioso del MIT en aquella era— estaba escrito casi en su totalidad en LISP.

1.1.2 La divergencia: LISP-1 contra LISP-2

A medida que LISP crecía durante los años 60 y 70, surgieron múltiples dialectos —MacLISP, InterLISP, ZetaLISP— y con ellos un problema de diseño profundo que dividió a la familia LISP en dos ramas que coexisten hasta hoy: la cuestión de **cuántos espacios de nombres** debe tener el lenguaje.

La pregunta es sencilla: cuando el intérprete ve el símbolo `SUMA`, ¿lo busca siempre en el mismo diccionario, o tiene diccionarios separados según si aparece en posición de función o de variable?

LISP-1 tiene un único espacio de nombres. Las funciones y las variables viven en el mismo diccionario. Un símbolo como `LISTA` sólo puede apuntar a una cosa a la vez. El campeón de este enfoque fue **Scheme**, creado por Guy L. Steele y Gerald Jay Sussman en 1975, que además formalizó el **ámbito léxico** estricto (las funciones capturan el entorno donde fueron definidas, no donde son llamadas). Matemáticamente, el LISP-1 es fiel al cálculo lambda: las funciones son ciudadanos de primera clase exactamente igual que los números. Si tienes una función, la pasas a otra exactamente igual que si fuera un dato, sin ninguna sintaxis especial.

LISP-2 tiene espacios de nombres separados. Un símbolo tiene una *celda de valor* y una *celda de función* independientes. Cuando el intérprete ve `(SUMA A B)`, busca `SUMA` en la celda de función; cuando ve `SUMA` como argumento, busca en la celda de valor. El campeón de este modelo fue **Common LISP**, estandarizado en 1984 para unificar la fragmentación industrial de los dialectos de los años 70. Common LISP es pragmático: puedes tener una variable llamada `list` que contenga datos y seguir llamando a la función `(list 1 2 3)` sin que el intérprete se confunda. El precio es la verbosidad en programación funcional: para pasar una función como argumento hay que usar el operador `#'`, y para llamarla desde una variable hay que usar `funcall`.

```
;; En Common LISP (LISP-2): la sintaxis de la celda de valor
(mapcar #'doble '(1 2 3))

;; En Scheme / MYLISP (LISP-1): una función es un valor como cualquier otro
(MY-MAP DOBLE '(1 2 3))
```

La elección de Common LISP por el modelo LISP-2 fue, en gran medida, pragmática: mantener compatibilidad con el enorme código MacLISP existente en las máquinas LISP de los laboratorios del MIT y de empresas como Symbolics. Common LISP acabó siendo un lenguaje masivo y multiparadigma —incluye CLOS, uno de los sistemas de objetos más potentes jamás diseñados— orientado a construir software industrial a gran escala.

Scheme tomó el camino opuesto: el mínimo número de primitivas y reglas, máxima pureza conceptual. Fue diseñado para enseñar y razonar sobre la computación. Su objetivo declarado, en palabras de sus autores, era demostrar que "un lenguaje de programación puede ser poderoso y expresivo con muy pocas reglas".

1.1.3 Por qué MYLISP sigue el camino de Scheme

MYLISP es un LISP-1. Esta no es una decisión de implementación menor: define el modelo mental con el que el programador trabaja. Si el objetivo es enseñar LISP en un entorno con recursos limitados —el Sinclair QL— y construir un CAS desde los fundamentos entendiendo cada barrera de abstracción, el enfoque LISP-1 es el arquitectónicamente correcto.

Elimina la fricción cognitiva entre "datos" y "código". Cuando en el capítulo 10 pasemos una función de derivación como argumento a otra función, o devolvamos clausuras que capturan expresiones algebraicas, no necesitaremos sintaxis especial ni operadores adicionales. Todo —funciones, números, listas, expresiones simbólicas— es simplemente un valor, y se trata de la misma manera.

1.2 Las plataformas: Sinclair QL y ZX Spectrum Next

MYLISP existe en dos implementaciones independientes que comparten el mismo lenguaje y la misma filosofía, pero corren sobre hardware completamente diferente. Conocer ambas te ayuda a entender por qué algunas limitaciones son del lenguaje y otras son de la plataforma.

1.2.1 El Sinclair QL

El Sinclair QL (*Quantum Leap*) fue presentado por Sir Clive Sinclair en enero de 1984. Equipado con un procesador Motorola 68008 a 7,5 MHz y 128 KB de RAM en su configuración base, el QL fue el primer ordenador personal de 32 bits accesible al público en general. Su precio era notablemente inferior al de los sistemas profesionales de la época.

El QL incorporaba un sistema operativo multitarea propio —QDOS— y dos unidades de microdrive para almacenamiento. Los microdrive eran cartuchos de cinta magnética en bucle que funcionaban como si fueran discos, aunque con velocidades y fiabilidad inferiores. Los ficheros en el QL se nombran con una ruta del tipo `mdv1_nombre`, donde `mdv1` indica la unidad física.

Para hacer funcionar MYLISP/QL se necesita una expansión de memoria hasta al menos **640 KB**. Con esa RAM disponible, el intérprete dispone de un heap de **24.000 celdas**, puede mantener hasta 200 símbolos y gestiona sus propias tablas de cadenas y números reales. La implementación está escrita en **Prospero Pro Pascal** y compilada directamente para el procesador 68008.

El QL fue una máquina adelantada a su tiempo. Aunque nunca llegó a dominar el mercado doméstico, construyó una comunidad fiel de programadores que vieron en él una plataforma seria para proyectos ambiciosos. MYLISP es uno de esos proyectos.

1.2.2 El ZX Spectrum Next

El ZX Spectrum Next es una reimaginación moderna del ZX Spectrum original lanzada mediante crowdfunding en 2017. Monta un procesador Z80 mejorado capaz de correr a 3,5, 7 o **28 MHz**, tiene **2 MB de RAM** organizados en páginas de 8 KB, y corre **NextZXOS** como sistema operativo, que incorpora un sistema de ficheros compatible con FAT (esxdos) accesible desde tarjeta SD.

MYLISP/Next está escrito en **C con el compilador Z88DK** y explota la memoria paginada del Next para ofrecer un heap de **32.000 celdas** — más grande que en el QL — sin consumir memoria del espacio de direcciones base del Z80 (que solo dispone de 64 KB planos). Los ficheros se almacenan en tarjeta SD y se nombran con rutas al estilo Unix: `cas.lsp`, `utiles.lsp`, etc.

La diferencia arquitectónica más importante entre ambas versiones es el **evaluador**. La versión QL usa un evaluador recursivo clásico: cada llamada a una función LISP añade un marco a la pila de llamadas de Pascal. La versión Next usa un **evaluador iterativo** con una pila de tareas explícita almacenada en memoria paginada; la recursión de usuario no consume pila de CPU en absoluto, lo que permite recursiones de varios cientos de niveles sin problemas.

1.2.3 Convención usada en este libro

Cuando el comportamiento de MYLISP es idéntico en ambas plataformas —que es la mayoría de los casos— el texto no distingue entre ellas. Cuando hay una diferencia relevante para el programador, aparece una nota así:

QL: comportamiento o límite en el Sinclair QL. **Next:** comportamiento o límite en el ZX Spectrum Next.

Los ejemplos de código son válidos en ambas plataformas salvo que se indique lo contrario.

1.3 Qué es MYLISP

MYLISP es un intérprete del lenguaje LISP disponible en dos implementaciones: **MYLISP/QL**, escrito en Prospero Pro Pascal para el Sinclair QL, y **MYLISP/Next**, escrito en C con Z88DK para el ZX Spectrum Next. En ambos casos el ejecutable es autónomo y arranca directamente desde el medio de almacenamiento de la plataforma.

Su objetivo principal es servir de **motor para un Sistema de Álgebra Computacional** (CAS, *Computer Algebra System*). Un CAS es un programa capaz de manipular expresiones matemáticas de forma simbólica: puede derivar, simplificar, expandir o factorizar expresiones sin convertirlas nunca a números decimales. La fuerza de LISP para estas tareas reside precisamente en que una expresión como `(+ x (* 2 x))` es, en LISP, un dato ordinario —una lista— que puede ser inspeccionado y transformado por cualquier función.

MYLISP sigue el modelo de **Lisp-1**: funciones y variables comparten el mismo espacio de nombres. Esto lo acerca a Scheme más que a Common LISP, y simplifica considerablemente el modelo mental necesario para programar en él.

Las características principales de MYLISP son:

- **Aritmética exacta con enteros y racionales.** Los enteros no se convierten silenciosamente a coma flotante cuando superan un límite. En su lugar, el sistema produce un error explícito. Las fracciones como `1/2` o `3/7` son un tipo de dato nativo y se reducen automáticamente por el máximo común divisor.
- **Evaluador recursivo completo.** MYLISP implementa las formas especiales fundamentales de LISP: `QUOTE`, `IF`, `COND`, `AND`, `OR`, `PROGN`, `LET`, `LAMBDA`, `DEFINE` y `DEFUN`.
- **Clausuras léxicas.** Las funciones capturan el entorno en el que fueron definidas, lo que permite técnicas avanzadas de programación funcional.
- **Recolector de basura automático.** Utiliza el algoritmo *mark-and-sweep*. El usuario nunca necesita gestionar la memoria manualmente.

- **Carga de programas desde almacenamiento.** La forma especial `LOAD` permite cargar y ejecutar ficheros de código: desde los microdrive del QL o desde la tarjeta SD del Next.

1.4 El dialecto MYLISP: posición en la familia LISP

MYLISP es un híbrido fascinante: tiene el **motor arquitectónico de Scheme** —Lisp-1 y ámbito léxico— pero viste la **ropa sintáctica de MacLISP y Common LISP**: `T` / `NIL`, `DEFUN`, `COND`, `PROGN`. Quien llegue de Scheme notará la sintaxis familiar de Common LISP; quien llegue de Common LISP echará de menos muchas primitivas. Este apartado mapea exactamente dónde está MYLISP respecto a ambos estándares, para que no haya sorpresas.

1.4.1 Lo que MYLISP comparte con Scheme (y no con Common LISP)

El parentesco con Scheme es estructural: un único espacio de nombres para funciones y variables (Lisp-1), y **ámbito léxico** —las funciones capturan el entorno en el que fueron definidas, no en el que son llamadas. Esto es lo que hace posibles las clausuras del capítulo 7 y la programación funcional de alto orden del capítulo 9. Common LISP también tiene ámbito léxico por defecto, pero su herencia de MacLISP añade complejidades que Scheme —y MYLISP— evitan deliberadamente.

Además, MYLISP es funcionalmente **más puro que el propio Scheme** en un aspecto concreto: no existen primitivas destructivas. Scheme incluye `set!`, `set-car!` y `set-cdr!` para mutar variables y listas en su sitio. En MYLISP una vez que un valor está ligado a un símbolo, la única forma de cambiarlo es redefinirlo globalmente con `DEFINE`. No hay forma de modificar una lista existente; todas las operaciones de lista construyen nuevas estructuras. Esto no es una limitación accidental: es una decisión de diseño que favorece la claridad en un contexto educativo y elimina una clase entera de errores.

1.4.2 Lo que MYLISP toma de MacLISP y Common LISP

La **sintaxis superficial** de MYLISP es de la rama MacLISP. En Scheme los booleanos son `#t` y `#f`, y la lista vacía `()` es un objeto distinto del booleano falso. En MYLISP, como en Common LISP, `NIL` hace las dos funciones a la vez: es la lista vacía y el valor falso. `T` es el valor verdadero canónico, aunque cualquier valor no-`NIL` se considera verdadero en una condición. Los predicados de Scheme como `boolean?` no existen en MYLISP.

Del mismo modo, los nombres de las formas especiales siguen la tradición MacLISP: `DEFUN` en lugar del `define` de Scheme, `COND` en lugar de `cond` con sintaxis diferente, `PROGN` en lugar de `begin`.

1.4.3 Características de Scheme ausentes en MYLISP

Hay tres características estructurales de Scheme que MYLISP no implementa, por diseño o por las limitaciones del hardware objetivo:

Continuaciones (`call-with-current-continuation` o `call/cc`). Es la primitiva más poderosa y peculiar de Scheme: permite capturar el estado exacto de la ejecución y volver a él más tarde. Con continuaciones se pueden implementar excepciones, corutinas y generadores dentro del propio lenguaje. MYLISP no las implementa; en su lugar, `LOAD` cumple un papel análogo al nivel de sesión: permite cargar desde microdrive cualquier conjunto de definiciones y datos que reconstituyan exactamente el estado de trabajo deseado, al estilo de un fichero de lotes. Es un mecanismo más sencillo y apropiado para el entorno del QL.

Macros higiénicas (`define-syntax` / `syntax-rules`). Scheme permite redefinir el propio lenguaje añadiendo nuevas formas sintácticas de forma segura, sin riesgo de colisión de variables entre el macro y el código que lo usa. MYLISP no tiene sistema de macros; el lenguaje está fijo en las formas especiales que implementa el intérprete.

Optimización de llamadas de cola (TCO, *Tail Call Optimization*). El estándar R5RS exige que Scheme optimice las llamadas en posición de cola para que no consuman pila. MYLISP no implementa TCO en ninguna de sus versiones.

QL: cada llamada recursiva añade un marco a la pila de Pascal. Las recursiones muy profundas pueden agotar la pila; en la práctica los límites de entero y de heap suelen aparecer antes. **Next:** el evaluador es iterativo y la recursión de usuario no consume pila de CPU. El límite práctico de profundidad recursiva en el Next no es la pila sino el heap de 32.000 celdas — mucho más generoso para programas funcionales puros.

1.4.4 Variantes de LET ausentes

MYLISP tiene `LET` con vinculación paralela (todos los valores se evalúan en el entorno exterior antes de que ninguna nueva variable sea visible). Scheme además define `let*` —vinculación secuencial, donde cada variable puede ver las anteriores— y `letrec` —necesario para definir clausuras mutuamente recursivas locales. En MYLISP, `let*` se simula anidando `LET`s, y las funciones mutuamente recursivas se definen con `DEFINE` global (que funciona perfectamente porque las definiciones globales son visibles en el momento de la llamada, no de la definición).

1.4.5 Enteros: precisión fija vs. Bignums

Scheme (a partir de R5RS) exige precisión aritmética exacta ilimitada: los enteros pueden ser tan grandes como la memoria permita. MYLISP, en cambio, trabaja con enteros de precisión fija de 32 bits con signo (rango $\pm 2.147.483.647$) y produce un error explícito en lugar de convertir silenciosamente a coma flotante. Esta es una decisión de diseño que prioriza la **corrección algebraica** sobre el rango numérico: para el CAS simbólico, un entero que "miente" convirtiéndose en float silenciosamente sería peor que un error visible.

1.4.6 Primitivas ausentes que pueden definirse en LISP puro

MYLISP no implementa como primitivas: `ABS`, `MAX`, `MIN`, `SQRT`, `EXPT`, `REVERSE`, `LENGTH`, `MAP`, `FILTER`, `REDUCE` y `APPLY`. Tampoco las abreviaturas de Common LISP `CADR`, `CADDR`, etc., ni los predicados de búsqueda `ASSOC` y `MEMBER`. Todas ellas pueden definirse en unas pocas líneas de LISP puro; el apéndice A.6 y el capítulo 9 muestran cómo hacerlo.

Nota: Si en versiones futuras del intérprete se añaden primitivas que hoy hay que definir manualmente (como `EXPT` o `ABS`), este manual se actualizará para reflejarlo.

1.4.7 La sintaxis que MYLISP comparte con ambos

Independientemente de las diferencias, MYLISP y los dos estándares comparten lo esencial: la notación prefija con paréntesis, la estructura de S-expresión, `LAMBDA` para crear funciones, `QUOTE` o `'` para suprimir la evaluación, `CONS` / `CAR` / `CDR` para trabajar con pares, y la regla fundamental de que cualquier valor distinto de `NIL` (o `#f` en Scheme) es verdadero.

1.4.8 Nombres de símbolo limitados a 8 caracteres

El espacio de nombres de MYLISP almacena los nombres de símbolo en registros de exactamente **8 caracteres**. Cualquier nombre más largo queda **silenciosamente truncado**. Esta limitación no existe en ningún dialecto estándar de LISP y es característica exclusiva del entorno del Sinclair QL y el Next lo hereda.

```
MYLISP> (DEFINE FACTORIAL (LAMBDA (N) ...))
```

El nombre `FACTORIAL` tiene 9 caracteres. MYLISP lo almacena como `FACTORIA`. Si después intentamos llamar a `FACTORIAL`, el intérprete no lo encontrará —a menos que escribamos `FACTORIA`:

```
MYLISP> (FACTORIA 5)
120
```

La regla práctica es: **usar siempre nombres de 8 caracteres o menos**. En los ejemplos de este manual se seguirá esta convención estrictamente.

1.4.9 Salida del intérprete: BYE

Para terminar una sesión de MYLISP se escribe:

```
MYLISP> BYE
```

Sin paréntesis. `BYE` no es una función; es una palabra clave especial del REPL. Si escribiéramos `(BYE)`, el intérprete intentaría evaluar `BYE` como una función y produciría un error de símbolo no definido.

1.4.10 `=` y `EQUAL` no son lo mismo

En MYLISP hay dos predicados de igualdad para números:

- `=` compara numéricamente, convirtiendo los operandos a coma flotante. Por eso `(= 1/2 0.5)` devuelve `T`.
- `EQUAL` compara estructuralmente, tipo por tipo. `(EQUAL 1/2 0.5)` devuelve `NIL` porque uno es racional y el otro real.

Veremos esto en detalle en el capítulo 4.

1.5 MYLISP como herramienta para álgebra simbólica

Una de las razones de existir de MYLISP es demostrar que un ordenador de 8 bits —o casi: el 68008 tiene bus de datos de 8 bits aunque arquitectura interna de 32— puede realizar álgebra simbólica sin recurrir a la coma flotante.

Consideremos el problema de derivar la expresión $x^2 + 2x + 1$ respecto a x . En un lenguaje convencional, esto requeriría representar la expresión de alguna forma estructurada (un árbol sintáctico, quizás) y recorrerlo aplicando reglas de derivación. En LISP, la expresión matemática $x^2 + 2x + 1$ puede representarse directamente como la lista:

```
(+ (* x x) (* 2 x) 1)
```

Y esa lista es un dato ordinario. Una función `DERIV` en LISP puede inspeccionarla, reconocer que el primer elemento es `+` y aplicar la regla de que la derivada de una suma es la suma de las derivadas:

```
(DEFINE DERIV
  (LAMBDA (EXP VAR)
    (COND
      ((EQUAL EXP VAR) 1)
      ((ATOM EXP) 0)
      ((EQUAL (CAR EXP) (QUOTE +))
       (LIST (QUOTE +)
              (DERIV (CADR EXP) VAR)
              (DERIV (CADDR EXP) VAR)))
      ...)))
```

El capítulo 10 desarrolla un derivador simbólico completo paso a paso, empezando desde los conceptos más básicos. Pero el ejemplo anterior ilustra ya la idea clave: en LISP, **el código y los datos tienen la misma estructura**, y eso hace que la manipulación simbólica sea natural.

1.6 Estructura del manual

Este manual está pensado para leerse capítulo a capítulo, aunque cada uno es suficientemente independiente como para consultarlo de forma aislada.

Capítulo 2 te pone en marcha: cómo arrancar MYLISP en el QL, cómo funciona el REPL y cómo introducir tus primeras expresiones. También explica los errores más comunes de los primeros minutos.

Capítulo 3 explica el modelo mental fundamental: qué son las S-expresiones, los átomos, las listas y las reglas de evaluación. Este capítulo es la base de todo lo demás.

Capítulo 4 se adentra en los números: enteros, racionales, reales, los operadores aritméticos y la filosofía de la aritmética exacta que distingue a MYLISP.

Capítulo 5 cubre el trabajo con listas: `CAR`, `CDR`, `CONS`, `LIST`, `APPEND` y los predicados de igualdad. Las listas son el alma de LISP.

Capítulo 6 explica el control de flujo: `IF`, `COND`, `AND`, `OR`, `NOT` y `PROGN`.

Capítulo 7 muestra cómo definir tus propias funciones con `LAMBDA`, `DEFINE` y `DEFUN`, introduce las variables locales con `LET`, explica la recursión y dedica una sección completa a las clausuras léxicas.

Capítulo 8 cubre las cadenas de texto, la función `PRINT` y la carga de programas desde microdrive con `LOAD`.

Capítulo 9 es una galería de programas completos: factorial, Fibonacci, búsqueda, ordenación y la implementación en LISP puro de `MAP`, `FILTER` y `REDUCE`.

Los **apéndices** recogen la referencia rápida de todas las funciones, la arquitectura interna y los límites del intérprete, y la lista completa de mensajes de error.

1.7 Convenciones tipográficas

A lo largo del manual se utilizan las siguientes convenciones:

El texto en `letra monoespaciada` representa código LISP o comandos que se deben escribir literalmente. Las sesiones interactivas se muestran con el prompt del intérprete:

```
MYLISP> (+ 1 2)
3
```

La primera línea es lo que escribe el usuario. La segunda es la respuesta del intérprete.

Los comentarios en el código LISP van precedidos de punto y coma:

```
(+ 1 2) ; esto suma 1 más 2
```

Cuando un ejemplo incluye una definición que se usará más adelante, se indica con un encabezado:

```
; --- Definición ---
(DEFINE CUADRA (LAMBDA (X) (* X X)))

; --- Uso ---
MYLISP> (CUADRA 5)
25
```

Los recuadros de **¡Atención!** señalan limitaciones o comportamientos que difieren de otros dialectos LISP y que pueden causar confusión.

¡Atención! Los nombres de símbolo se truncan a 8 caracteres. `FACTORIAL` se convierte en `FACTORIA`.

Los recuadros de **Nota** aportan información complementaria no esencial para seguir el manual.

Nota: MYLISP es un Lisp-1, como Scheme. Las funciones y las variables comparten el mismo espacio de nombres.

Con todo esto en mente, estás listo para empezar. En el capítulo siguiente pondremos en marcha el intérprete en el Sinclair QL y escribiremos nuestras primeras expresiones.

Continúa en el Capítulo 2: Primeros pasos en el Sinclair QL

Capítulo 2 — Primeros pasos

2.1 Lo que necesitas antes de empezar

2.1.1 En el Sinclair QL

- Un Sinclair QL con al menos **640 KB de RAM**. La configuración base de 128 KB no es suficiente. Existen expansiones de memoria de diferentes fabricantes (Trump Card, Miracle Systems, etc.) que amplían el QL a 640 KB o más.
- Un **microdrive** con el fichero ejecutable de MYLISP. El fichero se llama, por convención, `MYLISP` y reside en la unidad `mdv1_` o `mdv2_` según dónde lo hayas copiado.
- Un monitor o televisor conectado al QL.

Si no dispones de hardware real, existen emuladores de Sinclair QL para PC, Mac y Linux (como **QemuLator** o **sQLux**) que permiten ejecutar MYLISP con total fidelidad. En los emuladores, los microdrive se emulan como imágenes de disco en el sistema anfitrión; consulta la documentación del emulador para saber cómo transferir el fichero MYLISP a la imagen.

2.1.2 En el ZX Spectrum Next

- Un ZX Spectrum Next (cualquier edición: Issue 2, 2A o 2B) con **NextZXOS** actualizado. MYLISP/Next no requiere expansión de RAM adicional: la memoria paginada del Next es más que suficiente.
- La **tarjeta SD** del Next con el fichero `MYLISP.BIN` (o el nombre que le hayas dado) copiado en la raíz o en cualquier carpeta.

Si no dispones de hardware real, el emulador **CSpect** (Windows/Mac/Linux) emula el Next con alta fidelidad incluyendo la tarjeta SD. Es la opción recomendada para desarrollo y pruebas.

2.2 Arrancar MYLISP

2.2.1 En el Sinclair QL

Enciende el QL. Verás el menú de arranque de QDOS. Para lanzar MYLISP desde SuperBASIC, escribe en el prompt de BASIC:

```
EXEC_W mdv1_MYLISP
```

Si el ejecutable está en el segundo microdrive:

```
EXEC_w mdv2_MYLISP
```

En unos instantes la pantalla se limpiará y aparecerá el mensaje de bienvenida del intérprete, seguido del prompt:

```
MYLISP - Interprete LISP para QL
Prospero Pro Pascal

>
```

¡Atención! Si el QL no tiene suficiente memoria, MYLISP puede no arrancar o comportarse de forma errática. Verifica que la expansión de RAM está correctamente instalada y reconocida por QDOS antes de lanzar el intérprete.

2.2.2 En el ZX Spectrum Next

Desde el navegador de NextZXOS, navega hasta el fichero `MYLISP.NEX` y púlsalo para ejecutarlo.

El punto inicial es el prefijo de NextZXOS para ejecutar programas del sistema de ficheros. En unos instantes aparecerá el prompt:

```
MYLISP - Interprete LISP para NEXT
Version 1.0

>
```

El símbolo `>` es el **prompt del REPL**. Indica que el intérprete está esperando que escribas una expresión. Es idéntico en ambas plataformas.

2.3 El REPL: leer, evaluar, imprimir

MYLISP funciona en modo interactivo mediante un **REPL** (*Read-Eval-Print Loop*, bucle de lectura-evaluación-impresión). El ciclo es el siguiente:

1. **Leer:** el intérprete espera a que el usuario escriba una expresión y pulse ENTER.
2. **Evaluar:** MYLISP evalúa la expresión según las reglas del lenguaje.
3. **Imprimir:** el resultado se muestra en pantalla.
4. **Repetir:** el prompt vuelve a aparecer.

Prueba la expresión más sencilla posible: sumar dos números.

```
MYLISP> (+ 1 2)
3
MYLISP>
```

MYLISP evalúa `(+ 1 2)` y muestra `3`. Observa la notación: el operador `+` va **antes** de los operandos, todo dentro de paréntesis. Esto se llama **notación prefija** o **notación polaca**. Es la sintaxis universal de LISP.

Otro ejemplo, una multiplicación:

```
MYLISP> (* 6 7)
42
MYLISP>
```

Y una expresión anidada:

```
MYLISP> (+ (* 2 3) (* 4 5))
26
MYLISP>
```

MYLISP evalúa primero `(* 2 3) = 6` y `(* 4 5) = 20`, y luego suma ambos resultados: 26.

2.4 Límites del REPL

El REPL de MYLISP tiene algunas restricciones que debes conocer. El límite más importante varía entre plataformas.

2.4.1 Límite de acumulación de líneas

MYLISP permite escribir expresiones en **varias líneas**. Si abres un paréntesis y pulsas ENTER sin cerrarlo, el intérprete acumula la entrada y espera más:

```
MYLISP> (DEFINE CUADRA
  (LAMBDA (X)
    (* X X)))
```

La longitud total acumulada de la expresión tiene un límite que depende de la plataforma:

QL: el límite es **80 caracteres** en total (suma de todas las líneas de una expresión multilínea). Si se supera, el intérprete muestra `ERROR: expresion demasiado larga`. Las expresiones más largas deben guardarse en fichero y cargarse con `LOAD`. **Next:** el límite es **500 caracteres**, tanto en el REPL como al cargar con `LOAD`. Esto permite escribir funciones más largas directamente en el prompt.

En ambas plataformas, cuando el buffer se llena el intérprete muestra el error y descarta la expresión incompleta. Los programas de más de unas pocas líneas deben guardarse en fichero y cargarse con `LOAD` en cualquier caso.

2.5 Tus primeras expresiones

Vamos a explorar las capacidades básicas del intérprete con una serie de ejemplos progresivos.

2.5.1 Aritmética básica

```
MYLISP> (+ 10 20 30)
60
```

Los operadores aritméticos de MYLISP son **variadicicos**: aceptan cualquier número de argumentos.

```
MYLISP> (* 2 3 4 5)
120
```

```
MYLISP> (- 100 25 10)
65
```

La resta con más de dos argumentos resta todos los siguientes al primero: `100 - 25 - 10 = 65`.

```
MYLISP> (/ 10 2)
5
```

```
MYLISP> (/ 1 3)
1/3
```

La división entre enteros produce un **racional** cuando no es exacta. El resultado `1/3` no es una aproximación decimal; es la fracción exacta un tercio.

2.5.2 Definir una variable

```
MYLISP> (DEFINE PI 3.14159)
PI
MYLISP> (* PI (* 5 5))
78.53975
```

`DEFINE` asocia un símbolo con un valor. Después de la definición, `PI` evalúa al número `3.14159`.

2.5.3 Definir una función sencilla

Nota: Los ejemplos que siguen usan `LAMBDA`, `CONS`, `CAR`, `CDR` y `NIL`. No te preocupes si la sintaxis no queda del todo clara ahora — el Capítulo 3 despiece exactamente cómo funciona cada pieza. Por ahora, copia los comandos, observa el resultado y quédate con la intuición general de lo que hacen.

```
MYLISP> (DEFINE CUADRA (LAMBDA (X) (* X X)))
CUADRA
MYLISP> (CUADRA 7)
49
MYLISP> (CUADRA 12)
144
```

`LAMBDA` crea una función anónima. `DEFINE` le da nombre. Esto se verá en detalle en el capítulo 7; por ahora, basta con entender el patrón.

2.5.4 Trabajar con listas

```
MYLISP> (CONS 1 (CONS 2 (CONS 3 NIL)))
(1 2 3)
```

`CONS` construye pares. Una cadena de `CONS` terminada en `NIL` (la lista vacía) forma una lista. No te preocupes por el `NIL` de momento: el Capítulo 3 explica exactamente qué es y por qué aparece aquí.

```
MYLISP> (LIST 10 20 30)
(10 20 30)
```

`LIST` es una abreviatura más cómoda para construir listas.

```
MYLISP> (CAR (LIST 10 20 30))
10
MYLISP> (CDR (LIST 10 20 30))
(20 30)
```

`CAR` devuelve el primer elemento. `CDR` devuelve el resto.

2.6 Errores comunes y cómo interpretarlos

En los primeros minutos con el intérprete es habitual cometer ciertos errores. Aquí están los más frecuentes y lo que significan.

2.6.1 Símbolo no definido

```
MYLISP> FACTORIAL
ERROR: simbolo no definido: FACTORIA
```

Ocurre cuando escribes un símbolo que no ha sido definido previamente. Fíjate en que MYLISP muestra `FACTORIA`, no `FACTORIAL` — porque ha truncado el nombre a 8 caracteres antes de buscarlo.

2.6.2 Paréntesis sin cerrar

```
MYLISP> (+ 1 2
```

Si pulsas ENTER con paréntesis abiertos, el intérprete espera más entrada. El prompt cambia (aparece ..) para indicar que la expresión está incompleta. Escribe el paréntesis de cierre y pulsa ENTER:

```
MYLISP> (+ 1 2
..      )
3
```

2.6.3 División por cero

```
MYLISP> (/ 5 0)
ERROR: division por cero
```

2.6.4 Aplicar CAR a NIL

```
MYLISP> (CAR NIL)
ERROR: CAR requiere una lista no vacia
```

2.6.5 Desbordamiento de entero

```
MYLISP> (* 1000000 1000000)
ERROR: desbordamiento de entero.
Use numeros reales (ej. 1.0) si necesita rangos mayores.
```

MYLISP no convierte silenciosamente los enteros a coma flotante. Si el resultado no cabe en un entero, produce este error. Para trabajar con números grandes, usa literales reales:

```
MYLISP> (* 1000000.0 1000000.0)
1.0E12
```

2.7 Cómo funciona la memoria interna

MYLISP gestiona la memoria mediante una **tabla de celdas** (*heap*). Cada celda puede almacenar un par de valores (la unidad básica de LISP, llamada *cons cell*). Las listas, los árboles, las clausuras y los entornos de evaluación están todos contruidos a partir de estas celdas.

QL: el heap tiene **24.000 celdas**. **Next:** el heap tiene **32.000 celdas**, almacenadas en memoria paginada fuera del espacio de 64 KB del Z80.

Además del heap, el intérprete mantiene una tabla de símbolos (200 entradas, nombres truncados a 8 caracteres) y una tabla de cadenas (50 entradas, máximo 36 caracteres). La versión QL tiene adicionalmente una tabla de reales para hasta 100 números en coma flotante; en la versión Next los flotantes van empaquetados directamente dentro de cada celda.

Cuando el heap se llena, el intérprete ejecuta automáticamente el **recolector de basura** (*garbage collector*, GC), usando el algoritmo *mark-and-sweep*. El GC es completamente silencioso; el usuario no lo nota. El umbral de disparo varía:

QL: 80% en el REPL y 50% durante `LOAD`. **Next:** 80% en el REPL. Durante `LOAD`, el GC **no se dispara automáticamente**: el heap de 32.000 celdas es generoso y se confía en que un solo fichero no lo agotará. Si quieres forzar una recolección entre cargas, usa `(CLEAN)` desde el REPL —no desde dentro de un fichero— entre una carga y la siguiente.

Nota: El GC nunca se activa en mitad de una evaluación, solo entre expresiones completas. Esto garantiza que el recolector no corrompe estructuras que estén siendo construidas en ese momento.

2.8 Salir del intérprete

Para terminar la sesión, escribe:

```
MYLISP> BYE
```

Sin paréntesis. El control regresa al sistema operativo QDOS.

¡Atención! No escribas `(BYE)`. MYLISP intentará evaluar `BYE` como una función y producirá el error `ERROR: símbolo no definido: BYE`.

BYE borra todo — sin excepción: Al escribir `BYE`, MYLISP destruye completamente la memoria de la sesión. Todas las funciones definidas en el REPL, todas las variables, todo el trabajo acumulado desaparece sin posibilidad de recuperación. MYLISP no guarda el estado automáticamente. La única forma de preservar el trabajo es **escribirlo siempre en un fichero** y cargarlo con `LOAD` al inicio de cada sesión. El REPL es para probar; los ficheros son para guardar. Adopta este hábito desde el primer día.

2.9 Sesión de ejemplo completa

Para terminar este capítulo, aquí tienes una sesión completa que muestra el flujo de trabajo típico con MYLISP: definir algunas funciones, probarlas y salir.

```
MYLISP - Interprete LISP para Sinclair QL
Prospero Pro Pascal

MYLISP> (DEFINE CUADRA (LAMBDA (X) (* X X)))
CUADRA
MYLISP> (DEFINE CUBO (LAMBDA (X) (* X X X)))
CUBO
MYLISP> (CUADRA 5)
25
MYLISP> (CUBO 3)
27
MYLISP> (+ (CUADRA 3) (CUADRA 4))
25
MYLISP> (DEFINE HIPOTENUSA
          (LAMBDA (A B)
            (CUADRA (+ (CUADRA A) (CUADRA B)))))
HIPOTENUSA
MYLISP> (HIPOTENUSA 3 4)
625
```

Espera — `625` no es la hipotenusa del triángulo 3-4-5. ¡Hemos elevado al cuadrado dos veces! La función correcta debería usar la raíz cuadrada. Pero `SQRT` no está implementada en MYLISP. Para el CAS esto no es un problema —trabajamos con expresiones simbólicas exactas, no con aproximaciones decimales— pero si necesitas la raíz cuadrada numérica, tendrás que aproximarla con el método de Newton, que veremos en el capítulo 9.

La sesión correcta habría sido:

```
MYLISP> (DEFINE HIPOT2
          (LAMBDA (A B)
            (+ (* A A) (* B B))))
HIPOT2
MYLISP> (HIPOT2 3 4)
25
```

25 es el cuadrado de la hipotenusa. La hipotenusa es $\sqrt{25} = 5$, pero eso lo sabemos nosotros; el intérprete nos da el valor exacto sin aproximaciones.

```
MYLISP> BYE
```

Continúa en el Capítulo 3: El modelo de datos de LISP

Capítulo 3 — El modelo de datos de LISP

3.1 Todo es una S-expresión

En LISP, **todo** —tanto los datos como el código— tiene la misma forma: la **S-expresión** (*Symbolic Expression*, expresión simbólica). Este concepto, introducido por McCarthy en 1958, es la idea más importante de todo el lenguaje. Quien lo entiende bien, entiende LISP.

Una S-expresión es, recursivamente, una de estas dos cosas:

1. Un **átomo**: la unidad indivisible de datos.
2. Un **par**: dos S-expresiones unidas, escrito como `(A . B)`.

A partir de pares anidados se construyen las **listas**, que son la estructura de datos fundamental de LISP.

3.2 Átomos

Un átomo es cualquier valor que no puede dividirse en partes más pequeñas. En MYLISP hay cinco tipos de átomo:

Símbolos. Son identificadores que representan nombres. Se usan para nombrar variables, funciones y constantes.

```
X  
SUMA  
CUADRA  
NIL  
T
```

Enteros. Números enteros sin parte decimal.

```
0  
42  
-17  
1000
```

Racionales. Fracciones exactas representadas como numerador/denominador.

```
1/2  
3/7  
-5/3
```

MYLISP reduce automáticamente las fracciones: `(/ 4 6)` devuelve `2/3`, no `4/6`.

Reales. Números en coma flotante (aproximados).

```
3.14159
-2.718
1.0E10
```

Cadenas. Texto entre comillas dobles (limitado a 36 caracteres).

```
"hola"
"resultado:"
"MYLISP 1.0"
```

Los dos átomos más especiales son `NIL` y `T`:

- `NIL` representa la **lista vacía** y el valor **falso**. Es el único valor que LISP considera falso en las condiciones.
- `T` representa el valor **verdadero**. Sin embargo, en MYLISP, **cualquier valor distinto de `NIL` es verdadero** en un contexto condicional.

3.3 Pares y listas

El par `(A . B)` —llamado *cons cell* o *par punteado*— es la unidad de construcción de las listas. El elemento izquierdo se llama `CAR` y el derecho `CDR` (nombres heredados del IBM 704, la máquina en que se implementó el primer LISP).

Una lista `(1 2 3)` es, en realidad, una cadena de pares anidados:

```
(1 . (2 . (3 . NIL)))
```

Que visualmente se representa así:

```
[1 | •]—>[2 | •]—>[3 | NIL]
```

Cada caja representa una cons cell. El lado izquierdo contiene el valor `(1, 2, 3)` y el lado derecho apunta a la siguiente celda. La última celda apunta a `NIL`, indicando el fin de la lista.

Cuando MYLISP imprime una lista, muestra la notación abreviada `(1 2 3)` en lugar de `(1 . (2 . (3 . NIL)))`. Pero internamente, ambas representaciones son idénticas.

Podemos construir pares y listas explícitamente con `CONS`:

```
MYLISP> (CONS 1 NIL)
(1)
MYLISP> (CONS 1 (CONS 2 NIL))
(1 2)
MYLISP> (CONS 1 (CONS 2 (CONS 3 NIL)))
(1 2 3)
```

O de forma más cómoda con `LIST`:

```
MYLISP> (LIST 1 2 3)
(1 2 3)
```

3.3.1 Listas anidadas

Las listas pueden contener otras listas como elementos:


```
MYLISP> (LIST (LIST 1 2) (LIST 3 4))  
((1 2) (3 4))
```

Internamente, esto es:

```
((1 2) . ((3 4) . NIL))
```

Una lista anidada es simplemente un par cuyo `CAR` es otra lista.

3.3.2 La lista vacía

`NIL` y `()` son la misma cosa: la lista vacía.

```
MYLISP> NIL  
NIL  
MYLISP> (LIST)  
NIL
```

3.4 Cómo MYLISP evalúa las expresiones

Cuando escribes una expresión en el REPL, MYLISP sigue un conjunto de reglas para evaluarla. Estas reglas son simples pero poderosas.

3.4.1 Regla 1: Los átomos auto-evaluantes

Los literales numéricos y las cadenas evalúan a sí mismos:

```
MYLISP> 42  
42  
MYLISP> 3.14  
3.14  
MYLISP> 1/3  
1/3  
MYLISP> "hola"  
"hola"
```

`NIL` y `T` también son auto-evaluantes:

```
MYLISP> NIL  
NIL  
MYLISP> T  
T
```

3.4.2 Regla 2: Los símbolos buscan su valor en el entorno

Un símbolo evalúa al valor que tiene asociado en el entorno actual. Si no tiene valor, es un error.

```
MYLISP> (DEFINE X 10)  
X  
MYLISP> X  
10  
MYLISP> Y  
ERROR: simbolo no definido: Y
```

3.4.3 Regla 3: Las listas son llamadas a función (o formas especiales)

Una lista `(F A B C ...)` se evalúa evaluando primero `F`, luego los argumentos `A`, `B`, `C`... y finalmente aplicando la función resultante a los argumentos evaluados.

```
MYLISP> (+ 1 2)
```

1. Se evalúa `+` → la función suma primitiva.
2. Se evalúa `1` → `1`.
3. Se evalúa `2` → `2`.
4. Se aplica la suma: `1 + 2 = 3`.

```
MYLISP> (+ (* 2 3) (- 10 4))
```

1. Se evalúa `+` → suma.
2. Se evalúa `(* 2 3)` → `6`.
3. Se evalúa `(- 10 4)` → `6`.
4. Se aplica la suma: `6 + 6 = 12`.

3.4.4 Excepción: las formas especiales no siguen la regla 3

Algunas construcciones del lenguaje —llamadas **formas especiales**— no evalúan todos sus argumentos automáticamente. Por ejemplo, `IF` sólo evalúa la rama que corresponde:

```
MYLISP> (IF T 1 (/ 1 0))
1
```

La expresión `(/ 1 0)` no se evalúa (lo que evitaría el error de división por cero) porque la condición `T` es verdadera y sólo se evalúa la rama "verdadera". Las formas especiales de MYLISP son: `QUOTE`, `IF`, `COND`, `AND`, `OR`, `PROGN`, `LET`, `LAMBDA`, `DEFINE`, `DEFUN` y `LOAD`.

3.5 QUOTE: detener la evaluación

A veces queremos que MYLISP trate una expresión como un **dato** en lugar de evaluarla. Para eso existe `QUOTE`:

```
MYLISP> (QUOTE (+ 1 2))
(+ 1 2)
```

Sin `QUOTE`, `(+ 1 2)` evaluaría a `3`. Con `QUOTE`, devuelve la lista tal cual: los tres elementos `+`, `1` y `2`.

`QUOTE` tiene una forma abreviada: el apóstrofe `'`:

```
MYLISP> '(+ 1 2)
(+ 1 2)
MYLISP> '(a b c)
(A B C)
MYLISP> 'hola
HOLA
```

Fíjate en que los símbolos se muestran en mayúsculas —MYLISP convierte todos los símbolos a mayúsculas internamente.

`QUOTE` es fundamental para construir datos literales:

```
MYLISP> (DEFINE COLORES '(ROJO VERDE AZUL))
COLORES
MYLISP> COLORES
(ROJO VERDE AZUL)
MYLISP> (CAR COLORES)
ROJO
MYLISP> (CDR COLORES)
(VERDE AZUL)
```

Sin `QUOTE`, `(DEFINE COLORES (ROJO VERDE AZUL))` intentaría evaluar `ROJO` como una función, lo que produciría un error.

3.6 El principio de la homoiconicidad

La consecuencia más profunda del modelo de datos de LISP es que **el código tiene la misma estructura que los datos**.

La expresión `(+ 1 2)` es, al mismo tiempo:

- Un programa que calcula `3` cuando se evalúa.
- Una lista de tres elementos `(+, 1, 2)` que puede ser inspeccionada y manipulada como cualquier otra lista.

Esto significa que podemos escribir programas que **generan** y **ejecutan** otros programas. La función `EVAL` aplica las reglas de evaluación a cualquier lista:

```
MYLISP> (EVAL '(+ 1 2))
3
MYLISP> (EVAL (LIST '+ 10 20))
30
```

En el segundo ejemplo, construimos la lista `(+ 10 20)` en tiempo de ejecución con `LIST` y `QUOTE`, y luego la evaluamos con `EVAL`. El resultado es exactamente el mismo que si hubiéramos escrito `(+ 10 20)` directamente.

Esta capacidad —construir código como datos y luego ejecutarlo— es la base del CAS. La expresión matemática $x^2 + 2x + 1$ se representa como la lista `(+ (* x x) (* 2 x) 1)`, y una función de derivación la transforma en `(+ (* 2 x) 2)` aplicando las reglas del cálculo sobre la estructura de la lista.

3.7 Predicados de tipo

MYLISP incluye funciones para preguntar de qué tipo es un valor. Se llaman **predicados** porque devuelven `T` o `NIL`.

```
MYLISP> (ATOM 42)
T
MYLISP> (ATOM '(1 2 3))
NIL
MYLISP> (ATOM NIL)
T
```

`ATOM` devuelve `T` para cualquier átomo, incluyendo `NIL`. Devuelve `NIL` para los pares (listas).

```
MYLISP> (NUMBERP 42)
T
MYLISP> (NUMBERP 3.14)
T
MYLISP> (NUMBERP 1/2)
T
MYLISP> (NUMBERP 'X)
NIL
```

`NUMBERP` devuelve `T` para cualquier número (entero, racional o real).

```
MYLISP> (SYMBOLP 'HOLA)
T
MYLISP> (SYMBOLP 42)
NIL
```

```
MYLISP> (LISTP '(1 2 3))
T
MYLISP> (LISTP NIL)
T
MYLISP> (LISTP 42)
NIL
```

`LISTP` devuelve `T` para listas, incluyendo la lista vacía `NIL`.

```
MYLISP> (NULL NIL)
T
MYLISP> (NULL '(1 2))
NIL
MYLISP> (NULL 0)
NIL
```

`NULL` devuelve `T` únicamente para `NIL`. Nótese que `0` no es `NIL`: en MYLISP, como en todos los dialectos LISP, el único valor falso es `NIL`.

3.8 Igualdad: EQ, =, y EQUAL

MYLISP tiene tres formas de comparar valores, y es importante entender las diferencias.

3.8.1 EQ: identidad de símbolo

`EQ` compara si dos objetos son **exactamente el mismo símbolo** o ambos son `NIL`. Es la comparación más primitiva.

```
MYLISP> (EQ 'A 'A)
T
MYLISP> (EQ 'A 'B)
NIL
MYLISP> (EQ NIL NIL)
T
MYLISP> (EQ '(1 2) '(1 2))
NIL
```

El último ejemplo devuelve `NIL` porque las dos listas `(1 2)`, aunque iguales en contenido, son objetos distintos en memoria. `EQ` comprueba identidad, no igualdad de contenido.

3.8.2 =: igualdad numérica

`=` compara valores numéricos convirtiendo ambos operandos a coma flotante internamente. Por eso puede comparar enteros con reales y con racionales:

```
MYLISP> (= 1 1)
T
MYLISP> (= 1 1.0)
T
MYLISP> (= 1/2 0.5)
T
MYLISP> (= 2 3)
NIL
```

Útil cuando queremos igualdad numérica sin preocuparnos del tipo concreto.

3.8.3 EQUAL: igualdad estructural

`EQUAL` compara recursivamente la estructura y el tipo de dos expresiones. Es la comparación más precisa.

```
MYLISP> (EQUAL '(1 2 3) '(1 2 3))
T
MYLISP> (EQUAL '(1 2) '(1 3))
NIL
MYLISP> (EQUAL 1 1.0)
NIL
MYLISP> (EQUAL 1/2 0.5)
NIL
```

Los dos últimos ejemplos devuelven `NIL` porque, aunque numéricamente iguales, `1` es un entero y `1.0` es un real; `1/2` es un racional y `0.5` es un real. `EQUAL` distingue los tipos.

Resumen: usa `EQ` para comparar símbolos; usa `=` para comparar números sin importar el tipo; usa `EQUAL` para comparar listas o cuando el tipo importa.

3.9 Construir y desmontar listas

Las operaciones fundamentales sobre listas son `CAR`, `CDR` y `CONS`. Todo lo demás se construye a partir de ellas.

3.9.1 CAR y CDR

`CAR` devuelve el primer elemento de una lista (el elemento izquierdo del primer par):

```
MYLISP> (CAR '(10 20 30))
10
MYLISP> (CAR '((A B) C D))
(A B)
```

`CDR` devuelve el resto de la lista (todo excepto el primer elemento):

```
MYLISP> (CDR '(10 20 30))
(20 30)
MYLISP> (CDR '((10)))
NIL
```

Combinando `CAR` y `CDR` podemos acceder a cualquier elemento:

```
MYLISP> (CAR (CDR '(10 20 30)))
20
MYLISP> (CAR (CDR (CDR '(10 20 30))))
30
```

`(CAR (CDR lista))` accede al segundo elemento; `(CAR (CDR (CDR lista)))` al tercero. La combinación es tan común que se abrevia como `CADR` y `CADDR`, aunque en MYLISP estas abreviaturas no están predefinidas como primitivas —tendrás que definir las tú mismo si las necesitas:

```
(DEFINE CADR (LAMBDA (L) (CAR (CDR L))))
(DEFINE CADDR (LAMBDA (L) (CAR (CDR (CDR L)))))
```

3.9.2 CONS

`CONS` construye un nuevo par. Dados un elemento `A` y una lista `L`, devuelve una nueva lista con `A` como primer elemento seguido de los elementos de `L`:

```
MYLISP> (CONS 0 '(1 2 3))
(0 1 2 3)
MYLISP> (CONS 'X '(Y Z))
(X Y Z)
MYLISP> (CONS '(1 2) '(3 4))
((1 2) 3 4)
```

`CONS` no modifica la lista original; crea una nueva estructura. Esta propiedad —la **inmutabilidad** de las estructuras LISP— hace que los programas funcionales sean seguros: compartir estructuras entre variables no produce efectos secundarios.

3.9.3 APPEND

`APPEND` concatena dos listas:

```
MYLISP> (APPEND '(1 2) '(3 4))
(1 2 3 4)
MYLISP> (APPEND '(A B) '(C D) )
```

Nota: En MYLISP, `APPEND` acepta exactamente dos listas. No es variádica.

```
MYLISP> (APPEND '(A B) '(C D))
(A B C D)
```

3.10 Las listas como estructura universal

Las listas son tan flexibles que pueden representar prácticamente cualquier estructura de datos:

Pila (stack): el frente de la lista es la cima.

```
(DEFINE PILA NIL)
(DEFINE PUSH (LAMBDA (ELEM P) (CONS ELEM P)))
(DEFINE POP (LAMBDA (P) (CDR P)))
(DEFINE CIMA (LAMBDA (P) (CAR P)))
```

Par clave-valor (association list o a-list): lista de pares `(clave . valor)`.

```
MYLISP> (DEFINE AGENDA
          '((ANA . 555-1234)
            (LUIS . 555-5678)
            (MARIA . 555-9012)))

AGENDA
```

Árbol: lista de listas anidadas.

```
MYLISP> (DEFINE ARBOL '(RAIZ (IZQ A B) (DER C D)))
ARBOL
MYLISP> (CAR ARBOL)
RAIZ
MYLISP> (CADR ARBOL) ; necesita CADR definida
(IZQ A B)
```

En los próximos capítulos veremos cómo estas estructuras se utilizan en programas reales.

Continúa en el Capítulo 4: Números y aritmética exacta

Capítulo 4 — Números y aritmética exacta

4.1 Los tres tipos numéricos de MYLISP

MYLISP distingue tres tipos de número, y cada uno tiene un comportamiento bien definido. Entender las diferencias es esencial, especialmente si usas MYLISP para álgebra simbólica, donde la exactitud importa más que la velocidad.

4.1.1 Enteros

Son números enteros sin parte decimal, positivos o negativos. En MYLISP los enteros son de **precisión fija de 32 bits con signo**, con rango de $-2.147.483.647$ a $+2.147.483.647$.

```
MYLISP> 0
0
MYLISP> 42
42
MYLISP> -1000
-1000
MYLISP> 32767
32767
```

Si el resultado de una operación excede este rango, MYLISP produce un error en lugar de convertir silenciosamente el valor a coma flotante. Esto es una decisión de diseño deliberada que se explica en la sección 4.4.

4.1.2 Racionales

Un racional es una fracción exacta expresada como `numerador/denominador`. MYLISP acepta racionales como literales directamente:

```
MYLISP> 1/2
1/2
MYLISP> 3/4
3/4
MYLISP> -2/5
-2/5
```

Los racionales se reducen **automáticamente** al máximo común divisor. No es necesario reducirlos manualmente:

```
MYLISP> (/ 4 6)
2/3
MYLISP> (/ 10 5)
2
MYLISP> (+ 1/3 1/6)
1/2
```

Observa que `(/ 10 5)` devuelve el entero `2`, no el racional `2/1`. MYLISP simplifica el resultado al tipo más simple posible.

4.1.3 Reales

Los reales son números en coma flotante de precisión simple. Se escriben con punto decimal o con notación científica:

```
MYLISP> 3.14159
3.14159
MYLISP> -2.718
-2.718
MYLISP> 1.0E10
1.0E10
MYLISP> 0.001
0.001
```

Los reales son aproximados: `(+ 0.1 0.2)` puede no dar exactamente `0.3`. Para cálculo simbólico y algebraico, usa siempre enteros y racionales.

Nota: MYLISP almacena hasta 100 números reales distintos en una tabla interna. Esta tabla se comparte entre todos los valores reales creados durante la sesión. Si necesitas crear muchos valores reales diferentes, puedes agotar esta tabla.

4.2 Operadores aritméticos

Los cuatro operadores básicos son `+`, `-`, `*` y `/`. Todos son **variadicos**: aceptan uno o más argumentos.

4.2.1 Suma

```
MYLISP> (+ 1 2)
3
MYLISP> (+ 1 2 3 4 5)
15
MYLISP> (+ 1/2 1/3)
5/6
MYLISP> (+ 1 0.5)
1.5
```

Cuando los operandos son de tipos mezclados, MYLISP aplica la siguiente jerarquía de promoción: entero → racional → real. El resultado adopta el tipo más "amplio" presente en los operandos.


```
MYLISP> (+ 1 1/2)
3/2
MYLISP> (+ 1/2 0.5)
1.0
```

4.2.2 Resta

Con dos o más argumentos, resta el segundo y siguientes al primero:

```
MYLISP> (- 10 3)
7
MYLISP> (- 10 3 2)
5
```

Con **un solo argumento**, devuelve el negativo (cambio de signo):

```
MYLISP> (- 5)
-5
MYLISP> (- -3)
3
MYLISP> (- 1/4)
-1/4
```

4.2.3 Multiplicación

```
MYLISP> (* 3 4)
12
MYLISP> (* 2 3 4 5)
120
MYLISP> (* 2 1/3)
2/3
MYLISP> (* 1/2 1/3)
1/6
```

4.2.4 División

Con dos o más argumentos, divide el primero entre el segundo y siguientes:

```
MYLISP> (/ 10 2)
5
MYLISP> (/ 1 3)
1/3
MYLISP> (/ 12 4 3)
1
MYLISP> (/ 3.0 2)
1.5
```

Con **un solo argumento**, devuelve el inverso:

```
MYLISP> (/ 2)
1/2
MYLISP> (/ 4)
1/4
MYLISP> (/ 3.0)
0.33333...
```

¡Atención! La división entre enteros siempre produce un racional si no es exacta. Para obtener un resultado real, usa al menos un operando real: `(/ 1.0 3)` devuelve `0.33333...`; `(/ 1 3)` devuelve `1/3`.

4.2.5 División por cero

Cualquier intento de dividir por cero produce un error inmediato:

```
MYLISP> (/ 5 0)
ERROR: division por cero
MYLISP> (/ 1 0)
ERROR: division por cero
```

4.3 Comparaciones numéricas

MYLISP incluye los seis operadores de comparación habituales. Todos devuelven `T` o `NIL`.

```
MYLISP> (= 3 3)
T
MYLISP> (= 3 4)
NIL
MYLISP> (< 2 5)
T
MYLISP> (> 10 3)
T
MYLISP> (<= 4 4)
T
MYLISP> (>= 5 3)
T
```

Todos aceptan mezcla de tipos numéricos, convirtiendo internamente a real para comparar:

```
MYLISP> (< 1/3 0.5)
T
MYLISP> (= 1/2 0.5)
T
```

4.4 La filosofía de la aritmética exacta

La decisión de diseño más llamativa de MYLISP es su **negativa a promover enteros a reales automáticamente**. En la mayoría de los lenguajes, cuando una operación entre enteros produce un resultado fuera de rango, el sistema convierte silenciosamente el valor a coma flotante. En MYLISP no:

```
MYLISP> (* 1000 1000)
ERROR: desbordamiento de entero.
Use numeros reales (ej. 1.0) si necesita rangos mayores.
```

¿Por qué este comportamiento? La razón es la vocación de MYLISP como motor de CAS. En álgebra simbólica, la diferencia entre un entero y un real no es trivial. Considera la fracción `1/3`:

- Como racional: `1/3` (exacto, sin pérdida de información).
- Como real: `0.33333...` (aproximación de precisión finita).

Si MYLISP convirtiera automáticamente `1/3` a `0.33333...`, las operaciones algebraicas posteriores acumularían errores de redondeo y el CAS dejaría de ser fiable. La aritmética exacta es el precio de la corrección algebraica.

¿Qué hacer cuando necesitas números grandes? Si el problema requiere números que excedan el rango de los enteros de 32 bits (más de ± 2.147 millones), usa reales desde el principio. Pero sé consciente de que perderás exactitud:

```
MYLISP> (* 1000.0 1000.0)
1.0E6
```

Para la mayoría de los cálculos de álgebra simbólica (derivadas, simplificaciones, manipulación de polinomios con coeficientes pequeños), los enteros de 32 bits son más que suficientes.

4.5 Racionales: la fracción exacta

Los racionales son quizás la característica más útil de MYLISP para álgebra. Permiten representar cualquier número racional con **exactitud total**, sin aproximaciones.

Veamos cómo se comportan en operaciones:

```
MYLISP> (+ 1/4 1/4)
1/2
MYLISP> (+ 1/3 1/6)
1/2
MYLISP> (* 2/3 3/4)
1/2
MYLISP> (- 1 1/3)
2/3
```

Todas las operaciones entre racionales producen racionales (o enteros cuando el resultado es exacto).

La reducción automática garantiza que el resultado siempre esté en su forma más simple:

```
MYLISP> (+ 1/6 1/6 1/6)
1/2
MYLISP> (* 6/7 7/6)
1
```

Los racionales se comparan correctamente con los operadores numéricos:

```
MYLISP> (< 1/3 1/2)
T
MYLISP> (= 2/4 1/2)
T
```

4.6 La diferencia crucial entre `=` y `EQUAL`

La distinción completa entre `=`, `EQ` y `EQUAL` se explica en la sección 3.8 con ejemplos detallados. En resumen: `=` compara numéricamente (convirtiendo a coma flotante), `EQUAL` compara estructura y tipo, y `EQ` compara identidad de objeto en memoria.

4.7 Operaciones mixtas y la jerarquía de tipos

Cuando una operación mezcla tipos, MYLISP aplica la siguiente regla: el resultado adopta el tipo del operando más "amplio" según la jerarquía:

```
entero < racional < real
```

Algunos ejemplos:

```
MYLISP> (+ 1 1/2)
3/2      ; entero + racional → racional
MYLISP> (+ 1 0.5)
1.5      ; entero + real → real
MYLISP> (+ 1/2 0.5)
1.0      ; racional + real → real
MYLISP> (+ 1 1/2 0.1)
1.6      ; el real domina sobre todo
```

Esta jerarquía refleja la "pérdida de información": una vez que aparece un real, el resultado ya no puede ser exacto.

4.8 Ejemplos prácticos

4.8.1 Cálculo de una media exacta

```
MYLISP> (/ (+ 1 2 3 4 5) 5)
3
```

La media de 1 a 5 es exactamente 3.

```
MYLISP> (/ (+ 1 2 3 4) 4)
5/2
```

La media de 1 a 4 es exactamente $5/2$. No hay redondeo.

4.8.2 Verificar una identidad algebraica

¿Es $(a/b) * (b/a) = 1$ para valores concretos?

```
MYLISP> (* 3/7 7/3)
1
MYLISP> (* 5/11 11/5)
1
```

La aritmética racional exacta confirma la identidad sin margen de error.

4.8.3 Polinomio con coeficientes racionales

Evaluar $P(x) = x^2 + (1/2)x - 3/4$ en $x = 1/2$:

```
(DEFINE P
  (LAMBDA (X)
    (+ (* X X)
      (* 1/2 X)
      (- 3/4))))
```

```
MYLISP> (P 1/2)
```

Calculemos: $(1/2)^2 + (1/2)(1/2) - 3/4 = 1/4 + 1/4 - 3/4 = -1/4$

```
MYLISP> (P 1/2)
-1/4
```

El resultado exacto: $-1/4$. No -0.25 ni $-0.24999\dots$.

4.8.4 Suma de series

Aproximación de $\pi/4$ mediante la serie de Leibniz: $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$

```
MYLISP> (+ 1 (- 1/3) 1/5 (- 1/7) 1/9 (- 1/11))
341/429
```

$341/429 \approx 0.7948\dots$, mientras que $\pi/4 \approx 0.7854$. La aproximación mejora con más términos. La aritmética racional garantiza que la suma de fracciones es exacta.

4.9 División entera: MOD y DIV

Cuando trabajamos con enteros, a veces necesitamos el **cociente** y el **resto** de una división, no el resultado exacto. Para eso MYLISP ofrece dos primitivas específicas que solo operan sobre enteros (`TINT`).

4.9.1 DIV: cociente entero

`DIV` devuelve la parte entera del cociente, truncando hacia cero:

```
MYLISP> (DIV 17 5)
3
MYLISP> (DIV 10 3)
3
MYLISP> (DIV 15 5)
3
MYLISP> (DIV -7 3)
-2
```

Nótese que `(DIV -7 3)` devuelve -2 (trunca hacia cero), no -3 .

4.9.2 MOD: resto entero

`MOD` devuelve el resto de la división entera. El signo del resultado es el mismo que el del dividendo:

```
MYLISP> (MOD 17 5)
2
MYLISP> (MOD 10 3)
1
MYLISP> (MOD 15 5)
0
MYLISP> (MOD -7 3)
-1
```

La relación entre `/`, `DIV` y `MOD` es siempre: $A = (+ (* (DIV A B) B) (MOD A B))$.

4.9.3 DIV y MOD solo aceptan enteros

Si se pasa un racional o un real, se produce un error. Para calcular el resto de una división racional usa `/` directamente:

```
MYLISP> (DIV 1/2 3)
ERROR: ...
MYLISP> (/ 1 2)
1/2 ; esto es lo correcto para racionales
```

4.9.4 Aplicaciones inmediatas

Comprobar si un número es par o impar:

```
(DEFUN PAREP (N) (= 0 (MOD N 2)))
(DEFUN IMPARP (N) (NOT (PAREP N)))
```

```
MYLISP> (PAREP 4)
T
MYLISP> (IMPARP 7)
T
```

Calcular el máximo común divisor (algoritmo de Euclides):

```
(DEFUN MCD (A B)
  (IF (= B 0) A (MCD B (MOD A B))))
```

```
MYLISP> (MCD 48 18)
6
MYLISP> (MCD 100 75)
25
```

Continúa en el Capítulo 5: Trabajar con listas

Capítulo 5 — Trabajar con listas

5.1 Tu primera librería: el fichero de utilidades

Antes de adentrarnos en las operaciones de lista, hay que resolver un problema práctico: MYLISP no incluye como primitivas algunas funciones de conveniencia que usaremos constantemente — `CADR`, `CADDR`, `LONGIT`, `ABSOL` y otras. Cada vez que arrancas MYLISP empiezas con un entorno completamente vacío: las definiciones de la sesión anterior no se

conservan.

La solución es crear un **fichero de utilidades** que cargues al inicio de cada sesión de trabajo. Es la técnica habitual en todos los entornos LISP de recursos limitados: construyes tu propia "biblioteca estándar" y la tienes lista en el medio de almacenamiento.

5.1.1 Crear el fichero de utilidades

El nombre del fichero depende de tu plataforma:

QL: guárdalo en el microdrive como `mdv1_utiles` (usando Quill u otro editor del QL). Se carga con `(LOAD "mdv1_utiles")`. **Next:** guárdalo en la tarjeta SD como `utiles.lsp` (o cualquier nombre que prefieras). Se carga con `(LOAD "utiles.lsp")`.

El código es idéntico en ambas plataformas:

```
; utiles — Funciones de utilidad para MYLISP
; QL:   (LOAD "mdv1_utiles")
; Next: (LOAD "utiles.lsp")

; Acceso a elementos de lista
(DEFUN CADR  (L) (CAR (CDR L)))
(DEFUN CADDR (L) (CAR (CDR (CDR L))))
(DEFUN CADDDR (L) (CAR (CDR (CDR (CDR L)))))

; Longitud de lista
(DEFUN LONGIT (L)
  (IF (NULL L) 0 (+ 1 (LONGIT (CDR L)))))

; N-ésimo elemento (base 0)
(DEFUN NTHEL (L N)
  (IF (= N 0) (CAR L) (NTHEL (CDR L) (- N 1))))

; Valor absoluto
(DEFUN ABSOL (X) (IF (< X 0) (- X) X))

; Comparadores de símbolo
(DEFUN SYM< (A B) (STR< A B))

; Máximo y mínimo
(DEFUN MAXI (A B) (IF (> A B) A B))
(DEFUN MINI (A B) (IF (< A B) A B))

; MCD (Euclides)
(DEFUN MCD (A B) (IF (= B 0) A (MCD B (MOD A B))))
(DEFUN MCM (A B) (/ (* A B) (MCD A B)))

(PRINT "utiles cargadas")
```

5.1.2 Cargarlo al arrancar

Cada vez que inicies una sesión de trabajo, carga el fichero según tu plataforma:

```
MYLISP> (LOAD "mdv1_utiles") ; QL
MYLISP> (LOAD "utiles.lsp")  ; Next
utiles cargadas
```

A partir de ese momento, `CADR`, `CADDR`, `LONGIT`, `ABSOL` y el resto estarán disponibles para toda la sesión. **Los ejemplos de este capítulo en adelante asumen que has cargado el fichero de utilidades.**

Hábito fundamental: Escribe siempre tu código en ficheros, no sólo en el REPL. Al escribir `BYE` para salir, MYLISP **borra todo el trabajo de la memoria** sin guardar nada. Si defines funciones directamente en el REPL y luego sales, las pierdes para siempre. El flujo correcto es: edita en un fichero → carga con `LOAD` → prueba en el REPL → vuelve al editor para corregir.

5.2 Las listas son el corazón de LISP

El nombre LISP significa *LISt Processing* —procesamiento de listas. No es casualidad. Las listas son la estructura de datos fundamental del lenguaje: con ellas se representan los programas, los datos, las expresiones algebraicas, los árboles de derivación, los entornos de variables... prácticamente todo.

Este capítulo explora en profundidad cómo construir, desmontar, inspeccionar y transformar listas en MYLISP.

5.3 Construir listas

Ya conocemos las tres formas básicas de construir listas. Las revisamos con más detalle.

5.3.1 CONS

`CONS` (de *CONStruct*) crea un nuevo par formado por un elemento y una lista existente. El elemento se convierte en el nuevo primer elemento de la lista resultante.

```
MYLISP> (CONS 1 NIL)
(1)
MYLISP> (CONS 1 '(2 3))
(1 2 3)
MYLISP> (CONS 'A '(B C D))
(A B C D)
MYLISP> (CONS '(1 2) '(3 4))
((1 2) 3 4)
```

`CONS` nunca modifica su segundo argumento. Siempre crea una nueva estructura compartiendo el interior de la lista existente. Esto es eficiente y seguro.

Podemos construir cualquier lista encadenando `CONS`:

```
MYLISP> (CONS 1 (CONS 2 (CONS 3 (CONS 4 NIL))))
(1 2 3 4)
```

5.3.2 LIST

`LIST` es un atajo para construir listas de elementos conocidos:

```
MYLISP> (LIST 1 2 3)
(1 2 3)
MYLISP> (LIST 'A 'B 'C)
(A B C)
MYLISP> (LIST 1 '(2 3) 4)
(1 (2 3) 4)
MYLISP> (LIST)
NIL
```

`(LIST)` sin argumentos devuelve `NIL`, la lista vacía.

5.3.3 APPEND

`APPEND` concatena dos listas:

```
MYLISP> (APPEND '(1 2) '(3 4))
(1 2 3 4)
MYLISP> (APPEND '(A B C) '(D))
(A B C D)
MYLISP> (APPEND NIL '(1 2))
(1 2)
MYLISP> (APPEND '(1 2) NIL)
(1 2)
```

`APPEND` no modifica ninguna de las listas originales; construye una nueva lista que contiene todos los elementos de la primera seguidos de los de la segunda.

¡Atención! En MYLISP, `APPEND` acepta exactamente **dos** listas. No es variádica. Para concatenar tres listas, anida las llamadas: `(APPEND '(1 2) (APPEND '(3 4) '(5 6)))` → `(1 2 3 4 5 6)`.

5.4 Acceder a los elementos

5.4.1 CAR y CDR

`CAR` devuelve el primer elemento; `CDR` devuelve el resto.

```
MYLISP> (CAR '(10 20 30))
10
MYLISP> (CDR '(10 20 30))
(20 30)
MYLISP> (CAR '((A B) C D))
(A B)
MYLISP> (CDR '((A B) C D))
(C D)
```

Ambas funciones generan error si se aplican a `NIL`:

```
MYLISP> (CAR NIL)
ERROR: CAR requiere una lista no vacia
MYLISP> (CDR NIL)
ERROR: CDR requiere una lista no vacia
```

5.4.2 Combinaciones de CAR y CDR

Para acceder a elementos más allá del primero, combinamos `CAR` y `CDR`:

Expresión	Significado
<code>(CAR L)</code>	primer elemento
<code>(CAR (CDR L))</code>	segundo elemento
<code>(CAR (CDR (CDR L)))</code>	tercer elemento
<code>(CDR (CDR L))</code>	lista desde el tercer elemento

Ejemplo con la lista (A B C D):

```
MYLISP> (CAR '(A B C D))
A
MYLISP> (CAR (CDR '(A B C D)))
B
MYLISP> (CAR (CDR (CDR '(A B C D))))
C
```

Para mayor comodidad, podemos definir las abreviaturas que faltan:

```
(DEFINE CADR (LAMBDA (L) (CAR (CDR L))))
(DEFINE CADDR (LAMBDA (L) (CAR (CDR (CDR L)))))
(DEFINE CADDDR (LAMBDA (L) (CAR (CDR (CDR (CDR L))))))
```

Ahora:

```
MYLISP> (CADR '(A B C D))
B
MYLISP> (CADDR '(A B C D))
C
MYLISP> (CADDDR '(A B C D))
D
```

¡Atención! CADDDR tiene 7 caracteres: cabe en el límite de 8. Pero CADDDDR tendría 8 caracteres justos. Ten cuidado con nombres de combinaciones largas.

5.5 Predicados de lista

5.5.1 NULL

NULL devuelve T si su argumento es la lista vacía (NIL), y NIL en caso contrario. Es fundamental para controlar la terminación de la recursión sobre listas.

```
MYLISP> (NULL NIL)
T
MYLISP> (NULL '())
T
MYLISP> (NULL '(1 2))
NIL
MYLISP> (NULL 0)
NIL
```

Nota: NULL y NOT producen el mismo resultado para NIL, pero conceptualmente son distintos: NULL pregunta "¿es esta la lista vacía?"; NOT niega un valor booleano. En la práctica, en MYLISP son equivalentes porque el único valor falso es NIL.

5.5.2 ATOM

ATOM devuelve T para cualquier átomo (número, símbolo, cadena, NIL, T), y NIL para los pares.

```
MYLISP> (ATOM 42)
T
MYLISP> (ATOM 'X)
T
MYLISP> (ATOM NIL)
T
MYLISP> (ATOM '(1 2))
NIL
```

5.5.3 LISTP

`LISTP` devuelve `T` para listas (incluyendo `NIL`) y `NIL` para cualquier átomo no-`NIL`.

```
MYLISP> (LISTP '(1 2 3))
T
MYLISP> (LISTP NIL)
T
MYLISP> (LISTP 'A)
NIL
MYLISP> (LISTP 42)
NIL
```

5.6 Recorrer listas recursivamente

El patrón más importante en programación LISP es el **recorrido recursivo de listas**. La estructura es casi siempre la misma:

```
(DEFINE MI-FUN
  (LAMBDA (LISTA)
    (COND
      ((NULL LISTA) ...) ; caso base: lista vacía
      (T (... (CAR LISTA) ; procesar el primer elemento
              (MI-FUN (CDR LISTA)))))) ; y recurrir sobre el resto
```

Veamos algunos ejemplos concretos.

5.6.1 Suma de los elementos de una lista

```
(DEFINE SUMAR
  (LAMBDA (L)
    (COND
      ((NULL L) 0)
      (T (+ (CAR L) (SUMAR (CDR L))))))
```

```
MYLISP> (SUMAR '(1 2 3 4 5))
15
MYLISP> (SUMAR '(1/3 1/3 1/3))
1
MYLISP> (SUMAR NIL)
0
```

5.6.2 Buscar un elemento en una lista

```
(DEFINE MEMBER
  (LAMBDA (ELEM L)
    (COND
      ((NULL L) NIL)
      ((EQUAL (CAR L) ELEM) T)
      (T (MEMBER ELEM (CDR L))))))
```

```
MYLISP> (MEMBER 3 '(1 2 3 4))
T
MYLISP> (MEMBER 9 '(1 2 3 4))
NIL
MYLISP> (MEMBER 'B '(A B C))
T
```

5.6.3 Contar los elementos de una lista

Como `LENGTH` no está implementada, la definimos:

```
(DEFINE LONGIT
  (LAMBDA (L)
    (COND
      ((NULL L) 0)
      (T (+ 1 (LONGIT (CDR L))))))
```

```
MYLISP> (LONGIT '(A B C D E))
5
MYLISP> (LONGIT NIL)
0
```

Nota: Usamos `LONGIT` (7 caracteres) en lugar de `LENGTH` (6 caracteres). Ambos cabrían en el límite de 8. Pero `LENGTH` podría colisionar con expectativas de otros dialectos, así que preferimos el nombre propio.

5.6.4 Invertir una lista

```
(DEFINE INVERT
  (LAMBDA (L)
    (COND
      ((NULL L) NIL)
      (T (APPEND (INVERT (CDR L))
                  (LIST (CAR L))))))
```

```
MYLISP> (INVERT '(1 2 3 4 5))
(5 4 3 2 1)
MYLISP> (INVERT '(A B C))
(C B A)
```

Nota: Esta implementación de `INVERT` es sencilla pero $O(n^2)$ en tiempo. Para una versión eficiente con acumulador, véase el capítulo 9.

5.7 Transformar listas

5.7.1 Aplicar una función a cada elemento

```
(DEFINE MI-MAP
  (LAMBDA (F L)
    (COND
      ((NULL L) NIL)
      (T (CONS (F (CAR L))
                (MI-MAP F (CDR L)))))))
```

```
MYLISP> (MI-MAP (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
MYLISP> (MI-MAP (LAMBDA (X) (+ X 10)) '(1 2 3))
(11 12 13)
```

5.7.2 Filtrar elementos que cumplen una condición

```
(DEFINE FILTR
  (LAMBDA (PRED L)
    (COND
      ((NULL L) NIL)
      ((PRED (CAR L))
       (CONS (CAR L) (FILTR PRED (CDR L))))
      (T (FILTR PRED (CDR L)))))
```

```
MYLISP> (FILTR (LAMBDA (X) (> X 3)) '(1 2 3 4 5 6))
(4 5 6)
MYLISP> (FILTR (LAMBDA (X) (= 0 (MOD X 2)))
            '(1 2 3 4 5 6 7 8))
```

El último ejemplo filtra los números pares usando `MOD`, que es una primitiva de MYLISP.

5.8 Listas de asociación (A-lists)

Una lista de asociación (*association list* o *a-list*) es una lista de pares `(clave . valor)`. Es la estructura más sencilla para implementar una tabla o diccionario.

```
(DEFINE AGENDA
  '( (ANA . 555-0100)
      (LUIS . 555-0200)
      (EVA . 555-0300)))
```

Para buscar un elemento en una a-list, podemos definir:

```
(DEFINE ASOC
  (LAMBDA (CLAVE ALIST)
    (COND
      ((NULL ALIST) NIL)
      ((EQUAL (CAR (CAR ALIST)) CLAVE)
       (CAR ALIST))
      (T (ASOC CLAVE (CDR ALIST)))))
```

```
MYLISP> (ASOC 'LUIS AGENDA)
(LUIS . 555-0200)
MYLISP> (ASOC 'EVA AGENDA)
(EVA . 555-0300)
MYLISP> (ASOC 'PEDRO AGENDA)
NIL
```

Para obtener sólo el valor (no el par completo):

```
MYLISP> (CDR (ASOC 'ANA AGENDA))
555-0100
```

Las a-lists son muy útiles para representar entornos de variables, sustituciones algebraicas o cualquier mapeo clave-valor.

5.8.1 A-list como entorno de sustitución algebraica

```
(DEFINE ENTORNO
  '((X . 3) (Y . 5) (Z . 7)))

(DEFINE BUSCAR
  (LAMBDA (VAR ENV)
    (COND
      ((NULL ENV) NIL)
      ((EQUAL (CAR (CAR ENV)) VAR)
       (CDR (CAR ENV)))
      (T (BUSCAR VAR (CDR ENV))))))

MYLISP> (BUSCAR 'X ENTORNO)
3
MYLISP> (BUSCAR 'Y ENTORNO)
5
MYLISP> (BUSCAR 'W ENTORNO)
NIL
```

Este patrón es exactamente cómo un evaluador de expresiones algebraicas sustituye variables por valores. Será el punto de partida del CAS en la Parte II.

5.9 Listas como árboles

Las listas anidadas representan naturalmente **árboles**. Un árbol binario puede representarse como una lista de tres elementos (VALOR IZQUIERDO DERECHO):

```
(DEFINE ARBOL
  '(5
    (3 (1 NIL NIL) (4 NIL NIL))
    (8 (7 NIL NIL) (9 NIL NIL))))
```

Esto representa el árbol:

```
      5
     / \
    3   8
   / \ / \
  1  4 7  9
```

Funciones para acceder al árbol:

```
(DEFINE RAIZ (LAMBDA (A) (CAR A)))  
(DEFINE IZQ (LAMBDA (A) (CADR A)))  
(DEFINE DER (LAMBDA (A) (CADDR A)))
```

```
MYLISP> (RAIZ ARBOL)  
5  
MYLISP> (RAIZ (IZQ ARBOL))  
3  
MYLISP> (RAIZ (DER ARBOL))  
8
```

Para recorrer el árbol en orden (izquierda-raíz-derecha):

```
(DEFINE INORDEN  
  (LAMBDA (ARBOL)  
    (COND  
      ((NULL ARBOL) NIL)  
      (T (APPEND  
          (INORDEN (IZQ ARBOL))  
          (LIST (RAIZ ARBOL))  
          (INORDEN (DER ARBOL)))))))
```

```
MYLISP> (INORDEN ARBOL)  
(1 3 4 5 7 8 9)
```

El recorrido en orden de un árbol de búsqueda binario devuelve los elementos ordenados.

Continúa en el Capítulo 6: Control del flujo

Capítulo 6 — Control del flujo

6.1 Formas especiales de control

En MYLISP, el control del flujo no se realiza mediante sentencias (como en Pascal o BASIC) sino mediante **formas especiales**: construcciones del lenguaje que controlan cuándo y cómo se evalúan sus subexpresiones.

A diferencia de las funciones ordinarias, las formas especiales **no evalúan todos sus argumentos automáticamente**. Esto es lo que las hace útiles para el control del flujo: si `IF` evaluara siempre ambas ramas, no serviría para nada.

Las formas especiales de control en MYLISP son: `IF`, `COND`, `AND`, `OR`, `NOT` y `PROGN`.

6.2 IF: la bifurcación más sencilla

`IF` evalúa una condición y ejecuta una de dos ramas:

```
(IF condición expresión-verdadera expresión-falsa)
```

```
MYLISP> (IF T 1 2)
1
MYLISP> (IF NIL 1 2)
2
MYLISP> (IF (> 5 3) "mayor" "menor")
"mayor"
```

Sólo se evalúa la rama correspondiente. La otra rama **no se toca**:

```
MYLISP> (IF T 42 (/ 1 0))
42
```

La expresión `(/ 1 0)` no se evalúa porque la condición es verdadera. Si se evaluara, produciría un error de división por cero.

6.2.1 IF con una sola rama

La rama "falsa" es opcional. Si se omite y la condición es falsa, `IF` devuelve `NIL`:

```
MYLISP> (IF NIL 99)
NIL
MYLISP> (IF T 99)
99
```

6.2.2 Ejemplos prácticos

Valor absoluto (simulado sin `ABS`):

```
(DEFINE ABSOL
  (LAMBDA (X)
    (IF (< X 0) (- X) X)))
```

```
MYLISP> (ABSOL -5)
5
MYLISP> (ABSOL 3)
3
MYLISP> (ABSOL 0)
0
```

Máximo de dos números (sin `MAX`):

```
(DEFINE MAXI
  (LAMBDA (A B)
    (IF (> A B) A B)))
```

```
MYLISP> (MAXI 7 3)
7
MYLISP> (MAXI 3 7)
7
MYLISP> (MAXI 5 5)
5
```

6.3 COND: el multi-caso

`COND` generaliza `IF` a múltiples condiciones. Su estructura es:

```
(COND
  (condición-1 expresión-1)
  (condición-2 expresión-2)
  ...
  (T      expresión-por-defecto))
```

MYLISP evalúa las condiciones **de arriba a abajo** y ejecuta la expresión correspondiente a la **primera condición verdadera**. Las condiciones restantes no se evalúan.

```
MYLISP> (COND
  ((= 1 2) "uno igual a dos")
  ((= 1 1) "uno igual a uno")
  (T      "ninguna anterior"))
"uno igual a uno"
```

La cláusula `(T ...)` al final actúa como "en caso contrario": como `T` siempre es verdadero, se ejecuta si ninguna condición anterior lo fue.

6.3.1 Clasificar un número

```
(DEFINE SIGNO
  (LAMBDA (N)
    (COND
      ((> N 0) "positivo")
      ((< N 0) "negativo")
      (T      "cero"))))
```

```
MYLISP> (SIGNO 5)
"positivo"
MYLISP> (SIGNO -3)
"negativo"
MYLISP> (SIGNO 0)
"cero"
```

6.3.2 COND sin cláusula por defecto

Si ninguna condición es verdadera y no hay cláusula `T`, `COND` devuelve `NIL`:

```
MYLISP> (COND
  ((= 1 2) "nunca"))
NIL
```

6.3.3 COND en el derivador simbólico

`COND` es ideal para implementar funciones que se comportan diferente según el tipo de expresión —exactamente lo que necesita un derivador simbólico:

```
(DEFINE DERIV
  (LAMBDA (EXP VAR)
    (COND
      ((EQUAL EXP VAR)      1)
      ((ATOM EXP)           0)
```

```
((EQUAL (CAR EXP) '+)
 (LIST '+ (DERIV (CADR EXP) VAR)
         (DERIV (CADDR EXP) VAR)))
(EQUAL (CAR EXP) '*)
 (LIST '+
       (LIST '* (CADR EXP)
              (DERIV (CADDR EXP) VAR))
       (LIST '* (DERIV (CADR EXP) VAR)
              (CADDR EXP))))
(T (LIST 'D EXP VAR)))) ; derivada desconocida
```

El capítulo 10 desarrolla este derivador en profundidad.

6.4 AND: conjunción con cortocircuito

`AND` evalúa sus argumentos **de izquierda a derecha** y se detiene en cuanto uno es `NIL` (falso). Devuelve el primer valor falso encontrado, o el último argumento si todos son verdaderos.

```
MYLISP> (AND T T T)
T
MYLISP> (AND T NIL T)
NIL
MYLISP> (AND 1 2 3)
3
MYLISP> (AND 1 NIL 3)
NIL
```

El **cortocircuito** significa que los argumentos a la derecha de un `NIL` no se evalúan:

```
MYLISP> (AND NIL (/ 1 0))
NIL
```

La división por cero no llega a ejecutarse porque `AND` ve `NIL` y para.

6.4.1 Usos típicos de AND

Comprobar varias condiciones a la vez:

```
(DEFINE RANGOP
  (LAMBDA (X MIN MAX)
    (AND (>= X MIN) (<= X MAX))))
```

```
MYLISP> (RANGOP 5 1 10)
T
MYLISP> (RANGOP 15 1 10)
NIL
```

Validar antes de calcular:

```
(DEFINE DIVISI
  (LAMBDA (A B)
    (AND (NOT (= B 0)) (/ A B))))
```

```
MYLISP> (DIVISI 10 2)
5
MYLISP> (DIVISI 10 0)
NIL
```

6.5 OR: disyunción con cortocircuito

`OR` evalúa sus argumentos de izquierda a derecha y devuelve el **primer valor verdadero** encontrado, o `NIL` si todos son falsos.

```
MYLISP> (OR NIL NIL T)
T
MYLISP> (OR NIL NIL NIL)
NIL
MYLISP> (OR NIL 42 T)
42
```

Al igual que `AND`, `OR` usa cortocircuito:

```
MYLISP> (OR T (/ 1 0))
T
```

La división por cero no se ejecuta porque el primer argumento ya es verdadero.

6.5.1 Usos típicos de OR

Proporcionar un valor por defecto:

```
(DEFINE DEFAULT
  (LAMBDA (VALOR DEFECTO)
    (OR VALOR DEFECTO)))
```

```
MYLISP> (DEFAULT NIL 99)
99
MYLISP> (DEFAULT 42 99)
42
```

Buscar en varias fuentes:

```
(DEFINE OBTENER
  (LAMBDA (CLAVE ENV1 ENV2)
    (OR (BUSCAR CLAVE ENV1)
        (BUSCAR CLAVE ENV2))))
```

6.6 NOT: negación lógica

`NOT` devuelve `T` si su argumento es `NIL`, y `NIL` en caso contrario.

```
MYLISP> (NOT NIL)
T
MYLISP> (NOT T)
NIL
MYLISP> (NOT 42)
NIL
MYLISP> (NOT '(A B))
NIL
```

Cualquier valor no-`NIL` es tratado como verdadero: `NOT` devuelve `NIL` para todos ellos.

6.6.1 Combinando NOT con predicados

```
(DEFINE NULOP
  (LAMBDA (X)
    (NOT (NULL X)))) ; verdadero si x NO es vacío
```

```
MYLISP> (NULOP '(1 2))
T
MYLISP> (NULOP NIL)
NIL
```

```
(DEFINE DIFERNT
  (LAMBDA (A B)
    (NOT (EQUAL A B))))
```

```
MYLISP> (DIFERNT 'A 'B)
T
MYLISP> (DIFERNT 'A 'A)
NIL
```

6.7 PROGN: secuencia de expresiones

`PROGN` evalúa una secuencia de expresiones de izquierda a derecha y devuelve el valor de la **última**. Se usa cuando necesitas ejecutar varias expresiones donde normalmente cabría una sola.

```
MYLISP> (PROGN
  (DEFINE X 10)
  (DEFINE Y 20)
  (+ X Y))
30
```

`PROGN` es especialmente útil en la rama "verdadera" o "falsa" de un `IF` cuando quieres hacer más de una cosa:

```
(IF (> X 0)
  (PROGN
    (PRINT "positivo")
    (* X 2))
  (PROGN
    (PRINT "negativo o cero")
    (- X)))
```

Sin `PROGN`, `IF` sólo podría ejecutar una expresión por rama.

6.8 Combinar las formas de control

Las formas de control se combinan naturalmente para construir lógica compleja.

6.8.1 Clasificador de triángulos

```
(DEFINE TRIANG
  (LAMBDA (A B C)
    (COND
      ((NOT (AND (> A 0) (> B 0) (> C 0)))
        "lados invalidos")
      ((AND (= A B) (= B C))
        "equilatero")
      ((OR (= A B) (= B C) (= A C))
        "isosceles")
      (T
        "escaleno")))))
```

```
MYLISP> (TRIANG 3 3 3)
"equilatero"
MYLISP> (TRIANG 5 5 3)
"isosceles"
MYLISP> (TRIANG 3 4 5)
"escaleno"
MYLISP> (TRIANG -1 2 3)
"lados invalidos"
```

6.8.2 Buscar el máximo en una lista

```
(DEFINE MAXIMOL
  (LAMBDA (L)
    (COND
      ((NULL L) NIL)
      ((NULL (CDR L)) (CAR L))
      (T (IF (> (CAR L) (MAXIMOL (CDR L)))
              (CAR L)
              (MAXIMOL (CDR L)))))))
```

```
MYLISP> (MAXIMOL '(3 1 4 1 5 9 2 6))
9
MYLISP> (MAXIMOL '(7))
7
```

6.8.3 Verificar si una lista está ordenada

```
(DEFINE ORDENA
  (LAMBDA (L)
    (OR (NULL L)
        (NULL (CDR L))
        (AND (<= (CAR L) (CADR L))
              (ORDENA (CDR L))))))
```

```
MYLISP> (ORDENA '(1 2 3 4 5))
T
MYLISP> (ORDENA '(1 3 2 4))
NIL
MYLISP> (ORDENA NIL)
T
```

6.9 Tablas de verdad de AND, OR y NOT

Para referencia rápida:

A	B	(AND A B)	(OR A B)
T	T	T	T
T	NIL	NIL	T
NIL	T	NIL	T
NIL	NIL	NIL	NIL

A	(NOT A)
T	NIL
NIL	T
42	NIL
"hola"	NIL

Recuerda: en MYLISP, cualquier valor distinto de `NIL` es verdadero.

Continúa en el Capítulo 7: Definir funciones y clausuras léxicas

Capítulo 7 — Definir funciones y clausuras léxicas

7.1 Funciones en LISP: todo es un valor

En MYLISP, las funciones son **valores de primera clase**: pueden ser creadas, almacenadas en variables, pasadas como argumentos a otras funciones y devueltas como resultado de una evaluación. Esta propiedad, compartida con Scheme y los lenguajes funcionales modernos, es lo que hace de LISP una herramienta tan poderosa.

Este capítulo cubre las cuatro formas de definir funciones en MYLISP: `LAMBDA`, `DEFINE`, `DEFUN` y `LET`, así como los conceptos de recursión y clausuras léxicas.

7.2 LAMBDA: la función anónima

`LAMBDA` es la única forma que existe en MYLISP para *crear* una función. `DEFINE` y `DEFUN`, que veremos a continuación, no crean nada nuevo: se limitan a poner una etiqueta sobre una función que, por debajo, siempre se construyó con `LAMBDA`. Entender bien `LAMBDA` es entender de dónde sale realmente el poder de LISP para tratar el código como datos.

7.2.1 De dónde viene el nombre, y por qué importa

El nombre no es un capricho tipográfico. Viene directamente del **cálculo lambda** de Alonzo Church (1930s), la notación matemática que McCarthy usó como punto de partida para diseñar LISP en 1958 (§1.1.1). Church necesitaba una forma de escribir "la función que hace tal cosa" sin tener que bautizarla primero — igual que en matemáticas escribimos $x \mapsto x^2$ sin necesidad de llamarla `f`. `(LAMBDA (X) (* X X))` es exactamente esa idea trasladada a S-expresiones: "la función que, dado `x`, devuelve `x * x`", sin nombre alguno.

Esto es lo que distingue a `LAMBDA` de `DEFUN`: `DEFUN` responde a la pregunta "¿cómo se llama esta función?"; `LAMBDA` ni siquiera admite la pregunta — la función existe y es utilizable sin que nadie la haya nombrado nunca. Esta capacidad de crear funciones "de usar y tirar" es la base de la programación funcional de orden superior que se explota a fondo en el capítulo 9 y en el propio CAS (a partir del capítulo 10).

7.2.2 Anatomía de la expresión

Su estructura general es:

```
(LAMBDA (parámetro-1 parámetro-2 ...) cuerpo)
```

Dos partes, ambas obligatorias:

- **La lista de parámetros**, entre paréntesis. Puede tener cero parámetros (`(LAMBDA () ...)`, una función sin argumentos), uno, o varios — MYLISP no impone un límite práctico distinto del que impone el propio entorno de trabajo.
- **El cuerpo**: una única expresión que se evalúa cuando la función se llama, con los parámetros ya sustituidos por los argumentos recibidos. Si en algún momento necesitas ejecutar varios pasos, la solución es envolverlos con `PROGN` (§7.5) — pero la regla de `LAMBDA` en sí misma es siempre "una función, una expresión".

```
MYLISP> ((LAMBDA (X) (* X X)) 5)
25
```

Aquí `(LAMBDA (X) (* X X))` es la función "elevar al cuadrado". Al escribirla como el primer elemento de una lista que se evalúa, se **aplica inmediatamente** al argumento `5`: la propia regla 3 de evaluación (§3.4) no distingue entre `(CUADRA 5)` y `((LAMBDA (X) (* X X)) 5)` — en ambos casos, lo primero se evalúa hasta obtener una función, y esa función se aplica al resto.

```
MYLISP> ((LAMBDA (A B) (+ (* A A) (* B B))) 3 4)
25
```

Esta función anónima de dos parámetros calcula $a^2 + b^2$. El número de argumentos en la llamada debe corresponderse exactamente con el número de parámetros declarados: `LAMBDA` no admite parámetros opcionales ni "resto de argumentos" (no hay equivalente al `&rest` de Common LISP o a los argumentos variádicos de las primitivas aritméticas del capítulo 4 — esas son variádicas porque son primitivas del intérprete, no funciones definidas por el usuario).

7.2.3 LAMBDA es un valor, no una acción

Este es el punto que más cuesta interiorizar viniendo de lenguajes imperativos: **evaluar una expresión `LAMBDA` no ejecuta el cuerpo**. Solo lo *empaqueta*. El cuerpo `(* X X)` no se toca para nada hasta que la función resultante se aplica a un argumento concreto.

Dicho de otro modo: `LAMBDA` es una forma especial (§3.4) que, al evaluarse, produce un **valor de función** — internamente, MYLISP lo representa como una clausura de tipo `TCLOSURE` (Apéndice B.6): un par formado por la propia expresión `LAMBDA` y una referencia al entorno en el que apareció. Ese valor puede tratarse exactamente igual que un número o una lista: guardarse en una variable, pasarse como argumento, devolverse como resultado. Esta propiedad —**las funciones son valores de primera clase**— es la que se anunció en el §7.1, y `LAMBDA` es el mecanismo concreto que la hace posible.

Una función `LAMBDA` se puede almacenar en una variable con `DEFINE`:

```
MYLISP> (DEFINE CUADRA (LAMBDA (X) (* X X)))
CUADRA
MYLISP> (CUADRA 7)
49
```

Ahora `CUADRA` es un símbolo cuyo valor es la función. Llamar a `(CUADRA 7)` evalúa `CUADRA` (obteniendo la función), y luego aplica la función al argumento `7`. Nótese que `DEFINE` no le añade ningún poder a la función: `CUADRA` y `(LAMBDA (X) (* X X))` son, en todo momento, exactamente el mismo valor — uno simplemente tiene un nombre en el entorno global y el otro no.

7.2.4 Toda su potencia: pasar y devolver funciones

El que `LAMBDA` produzca un valor ordinario, y no una acción, es lo que permite dos usos que son el corazón de la programación funcional en LISP:

Pasar una función anónima como argumento. No hace falta darle nombre a una función que solo se va a usar una vez. Si en el capítulo 5 tenemos una función `MI-MAP` que aplica una función a cada elemento de una lista, no necesitamos definir `CUADRA` de antemano si solo la vamos a usar ahí:

```
MYLISP> (MI-MAP (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
```

La función que eleva al cuadrado se crea, se usa una vez, y se descarta — sin ensuciar el entorno global con un nombre (`CUADRA`) que solo hacía falta una vez. Este patrón —una `LAMBDA` anónima como argumento de una función de orden superior— es la forma normal de trabajar con `MI-MAP`, `FILTR` y `MI-REDUC` en el capítulo 5, y con sus versiones definitivas `MY-MAP`, `MY-FILTR` y `MY-REDUC` en el capítulo 9.

Devolver una función desde otra función. Como `LAMBDA` es un valor, una función puede construir y devolver otra función en tiempo de ejecución:

```
(DEFINE HACESUMA
  (LAMBDA (N)
    (LAMBDA (X) (+ X N))))
```

`(HACESUMA 5)` no calcula un número: evalúa el cuerpo de `HACESUMA`, que es a su vez una expresión `LAMBDA`, y esa expresión se evalúa hasta obtener *otra* función — una que ya lleva "incorporado" el valor `N = 5`. El resultado no es una función cualquiera: es una **clausura**, porque recuerda el entorno en el que fue creada incluso después de que `HACESUMA` haya terminado de ejecutarse. Ese mecanismo —qué es exactamente lo que se captura, y qué se puede y no se puede hacer con ello en MYLISP— es el tema completo del §7.9; por ahora basta con ver que es **la misma `LAMBDA` de siempre**, sin ninguna sintaxis adicional, la que lo hace posible.

Estas dos capacidades —función como argumento, función como resultado— son exactamente la definición operativa de "función de orden superior". No son una característica extra de MYLISP: son una consecuencia directa y gratuita de que `LAMBDA` evalúa a un valor de primera clase, tal como se explicó en el §7.2.3.

7.3 DEFINE: dar nombre a valores y funciones

`DEFINE` asocia un símbolo con un valor en el **entorno global**:

```
(DEFINE nombre valor)
```

Para funciones:


```
(DEFINE CUADRA (LAMBDA (X) (* X X)))  
(DEFINE CUBO (LAMBDA (X) (* X X X)))  
(DEFINE PI 3.14159)
```

```
MYLISP> (CUADRA 4)  
16  
MYLISP> (CUBO 3)  
27  
MYLISP> (* 2 PI 5)  
31.4159
```

`DEFINE` puede redefinir un símbolo existente. La nueva definición reemplaza a la anterior:

```
MYLISP> (DEFINE X 10)  
X  
MYLISP> X  
10  
MYLISP> (DEFINE X 99)  
X  
MYLISP> X  
99
```

7.4 DEFUN: sintaxis abreviada para funciones

`DEFUN` es una forma especial que abrevia el patrón `(DEFINE nombre (LAMBDA ...))`:

```
(DEFUN nombre (parámetros...) cuerpo)
```

Los dos siguientes son equivalentes:

```
; Con DEFINE y LAMBDA:  
(DEFINE CUADRA (LAMBDA (X) (* X X)))  
  
; Con DEFUN:  
(DEFUN CUADRA (X) (* X X))
```

`DEFUN` es más conciso y se asemeja a la sintaxis de Common LISP. En los ficheros de código es la forma preferida porque ocupa menos espacio en el límite de 80 caracteres del REPL o en los ficheros.

```
(DEFUN SUMAR (A B) (+ A B))  
(DEFUN MAYOR (A B) (IF (> A B) A B))  
(DEFUN PAREP (N) (= 0 (- N (* (/ N 2) 2))))
```

Nota: En el interior de MYLISP, `DEFUN` simplemente expande a `DEFINE` con un `LAMBDA`. No hay diferencia funcional entre ambas formas.

7.5 Funciones con múltiples expresiones en el cuerpo

El cuerpo de un `LAMBDA` o `DEFUN` es una sola expresión. Si necesitas ejecutar varias cosas, usa `PROGN`:

```
(DEFUN DEPURA (X)
  (PROGN
    (PRINT X)
    (* X X)))
```

```
MYLISP> (DEPURA 5)
5
25
```

`PRINT` muestra el valor intermedio; `(* X X)` es el valor devuelto.

7.6 Recursión

7.6.1 ¿Qué es la recursión?

En la mayoría de los lenguajes imperativos —BASIC, Pascal, C— la repetición se expresa con bucles: `FOR`, `WHILE`, `REPEAT`. En LISP no hay bucles. La repetición se expresa mediante **recursión**: una función que se llama a sí misma.

La idea parece circular a primera vista. ¿Cómo puede una función definirse en términos de sí misma sin entrar en un bucle infinito? La respuesta está en la estructura del problema: todo problema recursivo tiene al menos dos casos.

El **caso base** es la situación más sencilla posible, que tiene una respuesta inmediata y directa sin necesidad de recurrir. El **caso recursivo** descompone el problema en uno más pequeño del mismo tipo, y llama a la propia función sobre ese subproblema. La clave es que cada llamada recursiva se acerca al caso base. En algún momento se llega a él, la recursión se detiene y los resultados parciales se combinan de vuelta hasta la llamada original.

Para calcular el factorial de N, podemos razonar así: si N es 0, el resultado es 1 (caso base, por definición). Si N es mayor que 0, el factorial de N es N multiplicado por el factorial de N-1 (caso recursivo, que se acerca a 0 en cada paso). En notación matemática: $N! = N \times (N-1)!$. En MYLISP, esta definición se escribe casi literalmente:

```
FACT(5) = 5 × FACT(4)
        = 5 × 4 × FACT(3)
          = 5 × 4 × 3 × FACT(2)
            = 5 × 4 × 3 × 2 × FACT(1)
              = 5 × 4 × 3 × 2 × 1 × FACT(0)
                = 1
```

La recursión "desciende" hasta `FACT(0)` y luego "asciende" combinando los resultados: `1`, `1×1=1`, `2×1=2`, `3×2=6`, `4×6=24`, `5×24=120`.

7.6.2 El patrón general

Toda función recursiva en MYLISP sigue este esqueleto:

```
(DEFUN MI-FUN (ARGUMENTO)
  (COND
    (caso-base      resultado-directo)
    (T              (combinar (MI-FUN argumento-mas-pequeño)
                              otros-datos))))
```

"Argumento más pequeño" puede significar N-1 para un entero, o `(CDR lista)` para una lista (que tiene un elemento menos). "Combinar" puede ser una suma, una multiplicación, un `CONS`, un `APPEND`... lo que el problema requiera.

7.6.3 Factorial

Sin recursión, el único camino para calcular `5!` en MYLISP sería escribir cada operación a mano:

```
; Sin recursión: hay que expandir manualmente para cada N
MYLISP> (* 5 (* 4 (* 3 (* 2 (* 1 1)))))
120
```

Esto funciona para `5!`, pero no sirve para una función general: necesitaríamos una expresión distinta para cada valor de `N`. No hay ningún `FOR` ni `WHILE` con el que iterar. La recursión resuelve exactamente este problema: permite escribir la regla general una sola vez y que el intérprete la aplique tantas veces como sea necesario.

```
; Con recursión: una única definición válida para cualquier N
(DEFUN FACT (N)
  (IF (= N 0)
    1
    (* N (FACT (- N 1)))))
```

```
MYLISP> (FACT 5)
120
MYLISP> (FACT 0)
1
MYLISP> (FACT 10)
3628800
```

¡Atención! `FACT` tiene 4 caracteres, dentro del límite. Si hubieras escrito `FACTORIAL` (9 caracteres), quedaría truncado a `FACTORIA` y una llamada a `FACTORIAL` daría "símbolo no definido".

7.6.4 Fibonacci

La sucesión de Fibonacci es una secuencia de números en la que cada término es la suma de los dos anteriores: `0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...`. Los dos primeros términos son 0 y 1 por definición (los dos casos base), y a partir de ahí cada `FIBON(N) = FIBON(N-1) + FIBON(N-2)`. Es el ejemplo canónico de recursión con dos casos base y dos llamadas recursivas en el caso general.

```
(DEFUN FIBON (N)
  (COND
    ((= N 0) 0)
    ((= N 1) 1)
    (T (+ (FIBON (- N 1))
          (FIBON (- N 2))))))
```

```
MYLISP> (FIBON 0)
0
MYLISP> (FIBON 7)
13
MYLISP> (FIBON 10)
55
```

7.6.5 Potencia entera

Sin `EXPT`, la definimos:

```
(DEFUN POTENC (BASE EXP)
  (COND
    ((= EXP 0) 1)
    (> EXP 0) (* BASE (POTENC BASE (- EXP 1)))
    (T (/ 1 (POTENC BASE (- EXP)))))
```

```
MYLISP> (POTENC 2 10)
1024
MYLISP> (POTENC 3 4)
81
MYLISP> (POTENC 2 -3)
1/8
```

La potencia con exponente negativo devuelve un racional exacto.

7.6.6 Recursión de cola y el problema de la pila

El comportamiento bajo recursión profunda es la diferencia más notable entre las dos plataformas:

QL: cada llamada recursiva añade un marco a la pila de llamadas de Pascal. Si la recursión es suficientemente profunda, la pila se agota y el intérprete destruye todas las definiciones de usuario para recuperarse. En la práctica, el heap de 24.000 celdas suele agotarse antes que la pila para la mayoría de programas, pero la amenaza es real para listas muy largas o recursiones sin caso base. Para cálculos con muchos niveles, la versión con acumulador es la solución recomendada. **Next:** el evaluador es iterativo — la recursión de usuario no consume pila de CPU en absoluto. Puedes escribir funciones con cientos de niveles de recursión sin ningún problema. El único límite real es el heap de 32.000 celdas: cada llamada activa consume algunas celdas para el entorno y los argumentos, pero este límite es mucho más generoso que la pila del QL.

7.6.7 Recursión con acumulador (cola)

La versión con acumulador de factorial:

```
(DEFUN FACTA (N ACC)
  (IF (= N 0)
    ACC
    (FACTA (- N 1) (* N ACC))))

(DEFUN FACT (N)
  (FACTA N 1))
```

```
MYLISP> (FACT 5)
120
```

La recursión de cola no es optimizada automáticamente por MYLISP (no implementa TCO — *tail call optimization*), pero la estructura con acumulador puede ser más clara para el programador.

7.7 Cómo depurar funciones recursivas

La recursión es poderosa pero puede ser desconcertante cuando no funciona como esperamos. La técnica de depuración más efectiva en MYLISP es el **tracing manual con PRINT**: insertar llamadas a `PRINT` al principio del cuerpo de la función para ver cómo los argumentos cambian en cada nivel de recursión.

7.7.1 Tracing básico

```
(DEFUN FACT (N)
  (PROGN
    (PRINT N)                ; muestra el N en cada llamada
    (IF (= N 0)
      1
      (* N (FACT (- N 1))))))
```

```
MYLISP> (FACT 4)
4
3
2
1
0
24
```

La traza muestra el "descenso" hasta el caso base (`N = 0`) y luego el resultado final (`24`). Si el caso base nunca aparece, la función está en recursión infinita.

7.7.2 Detectar recursión infinita

El síntoma más común de un error en la recursión es que el intérprete deja de responder. Esto ocurre porque la función se llama a sí misma sin acercarse nunca al caso base, consumiendo recursos hasta que el intérprete no puede continuar. Cuando eso sucede, MYLISP **destruye todas las definiciones de usuario** y muestra el prompt de nuevo. Habrás perdido el trabajo de la sesión si no lo tenías guardado en fichero.

QL: lo que se agota es la pila de llamadas de Pascal. La profundidad de recursión antes del fallo es relativamente baja. **Next:** lo que se agota es el heap (32.000 celdas). La profundidad puede ser de cientos de niveles antes del fallo, pero el resultado final es el mismo.

Los dos errores más frecuentes son: olvidar el caso base, y no acercarse a él en el caso recursivo.

```
; ERROR: el caso base nunca se alcanza para N negativo
(DEFUN FACT (N)
  (IF (= N 0)
    1
    (* N (FACT (- N 1)))))

MYLISP> (FACT -1) ; llamará FACT(-1), FACT(-2), FACT(-3)... sin parar
```

La solución: añade comprobación de entrada.

```
(DEFUN FACT (N)
  (COND
    ((< N 0) (PRINT "error: negativo"))
    ((= N 0) 1)
    (T (* N (FACT (- N 1)))))
```

7.7.3 Tracing de dos argumentos

Para funciones con acumulador, imprime ambos:

```
(DEFUN FACTA (N ACC)
  (PROGN
    (DISPLAY "N=") (PRINT N)
    (DISPLAY "ACC=") (PRINT ACC)
    (IF (= N 0)
      ACC
      (FACTA (- N 1) (* N ACC)))))
```

```
MYLISP> (FACTA 3 1)
N=3
ACC=1
N=2
ACC=3
N=1
ACC=6
N=0
ACC=6
6
```

Puedes ver cómo `ACC` acumula el resultado a medida que `N` desciende. Una vez que la función funciona correctamente, elimina los `PRINT` del código definitivo.

7.8 LET: variables locales

`LET` introduce **variables locales** que sólo existen dentro de su cuerpo. Su estructura es:

```
(LET ((var-1 val-1)
      (var-2 val-2)
      ...))
expresión-única)
```

El cuerpo de LET acepta una sola expresión. A diferencia de Scheme y Common LISP, donde el cuerpo de `LET` tiene un `PROGN` implícito, en MYLISP el cuerpo es **una única expresión**. Si necesitas ejecutar varias cosas dentro del `LET`, envuélvelas en un `PROGN` explícito: `lisp (LET ((X 5)) (PRINT X) ; ERROR: esto es un segundo argumento, no un cuerpo (* X X)) (LET ((X 5)) (PROGN ; CORRECTO: PROGN agrupa las dos expresiones en una (PRINT X) (* X X)))`

```
MYLISP> (LET ((X 5)
              (Y 3))
  (+ X Y))
8
```

`X` e `Y` sólo existen dentro del `LET`. Fuera de él, no son accesibles (a menos que ya existieran en el entorno global).

7.8.1 LET es de ámbito paralelo

Una propiedad importante: en `LET`, **todos los valores de la derecha se evalúan en el entorno actual, antes de que ninguna nueva variable sea visible**. Esto se llama vinculación paralela.

```
(DEFINE X 10)
(LET ((X 1)
      (Y X)) ; este X es el global, no el nuevo
  (LIST X Y))
```

```
MYLISP> (LET ((X 1) (Y X))
          (LIST X Y))
(1 10)
```

`Y` toma el valor de `X` **antes** de que el `LET` cree el nuevo `X`. Por eso `Y = 10` (el `X` global), no `1`.

Si necesitas que `Y` use el nuevo `X`, debes anidar los `LET`:

```
MYLISP> (LET ((X 1))
          (LET ((Y X))
              (LIST X Y)))
(1 1)
```

7.8.2 Usar LET para evitar cálculos repetidos

```
(DEFUN HIPOTN (A B)
  (LET ((A2 (* A A))
        (B2 (* B B)))
    (+ A2 B2))) ; devuelve el cuadrado de la hipotenusa
```

```
MYLISP> (HIPOTN 3 4)
25
```

`A2` y `B2` se calculan una sola vez aunque se usen múltiples veces en el cuerpo.

7.8.3 LET con funciones auxiliares locales

```
(DEFUN NORMAL (L)
  (LET ((TOTAL (SUMAR L)))
    (MI-MAP (LAMBDA (X) (/ X TOTAL)) L)))
```

Aquí `TOTAL` es la suma de la lista, calculada una vez. Luego cada elemento se divide por ese total. El resultado es la lista normalizada (todos los elementos suman 1):

```
MYLISP> (NORMAL '(1 2 3 4))
(1/10 1/5 3/10 2/5)
```

7.9 Clausuras léxicas

Una **clausura** es una función que "recuerda" el entorno en el que fue creada, incluso después de que ese entorno ya no sea el activo. Es uno de los mecanismos más poderosos y elegantes de LISP.

Cuando MYLISP evalúa una expresión `LAMBDA`, no solo crea la función —crea una clausura: la función junto con una referencia al entorno léxico en el que fue definida.

7.9.1 Un ejemplo sencillo

```
(DEFINE HACESUMA
  (LAMBDA (N)
    (LAMBDA (X) (+ X N))))
```

`HACESUMA` devuelve una función. Llamarla con `5` devuelve una función que suma 5 a cualquier número:

```
MYLISP> (DEFINE SUMA5 (HACESUMA 5))
SUMA5
MYLISP> (SUMA5 10)
15
MYLISP> (SUMA5 3)
8
MYLISP> (SUMA5 100)
105
```

¿Qué ocurre aquí? Cuando evaluamos `(HACESUMA 5)`:

1. Se crea un entorno donde `N = 5`.
2. Se evalúa el cuerpo `(LAMBDA (X) (+ X N))`.
3. Se crea una clausura que captura ese entorno: la función recuerda que `N = 5`.
4. Se devuelve esa clausura.

Cuando después llamamos `(SUMA5 10)`, la función busca `N` en el entorno capturado y encuentra `5`. Calcula `10 + 5 = 15`.

7.9.2 Creando multiplicadores

```
(DEFINE HACEMUL
  (LAMBDA (FACTOR)
    (LAMBDA (X) (* X FACTOR))))

(DEFINE DOBLE (HACEMUL 2))
(DEFINE TRIPLE (HACEMUL 3))
```

```
MYLISP> (DOBLE 7)
14
MYLISP> (TRIPLE 4)
12
MYLISP> (DOBLE (TRIPLE 5))
30
```

7.9.3 Clausuras como "objetos": lo que MYLISP puede y no puede hacer

En lenguajes con mutación léxica (como Scheme con `set!`) las clausuras pueden encapsular estado mutable y actuar como objetos simples. En MYLISP esto **no es posible**: `DEFINE` siempre escribe en el entorno global, nunca en el entorno capturado por la clausura. Un "contador" basado en clausuras rompería silenciosamente: dos contadores distintos compartirían y machacarían la misma variable global.

Lo que sí funciona con elegancia en MYLISP son las **clausuras parametrizadas de comportamiento** — clausuras que capturan valores constantes, no estado mutable:

7.9.4 Clausuras para parametrizar comportamiento

Un caso de uso muy práctico: pasar funciones como argumentos.

```
(DEFUN APLICAR (F LISTA)
  (MI-MAP F LISTA))
```



```
MYLISP> (APLICAR (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
MYLISP> (APLICAR (LAMBDA (X) (+ X 100)) '(1 2 3))
(101 102 103)
```

Aquí la función `F` es una clausura anónima que se crea en el momento de la llamada.

7.10 El espacio de nombres único (Lisp-1)

MYLISP es un **Lisp-1**: funciones y variables comparten el mismo espacio de nombres. Esto contrasta con Common LISP (Lisp-2), donde hay espacios separados.

La consecuencia práctica es que puedes usar variables cuyo nombre coincide con el de una función:

```
(DEFINE SUMA (LAMBDA (A B) (+ A B)))
(DEFINE SUMA 99) ; ahora SUMA es un número, no una función
```

```
MYLISP> (SUMA 3 4)
ERROR: ... ; SUMA ya no es una función
```

Esto rara vez es un problema en la práctica, pero hay que ser consciente de que redefinir un símbolo afecta tanto a su uso como variable como a su uso como función.

7.10.1 Colisión con primitivas: un peligro real

El caso más peligroso es redefinir accidentalmente una primitiva del intérprete. En MYLISP no hay protección contra ello:

```
MYLISP> (DEFINE CAR 5)
5
MYLISP> (CAR '(A B C))
ERROR: ... ; CAR ya no es una función, es el número 5
```

La primitiva `CAR` ha sido destruida para esta sesión. **No hay forma de restaurarla sin reiniciar el intérprete.** La única salida es:

1. Escribir `BYE` para salir de MYLISP.
2. Relanzar el intérprete desde el QL.
3. Recargar todos tus ficheros de trabajo con `LOAD`.

Para evitar este problema, elige nombres de funciones y variables que no colisionen con las primitivas. Una convención útil es usar prefijos propios (`MI-CAR`, `MIS-`, etc.) para todo el código de usuario. La lista completa de primitivas está en el Apéndice A.

7.10.2 La ventaja del Lisp-1

La ventaja del Lisp-1 es que las funciones de orden superior son más sencillas de escribir: no hay que usar `FUNCALL` ni `#'`:

```
; En Common LISP (Lisp-2):
(mapcar #'(lambda (x) (* x x)) '(1 2 3))

; En MYLISP (Lisp-1):
(MI-MAP (LAMBDA (X) (* X X)) '(1 2 3))
```

7.11 Funciones mutuamente recursivas

Dos funciones que se llaman la una a la otra. En MYLISP, como `DEFINE` es global, podemos definirlas en cualquier orden siempre que ambas estén definidas antes de llamar a ninguna:

```
(DEFUN PARP (N)
  (IF (= N 0)
    T
    (IMPARP (- N 1))))

(DEFUN IMPARP (N)
  (IF (= N 0)
    NIL
    (PARP (- N 1))))
```

```
MYLISP> (PARP 4)
T
MYLISP> (PARP 7)
NIL
MYLISP> (IMPARP 5)
T
```

Como ambas están en el entorno global cuando se llaman, no hay problema de referencias hacia adelante.

Continúa en el Capítulo 8: Cadenas, PRINT y LOAD

Capítulo 8 — Cadenas, PRINT y LOAD

8.1 Cadenas de texto en MYLISP

MYLISP soporta cadenas de texto como tipo de dato nativo. Una cadena es una secuencia de caracteres delimitada por comillas dobles:

```
"hola mundo"
"resultado: "
"MYLISP 1.0"
```

Las cadenas son **literales**: evalúan a sí mismas.

```
MYLISP> "hola"
"hola"
MYLISP> (DEFINE TITULO "MYLISP para QL")
TITULO
MYLISP> TITULO
"MYLISP para QL"
```

8.1.1 Limitaciones de las cadenas

MYLISP tiene límites estrictos para las cadenas:

- **Longitud máxima:** 36 caracteres por cadena.
- **Número máximo:** 50 cadenas distintas en toda la sesión (incluidas las definidas en ficheros cargados con `LOAD`).

```
MYLISP> "Esta cadena tiene exactamente 36 car"  
"Esta cadena tiene exactamente 36 car"
```

Si intentas crear una cadena de más de 36 caracteres, será truncada o producirá un error según la implementación.

¡Atención! La tabla de cadenas tiene capacidad para 50 entradas. Si tu programa crea muchas cadenas distintas (por ejemplo, cadenas generadas dinámicamente en un bucle), puedes agotarla. Reutiliza cadenas siempre que sea posible.

8.1.2 Usar cadenas como mensajes

Las cadenas son útiles para añadir texto descriptivo a los resultados:

```
(DEFUN MOSINFO (LABEL VALOR)  
  (PROGN  
    (PRINT LABEL)  
    (PRINT VALOR)))
```

```
MYLISP> (MOSINFO "Resultado:" (* 6 7))  
"Resultado:"  
42  
42
```

Nótese que `PRINT` imprime el valor y devuelve el valor impreso. La última línea `42` es el valor devuelto por la función `MOSINFO` (que devuelve lo que devuelve `PRINT VALOR`).

8.2 Funciones de salida: PRINT, DISPLAY, NEWLINE y SYMNAME

MYLISP dispone de cuatro primitivas para escribir en pantalla. Cada una tiene un comportamiento distinto que conviene conocer bien para construir salidas legibles.

8.2.1 PRINT

`PRINT` imprime su argumento en la pantalla **con el formato del lector** (las cadenas van entre comillas, los símbolos en mayúsculas) seguido de un salto de línea, y devuelve ese mismo argumento.

```
MYLISP> (PRINT 42)  
42  
42  
MYLISP> (PRINT "hola")  
"hola"  
"hola"  
MYLISP> (PRINT '(1 2 3))  
(1 2 3)  
(1 2 3)
```

La primera línea de cada bloque es lo que `PRINT` escribe; la segunda es el valor de retorno que muestra el REPL (que es el mismo valor). `PRINT` es la herramienta de depuración fundamental: como devuelve su argumento, puede insertarse en cualquier punto de una expresión sin alterar el resultado.

```
(DEFUN FACT (N)
  (PROGN
    (PRINT N)
    (IF (= N 0)
      1
      (* N (FACT (- N 1))))))
```

```
MYLISP> (FACT 4)
4
3
2
1
0
24
```

Muestra la traza de llamadas recursivas antes de devolver el resultado final.

8.2.2 DISPLAY

`DISPLAY` imprime su argumento **sin el formato del lector**: las cadenas se muestran sin comillas y los caracteres de escape se interpretan. No añade salto de línea al final. Devuelve el valor impreso.

```
MYLISP> (DISPLAY "hola")
hola
MYLISP> (DISPLAY 42)
42
MYLISP> (DISPLAY '(1 2 3))
(1 2 3)
```

La diferencia con `PRINT` se nota especialmente en las cadenas: `(PRINT "hola")` escribe `"hola"` con comillas y el REPL muestra el valor devuelto; `(DISPLAY "hola")` escribe `hola` sin comillas y **no muestra nada más** (devuelve VOID, un valor especial que el REPL no imprime). `DISPLAY` es la primitiva adecuada para construir salidas de texto destinadas al usuario final, no al depurador.

Para separar varias llamadas a `DISPLAY` en líneas distintas, usa `NEWLINE`.

8.2.3 NEWLINE

`NEWLINE` emite un salto de línea en la salida y devuelve `NIL`. No toma argumentos.

```
MYLISP> (NEWLINE)

NIL
```

Combinada con `DISPLAY`, permite construir salidas con formato preciso:

```
(DEFUN INFORMA (ETIQ VAL)
  (PROGN
    (DISPLAY ETIQ)
    (DISPLAY " ")
    (DISPLAY VAL)
    (NEWLINE)))
```

```
MYLISP> (INFORMA "Resultado:" 42)
Resultado: 42
NIL
```

Compara esto con `PRINT`: habría mostrado `"Resultado:"`, `" "` y `42` cada uno en su propia línea y entre comillas si fueran cadenas. `DISPLAY` + `NEWLINE` da control total sobre el formato.

8.2.4 SYMNAME

`SYMNAME` devuelve el nombre de un símbolo como una **cadena de texto**. Es la puerta entre el mundo de los símbolos y el de las cadenas.

```
MYLISP> (SYMNAME 'HOLA)
"HOLA"
MYLISP> (SYMNAME 'X)
"X"
MYLISP> (SYMNAME 'FACT)
"FACT"
```

`SYMNAME` es especialmente útil en el CAS cuando queremos imprimir expresiones algebraicas de forma legible. Sin `SYMNAME`, un símbolo como `x` se imprime con `PRINT` como `x` (correcto para el REPL), pero si quieres construir una cadena que contenga ese nombre —para un mensaje de error descriptivo o para generar texto— necesitas convertirlo a cadena primero.

```
(DEFUN ERR-VAR (VAR)
  (PROGN
    (DISPLAY "Variable no definida: ")
    (DISPLAY (SYMNAME VAR))
    (NEWLINE)))
```

```
MYLISP> (ERR-VAR 'ALPHA)
Variable no definida: ALPHA
NIL
```

Nota: `SYMNAME` sólo acepta símbolos. Si se le pasa un número, una lista u otro tipo, el comportamiento es indefinido. Comprueba con `SYMBOLP` antes de llamarla si el tipo del argumento no está garantizado.

8.2.5 STR<: comparación alfabética

`STR<` compara dos valores lexicográficamente y devuelve `T` si el primero precede al segundo en orden alfabético. Acepta cadenas (`TSTRING`) o símbolos (`TSYM`) indistintamente.

```
MYLISP> (STR< "ALFA" "BETA")
T
MYLISP> (STR< "ZETA" "ALFA")
NIL
MYLISP> (STR< 'ANA 'LUIS)
T
MYLISP> (STR< 'ZETA 'ALFA)
NIL
```

Cuando se le pasan símbolos directamente, `STR<` los compara sin necesidad de convertirlos a cadena con `SYMNAME`, lo que es más eficiente. Esto permite definir un comparador de símbolos en una línea:

```
(DEFUN SYM< (A B) (STR< A B))
```

Y usarlo para ordenar listas de símbolos:

```
MYLISP> (QSORT '(LUIS ANA EVA MARIO) SYM<)
```

Nota: `STR<` compara según los códigos de carácter del QL. Como MYLISP convierte todos los símbolos a mayúsculas, la comparación es siempre entre mayúsculas y produce un orden alfabético natural.

8.2.6 STRCAT: concatenación de cadenas

`STRCAT` concatena dos valores convirtiéndolos a cadena y devuelve la cadena resultante. Acepta cadenas (`TSTRING`), símbolos (`TSYM`), enteros (`TINT`) y racionales (`TRAT`). No acepta reales (`TFLOAT`). El resultado se trunca a 36 caracteres (el límite de `STRLEN`).

```
MYLISP> (STRCAT "hola" " mundo")
"hola mundo"
MYLISP> (STRCAT "N=" 42)
"N=42"
MYLISP> (STRCAT "frac=" 1/3)
"frac=1/3"
MYLISP> (STRCAT "var=" 'X)
"var=X"
```

`STRCAT` es especialmente útil para construir mensajes de error descriptivos o nombres de fichero dinámicamente:

```
(DEFUN MSGERR (TIPO DETALLE)
  (PROGN
    (DISPLAY (STRCAT "Error en " TIPO))
    (DISPLAY ": ")
    (DISPLAY DETALLE)
    (NEWLINE)))
```

```
MYLISP> (MSGERR "DERIV" "variable desconocida")
Error en DERIV: variable desconocida
NIL
```

8.2.7 Resumen comparativo de salida

Función	Salto de línea	Comillas en cadenas	Devuelve
<code>PRINT</code>	Sí	Sí	El argumento
<code>DISPLAY</code>	No	No	VOID (no visible en REPL)
<code>NEWLINE</code>	Sí (sólo eso)	—	<code>NIL</code>
<code>SYMNAME</code>	No imprime	—	Cadena con el nombre del símbolo
<code>STR<</code>	No imprime	—	<code>T</code> o <code>NIL</code>
<code>STRCAT</code>	No imprime	—	Cadena concatenada

8.3 Cargar programas con LOAD

`LOAD` lee y evalúa un fichero de código LISP. Es la forma de trabajar con programas que son demasiado largos para el REPL.

`LOAD` acepta el nombre del fichero como **cadena** o como **símbolo**:

```
(LOAD "miprog")    ; forma principal con cadena
(LOAD 'MIPROG)     ; alternativa con símbolo (truncado a 8 caracteres)
```

La convención de nombres depende de la plataforma:

QL: los ficheros llevan el prefijo del dispositivo: `"mdv1_nombre"` (microdrive 1) o `"mdv2_nombre"` (microdrive 2). La forma con cadena es la recomendada porque el guión bajo en el nombre no es válido en un símbolo LISP. **Next:** los ficheros van en la tarjeta SD con nombres al estilo Unix: `"utiles.lsp"`, `"cas.lsp"`, etc. La extensión es opcional pero ayuda a identificar los ficheros.

8.3.1 Formato de los ficheros LISP

Un fichero LISP es simplemente texto. Contiene expresiones LISP separadas por espacios o saltos de línea. MYLISP lee y evalúa cada expresión en orden:

```
; Fichero de ejemplo (QL: mdv1_aritmet / Next: aritmet.lsp)

(DEFUN CUADRA (X) (* X X))
(DEFUN CUBO (X) (* X X X))
(DEFUN SUMCUA (A B) (+ (CUADRA A) (CUADRA B)))

(PRINT "Funciones aritmeticas cargadas")
```

Al cargar este fichero:

```
MYLISP> (LOAD "mdv1_aritmet")
"Funciones aritmeticas cargadas"
T
```

El `T` final indica que `LOAD` se completó sin errores.

8.3.2 GC durante LOAD

El comportamiento del GC durante `LOAD` varía entre plataformas:

QL: el GC se activa cuando el heap alcanza el **50%** de su capacidad (en lugar del 80% habitual). Esto es una medida de seguridad: los ficheros de código pueden definir muchas funciones y estructuras, y el GC más frecuente garantiza que haya espacio suficiente. **Next:** el GC **no se dispara** automáticamente durante `LOAD`. Se confía en que el heap de 32.000 celdas es suficiente para cualquier fichero razonable. Si quieres forzar una recolección entre cargas, usa `(CLEAN)` desde el REPL entre una carga y la siguiente.

8.3.3 Organizar el código en ficheros

Para proyectos grandes (como el CAS del capítulo 10), es buena práctica dividir el código en módulos y cargarlos en orden:

```
; Fichero principal (QL: mdv1_cas / Next: cas.lsp)

(LOAD "mdv1_utiles")      ; QL  - utilidades de lista
; (LOAD "utiles.lsp")    ; Next - mismas utilidades
(LOAD "mdv1_aritmB")      ; aritmética básica
(LOAD "mdv1_deriv")       ; derivador simbólico
(LOAD "mdv1_simplif")     ; simplificador

(PRINT "CAS cargado y listo")
```

8.3.4 Nombres de fichero y el límite de SYMLEN

Recuerda que los nombres de fichero en `LOAD` son **cadenas**, no símbolos. Las cadenas no están sujetas al límite de 8 caracteres de los símbolos. Sin embargo, el sistema de ficheros QDOS tiene sus propias restricciones de longitud de nombre.

```
MYLISP> (LOAD "mdv1_utilidades-de-lista")
```

Esto es válido siempre que el nombre quepa en el sistema de ficheros de la plataforma.

8.3.5 ¿Qué ocurre si hay un error durante LOAD?

Si se produce un error en alguna expresión del fichero, MYLISP muestra el mensaje de error y **continúa evaluando el resto del fichero**. Las funciones definidas antes del error quedan disponibles; las posteriores al error pueden estar incompletas.

Por eso es importante probar las funciones individualmente en el REPL antes de organizarlas en un fichero.

8.4 Flujo de trabajo típico con ficheros

El flujo recomendado para desarrollar con MYLISP es el siguiente:

1. **Diseñar** la función en papel o en la cabeza.
2. **Probar** en el REPL con casos simples:

```
MYLISP> (DEFUN HOLA (X) (+ X 1))
MYLISP> (HOLA 5)
6
```

3. **Escribir** el código en un fichero usando el editor de texto disponible en tu plataforma (Quill o el editor de SuperBASIC en el QL; cualquier editor de texto en el Next o en el PC si usas CSpect).
4. **Cargar** el fichero con `LOAD`:

```
MYLISP> (LOAD "mdv1_mifich")      ; QL
MYLISP> (LOAD "mifich.lsp")      ; Next
```

5. **Probar** las funciones cargadas en el REPL.
6. **Corregir** el fichero y repetir desde el paso 4 si hay errores.

8.5 Sesión completa: un programa de tablas

Como ejemplo completo de uso de `PRINT` y `LOAD`, aquí está un pequeño programa que genera la tabla de multiplicar:


```
; Fichero: mdv1_tablas (QL) / tablas.lsp (Next)
```

```
(DEFUN TABFILA (N M)
  (COND
    ((> M 10) NIL)
    (T (PROGN
        (PRINT (* N M))
        (TABFILA N (+ M 1))))))

(DEFUN TABLA (N)
  (PROGN
    (PRINT "Tabla del:")
    (PRINT N)
    (TABFILA N 1)))
```

Cargamos y usamos:

```
MYLISP> (LOAD "mdv1_tablas") ; QL
MYLISP> (LOAD "tablas.lsp") ; Next
"Tablas cargadas"
MYLISP> (TABLA 7)
"Tabla del:"
7
7
14
21
28
35
42
49
56
63
70
NIL
```

El `NIL` final es el valor de retorno cuando `M > 10` en el caso base.

8.6 Resumen de limitaciones prácticas

Al trabajar con cadenas y ficheros, ten en cuenta:

- Cadenas: máximo 36 caracteres, máximo 50 en memoria simultáneamente.
- `LOAD`: el nombre del fichero debe incluir el prefijo de dispositivo (`mdv1_`, `mdv2_`).
- El GC se activa al 50% durante `LOAD` (vs 80% en el REPL).
- No existe `NEWLINE` como función primitiva; el único salto de línea al imprimir lo produce `PRINT`.
- No hay función para leer desde teclado dentro de un programa; toda la entrada interactiva pasa por el REPL.

Continúa en el Capítulo 9: Galería de programas

Capítulo 9 — Galería de programas

9.1 Programar en MYLISP: el estilo funcional

Este capítulo reúne programas completos escritos en MYLISP. Cada uno ilustra técnicas que puedes reutilizar en tus propios proyectos. El estilo es funcional: las funciones no modifican sus argumentos, devuelven nuevos valores, y la composición de funciones sencillas construye soluciones a problemas complejos.

Todos los programas están pensados para cargarse desde fichero con `LOAD`. Los nombres tienen 8 caracteres o menos.

9.2 Factorial: tres versiones

El **factorial** de un número natural n , escrito $n!$, es el producto de todos los enteros positivos desde 1 hasta n : $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$, con el caso especial $0! = 1$ (el producto vacío). Así, $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$. Es la función que cuenta, por ejemplo, de cuántas formas distintas se pueden ordenar n objetos en fila, y aparece constantemente en combinatoria y en las series de Taylor.

La definición matemática ya es recursiva — $n! = n \cdot (n-1)!$, con caso base $0! = 1$ — así que se traduce a LISP casi palabra por palabra. Veamos tres formas de escribir esa misma idea, cada una con un matiz distinto.

9.2.1 Versión directa

```
(DEFUN FACT (N)
  (IF (= N 0) 1 (* N (FACT (- N 1)))))
```

Simple y clara. Construye una cadena de multiplicaciones en la pila de llamadas.

9.2.2 Versión con acumulador

```
(DEFUN FACTA (N ACC)
  (IF (= N 0) ACC (FACTA (- N 1) (* N ACC))))

(DEFUN FACT (N) (FACTA N 1))
```

El resultado se acumula en `ACC`. En cada llamada, el resultado parcial ya está calculado. Conceptualmente equivalente pero con estructura de cola.

9.2.3 Versión con COND para validar entrada

```
(DEFUN FACT (N)
  (COND
    ((< N 0) (PRINT "Error: negativo"))
    ((= N 0) 1)
    (T (* N (FACT (- N 1)))))
```

```
MYLISP> (FACT 6)
720
MYLISP> (FACT -1)
"Error: negativo"
"Error: negativo"
```

9.3 Números de Fibonacci

La **sucesión de Fibonacci** es una de las secuencias numéricas más conocidas: cada término es la suma de los dos anteriores, empezando por 0 y 1:

```
F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, F(6)=8, F(7)=13, F(8)=21, F(9)=34, F(10)=55 ...
```

Es decir: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... — cada número se obtiene sumando los dos que le preceden ($3 = 1 + 2$, $5 = 2 + 3$, $8 = 3 + 5$, y así sucesivamente). La definición matemática es la recurrencia:

$$F(n) = F(n-1) + F(n-2), \quad \text{con } F(0) = 0 \text{ y } F(1) = 1$$

Leonardo de Pisa (apodado *Fibonacci*) la popularizó en Europa en 1202 en su obra *Liber Abaci*, usándola para modelar el crecimiento de una población de conejos. Hoy reaparece en contextos muy distintos: en botánica (la disposición de las semillas de un girasol), en la razón áurea (el cociente $F(n+1)/F(n)$ converge a $\phi \approx 1,618$), y —lo que más nos interesa aquí— como el ejemplo canónico para ilustrar la diferencia entre una recursión "ingenua" y una recursión eficiente.

9.3.1 Versión directa (exponencial)

```
(DEFUN FIBON (N)
  (COND
    ((= N 0) 0)
    ((= N 1) 1)
    (T (+ (FIBON (- N 1)) (FIBON (- N 2))))))
```

Esta versión traduce la recurrencia matemática casi letra por letra, y por eso es la más fácil de leer. Pero es engañosamente cara: para calcular `(FIBON 5)` necesita `(FIBON 4)` y `(FIBON 3)`; para calcular `(FIBON 4)` vuelve a necesitar `(FIBON 3)` (¡otra vez!) y `(FIBON 2)`; y así sucesivamente. El mismo subproblema se recalcula una y otra vez, un número de veces que crece exponencialmente con `N`. Elegante pero lenta para `N` grande: calcula los mismos valores repetidamente.

```
MYLISP> (FIBON 10)
55
MYLISP> (FIBON 15)
610
```

9.3.2 Versión con acumulador (lineal)

```
(DEFUN FIBONA (N A B)
  (COND
    ((= N 0) A)
    ((= N 1) B)
    (T (FIBONA (- N 1) B (+ A B)))))

(DEFUN FIBON (N) (FIBONA N 0 1))
```

Aquí `A` y `B` son siempre dos Fibonacci consecutivos: al empezar, $A = F(0) = 0$ y $B = F(1) = 1$. En cada llamada recursiva, el par avanza una posición: el nuevo `A` es el antiguo `B`, y el nuevo `B` es $A + B$ —el siguiente término de la serie—. No hay ninguna llamada que se repita: `(FIBONA 5 0 1)` visita el par `(0,1)`, luego `(1,1)`, luego `(1,2)`, luego `(2,3)`, luego `(3,5)`, y devuelve `5`. Esta versión es $O(n)$: calcula cada número de Fibonacci una sola vez, en lugar de recalcularlo un número exponencial de veces.

```
MYLISP> (FIBON 20)
6765
MYLISP> (FIBON 25)
75025
```

9.3.3 Generar una lista de Fibonacci

```
(DEFUN FIBLIST (N)
  (COND
    ((= N 0) (LIST 0))
    (T (APPEND (FIBLIST (- N 1))
                (LIST (FIBON N))))))
```

```
MYLISP> (FIBLIST 8)
(0 1 1 2 3 5 8 13 21)
```

9.4 Máximo común divisor

El **máximo común divisor** de dos enteros A y B , escrito $\text{mcd}(A, B)$, es el mayor entero que divide exactamente a ambos. Por ejemplo, $\text{mcd}(48, 18) = 6$, porque 6 divide a los dos y no hay ningún divisor común mayor.

El **algoritmo de Euclides** —descrito en los *Elementos* de Euclides hace más de 2000 años, y considerado uno de los algoritmos más antiguos que se conocen— calcula el mcd sin necesidad de factorizar ningún número, apoyándose en una única observación: **el mcd de dos números no cambia si sustituimos el mayor por el resto de dividirlo entre el menor**. Formalmente:

$$\begin{aligned} \text{mcd}(A, B) &= \text{mcd}(B, A \bmod B) && (\text{si } B \neq 0) \\ \text{mcd}(A, 0) &= A && (\text{caso base}) \end{aligned}$$

Repitiendo esa sustitución, los números se hacen cada vez más pequeños hasta que uno de ellos llega a 0 —y en ese momento el otro es el mcd buscado. Trazado a mano para $\text{mcd}(48, 18)$:

$$\begin{aligned} \text{mcd}(48, 18) &= \text{mcd}(18, 48 \bmod 18) = \text{mcd}(18, 12) \\ &= \text{mcd}(12, 18 \bmod 12) = \text{mcd}(12, 6) \\ &= \text{mcd}(6, 12 \bmod 6) = \text{mcd}(6, 0) \\ &= 6 \end{aligned}$$

La traducción a MYLISP es directa gracias a la primitiva `MOD` (sección 4.9), que devuelve exactamente el resto de la división entera que necesita la recurrencia:

```
(DEFUN MCD (A B)
  (COND
    ((= B 0) A)
    (T (MCD B (MOD A B)))))
```

Cada llamada recursiva es exactamente un paso de la traza de arriba: $(\text{MCD } B \text{ (MOD } A \text{ } B))$ sustituye el par (A, B) por $(B, A \bmod B)$, tal como exige la fórmula. Cuando B llega a 0 , la cláusula base devuelve A —que en ese punto ya es el máximo común divisor.

```
MYLISP> (MCD 48 18)
6
MYLISP> (MCD 100 75)
25
MYLISP> (MCD 17 13)
1
```

El último ejemplo, $\text{mcd}(17, 13) = 1$, muestra dos números **coprimos**: no comparten ningún factor salvo el 1 .

9.4.1 Mínimo común múltiplo

El **mínimo común múltiplo** de A y B , $\text{mcm}(A, B)$, es el menor entero positivo que es múltiplo de ambos a la vez. En vez de buscarlo por fuerza bruta, se apoya en una identidad que relaciona mcd y mcm : el producto de los dos números es siempre igual al producto de su mcd por su mcm ($A \cdot B = \text{mcd}(A, B) \cdot \text{mcm}(A, B)$), así que basta con despejar:

```
(DEFUN MCM (A B)
  (/ (* A B) (MCD A B)))
```

```
MYLISP> (MCM 4 6)
12
MYLISP> (MCM 15 10)
30
```

Verifiquemos la identidad con el primer caso: $\text{mcd}(4, 6) = 2$ y $\text{mcm}(4, 6) = 12$, y en efecto $2 \cdot 12 = 24 = 4 \cdot 6$.

9.5 Ordenación de listas

Ordenar una lista significa reorganizar sus elementos de menor a mayor. Veremos dos estrategias clásicas y muy distintas entre sí: la **ordenación por inserción**, que construye el resultado insertando un elemento cada vez en el lugar que le corresponde, y **quicksort**, que divide el problema en trozos más pequeños y los resuelve por separado.

9.5.1 Inserción ordenada

```
(DEFUN INSORT (X L)
  (COND
    ((NULL L) (LIST X))
    (<= X (CAR L)) (CONS X L))
    (T (CONS (CAR L) (INSORT X (CDR L))))))
```

Inserta x en la posición correcta de la lista ya ordenada L :

```
MYLISP> (INSORT 4 '(1 3 5 7))
(1 3 4 5 7)
```

9.5.2 Ordenación por inserción

La idea de la **ordenación por inserción** es la misma que usa cualquiera para ordenar cartas en la mano: se toma una carta de la baraja y se inserta en su sitio entre las que ya están ordenadas; se repite con la siguiente carta; y así hasta que no queda ninguna por colocar. `ISORT` traduce esto a recursión: para ordenar toda la lista, primero se ordena recursivamente el resto (`CDR L`), y luego se inserta la cabeza (`CAR L`) en el lugar correcto de ese resto ya ordenado, usando `INSORT`.

```
(DEFUN ISORT (L)
  (COND
    ((NULL L) NIL)
    (T (INSORT (CAR L) (ISORT (CDR L))))))
```

```
MYLISP> (ISORT '(5 3 8 1 9 2 6))
(1 2 3 5 6 8 9)
MYLISP> (ISORT '(3 1 4 1 5 9 2 6))
(1 1 2 3 4 5 6 9)
```

Es sencilla de entender y de programar, pero su coste crece con el cuadrado del tamaño de la lista ($O(n^2)$): cada inserción puede recorrer, en el peor caso, toda la parte ya ordenada.

9.5.3 Quicksort

Quicksort sigue una estrategia distinta, de "dividir y vencer": en vez de insertar un elemento cada vez, elige un elemento cualquiera de la lista como **pivote** (aquí, siempre el primero), separa el resto en dos grupos —los menores que el pivote y los mayores o iguales—, ordena cada grupo por separado (recursivamente, con el mismo algoritmo), y finalmente concatena: menores ordenados, pivote, mayores ordenados. `MENORES` y `MAYORES` son los dos filtros que construyen esos grupos; `QSORT` los combina.

```
(DEFUN MENORES (X L)
  (COND
    ((NULL L) NIL)
    ((< (CAR L) X) (CONS (CAR L) (MENORES X (CDR L))))
    (T (MENORES X (CDR L)))))

(DEFUN MAYORES (X L)
  (COND
    ((NULL L) NIL)
    ((>= (CAR L) X) (CONS (CAR L) (MAYORES X (CDR L))))
    (T (MAYORES X (CDR L)))))

(DEFUN QSORT (L)
  (COND
    ((NULL L) NIL)
    ((NULL (CDR L)) L)
    (T (APPEND
        (QSORT (MENORES (CAR L) (CDR L)))
        (LIST (CAR L))
        (QSORT (MAYORES (CAR L) (CDR L)))))))
```

```
MYLISP> (QSORT '(3 6 8 10 1 2 1))
(1 1 2 3 6 8 10)
MYLISP> (QSORT '(5 4 3 2 1))
(1 2 3 4 5)
```

Trazado a mano, `(QSORT '(3 6 8 1))` toma `3` como pivote, separa `(1)` (menores) de `(6 8)` (mayores o iguales), ordena cada grupo por separado —trivial aquí, porque son listas de un solo elemento o ya ordenadas— y reconstruye `(1) ++ (3) ++ (6 8) = (1 3 6 8)`. En el caso medio, quicksort es mucho más rápido que la ordenación por inserción ($O(n \log n)$ frente a $O(n^2)$), pero como aquí el pivote es siempre el primer elemento, una lista que ya viniera ordenada (o inversamente ordenada) produciría el peor caso posible, también $O(n^2)$.

9.6 Búsqueda en listas

9.6.1 Búsqueda lineal

La **búsqueda lineal** es la estrategia más básica posible: recorrer la lista elemento a elemento, comparando cada uno con lo que buscamos, hasta encontrarlo o llegar al final. No necesita que la lista esté ordenada, pero en el peor caso —el elemento no está, o está al final— tiene que mirar todos los elementos: $O(n)$.

```
(DEFUN BUSCA (X L)
  (COND
    ((NULL L) NIL)
    ((EQUAL X (CAR L)) T)
    (T (BUSCA X (CDR L)))))
```

```
MYLISP> (BUSCA 5 '(1 3 5 7 9))
T
MYLISP> (BUSCA 4 '(1 3 5 7 9))
NIL
```

9.6.2 Búsqueda y devolver posición

```
(DEFUN POSIC (X L N)
  (COND
    ((NULL L) -1)
    ((EQUAL X (CAR L)) N)
    (T (POSIC X (CDR L) (+ N 1)))))

(DEFUN DONDE (X L) (POSIC X L 0))
```

```
MYLISP> (DONDE 'C '(A B C D E))
2
MYLISP> (DONDE 'Z '(A B C D E))
-1
```

9.6.3 Búsqueda binaria en lista ordenada

Cuando los datos están **ordenados**, no hace falta mirarlos uno a uno: la **búsqueda binaria** compara el elemento buscado con el que está justo en el medio de la colección. Si coincide, ya está. Si es menor, el elemento solo puede estar en la mitad izquierda; si es mayor, solo puede estar en la mitad derecha. Cada comparación descarta la mitad de lo que quedaba, así que el número de pasos crece con el logaritmo del tamaño ($O(\log n)$) en vez de con el tamaño mismo — sobre un array, buscar entre un millón de elementos ordenados solo necesita unas 20 comparaciones.

El problema es que esa eficiencia depende de poder saltar directamente al elemento del medio, y una lista enlazada de LISP no lo permite: para llegar al elemento N-ésimo hay que recorrer los N-1 anteriores con `CDR`, uno a uno.

```
(DEFUN NTHEL (L N)
  (IF (= N 0)
    (CAR L)
    (NTHEL (CDR L) (- N 1))))

(DEFUN MITAD (L)
  (NTHEL L (/ (LONGIT L) 2)))
```

`NTHEL` obtiene el elemento N-ésimo recorriendo la lista, y `MITAD` la usa para acceder al elemento central — pero ese propio acceso ya cuesta $O(n)$, lo mismo que recorrer la lista entera. El "ahorro" de la búsqueda binaria se pierde exactamente en el paso que debería ser instantáneo. Por eso decimos que la búsqueda binaria sobre listas es costosa: en la práctica no mejora a la búsqueda lineal. Para aprovechar de verdad la idea de "descartar la mitad en cada paso" hace falta una estructura con acceso directo, como los árboles de búsqueda (sección 5.8), donde bajar a la izquierda o a la derecha sí es una operación de coste constante.

9.7 MAP, FILTER y REDUCE definidos en LISP puro

Estas tres funciones de orden superior son fundamentales en programación funcional. MYLISP no las incluye como primitivas, pero podemos definirlas con unas pocas líneas.

9.7.1 MY-MAP: transformar cada elemento

```
(DEFUN MY-MAP (F L)
  (COND
    ((NULL L) NIL)
    (T (CONS (F (CAR L))
              (MY-MAP F (CDR L))))))
```

```
MYLISP> (MY-MAP (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
MYLISP> (MY-MAP (LAMBDA (X) (CONS X (LIST (* X X))))
              '(1 2 3 4))
((1 1) (2 4) (3 9) (4 16))
```

9.7.2 MY-FILTER: seleccionar elementos

```
(DEFUN MY-FILTER (PRED L)
  (COND
    ((NULL L) NIL)
    ((PRED (CAR L))
     (CONS (CAR L) (MY-FILTER PRED (CDR L))))
    (T (MY-FILTER PRED (CDR L)))))
```

```
MYLISP> (MY-FILTER (LAMBDA (X) (> X 3)) '(1 2 3 4 5 6))
(4 5 6)
MYLISP> (MY-FILTER (LAMBDA (X) (= 0 (MOD X 2)))
              '(1 2 3 4 5 6 7 8 9 10))
(2 4 6 8 10)
```

El segundo ejemplo filtra los números pares usando la primitiva `MOD` (explicada en la sección 4.9).

9.7.3 MY-REDUC: reducir a un valor acumulado

```
(DEFUN MY-REDUC (F INIT L)
  (COND
    ((NULL L) INIT)
    (T (MY-REDUC F
                  (F INIT (CAR L))
                  (CDR L)))))
```

```
MYLISP> (MY-REDUC + 0 '(1 2 3 4 5))
15
MYLISP> (MY-REDUC * 1 '(1 2 3 4 5))
120
MYLISP> (MY-REDUC (LAMBDA (A X) (CONS X A)) NIL '(1 2 3))
(3 2 1)
```

El último ejemplo invierte una lista usando `REDUCE`.

9.7.4 Composición de MAP, FILTER y REDUCE


```
; Suma de cuadrados de los impares de 1 a 10
MYLISP> (MY-REDUC
+
0
(MY-MAP
  (LAMBDA (X) (* X X))
  (MY-FILTER
    (LAMBDA (X) (NOT (= 0 (- X (* (/ X 2) 2)))))
    '(1 2 3 4 5 6 7 8 9 10))))
165
```

Los impares de 1 a 10 son (1 3 5 7 9). Sus cuadrados son (1 9 25 49 81). Su suma es 165.

9.8 Raíz cuadrada por el método de Newton

El **método de Newton** (o de Newton-Raphson) es una técnica general para aproximar raíces de una función mediante sucesivas mejoras de una estimación inicial. Para calcular $\sqrt{x}$, planteamos el problema como "encontrar la raíz de $f(G) = G^2 - x$ " y aplicamos la fórmula general de Newton, que produce esta regla de actualización:

$$G_{\text{nuevo}} = (G + x/G) / 2$$

Es decir: si G es una estimación de $\sqrt{x}$, la media entre G y x/G es una estimación *mejor*. Intuitivamente, si G se queda corto, x/G se pasa (y viceversa), así que promediar los acerca a ambos por el medio. Repitiendo el proceso, G converge muy rápido al valor real — el número de dígitos correctos aproximadamente se duplica en cada paso.

Trazado a mano para $\sqrt{2}$, partiendo de $G = 1.0$:

```
G0 = 1.0
G1 = (1.0 + 2.0/1.0) / 2 = 1.5
G2 = (1.5 + 2.0/1.5) / 2 = 1.41666...
G3 = (1.41666... + 2.0/1.41666...) / 2 = 1.41421...
```

En tres pasos ya coincide con $\sqrt{2} \approx 1.41421356$ en cinco cifras decimales. Como `SQRT` no está implementada como primitiva en MYLISP (Apéndice B.7), podemos aproximarla con este método:

```
(DEFUN BUENA (G X)
  (< (ABSOL (- (* G G) X)) 0.001))

(DEFUN MEJORG (G X)
  (/ (+ G (/ X G)) 2.0))

(DEFUN SQRTITR (G X)
  (IF (BUENA G X)
      G
      (SQRTITR (MEJORG G X) X)))

(DEFUN MYSQRT (X)
  (SQRTITR 1.0 X))

(DEFUN ABSOL (X)
  (IF (< X 0) (- X) X))
```

```

MYLISP> (MYSQRT 2.0)
1.41422...
MYLISP> (MYSQRT 9.0)
3.00009...
MYLISP> (MYSQRT 25.0)
5.00000...

```

La aproximación converge en pocos pasos. La función `BUENA` comprueba si el cuadrado de la estimación está suficientemente cerca del valor objetivo.

9.9 Invertir una lista (versión eficiente)

Invertir una lista significa producir una nueva lista con los mismos elementos en orden contrario: `(1 2 3 4 5)` invertida es `(5 4 3 2 1)`. La versión del capítulo 5 era $O(n^2)$ por usar `APPEND` en cada paso (`APPEND` tiene que recorrer entero su primer argumento, sección 5.3). Esta versión con acumulador evita `APPEND` por completo: en cada paso, mueve la cabeza de `L` a la cabeza de `ACC` con `CONS`, que es una operación de coste constante. El resultado es $O(n)$:

```

(DEFUN INVERTA (L ACC)
  (IF (NULL L)
      ACC
      (INVERTA (CDR L) (CONS (CAR L) ACC)))))

(DEFUN INVERT (L)
  (INVERTA L NIL))

```

```

MYLISP> (INVERT '(1 2 3 4 5))
(5 4 3 2 1)
MYLISP> (INVERT '(A B C D))
(D C B A)
MYLISP> (INVERT NIL)
NIL

```

9.10 Aplanar una lista anidada

"Aplanar" una lista significa recorrer una estructura con sublistas anidadas a cualquier profundidad y producir una única lista plana con todos los átomos que contenía, en el mismo orden. `APLANA` distingue dos casos al recorrer la lista: si la cabeza es un átomo (no una lista), se conserva tal cual y se sigue con el resto; si la cabeza es a su vez una lista, hay que aplanarla también y concatenar el resultado con el resto aplanado. Convertir una lista con sublistas en una lista plana:

```

(DEFUN APLANA (L)
  (COND
    ((NULL L) NIL)
    ((ATOM (CAR L))
     (CONS (CAR L) (APLANA (CDR L))))
    (T (APPEND (APLANA (CAR L))
                (APLANA (CDR L))))))

```

```

MYLISP> (APLANA '(1 (2 3) (4 (5 6)) 7))
(1 2 3 4 5 6 7)
MYLISP> (APLANA '((A B) (C (D E))))
(A B C D E)

```

9.11 Primos: criba de Eratóstenes

Un número **primo** es un entero mayor que 1 que solo es divisible por 1 y por sí mismo. La **criba de Eratóstenes**, atribuida al matemático griego Eratóstenes de Cirene (siglo III a.C.), es uno de los métodos más antiguos y elegantes para encontrar todos los primos hasta un límite dado: se parte de la lista de todos los enteros desde 2 hasta N, y se va tomando el primer número que quede (que siempre es primo, porque nada menor que él lo ha eliminado), eliminando de la lista todos sus múltiplos, y repitiendo con el siguiente número que sobreviva.

Trazado a mano para los primos hasta 20: partimos de (2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20). Tomamos 2 (primo) y eliminamos sus múltiplos: queda (3 5 7 9 11 13 15 17 19). Tomamos 3 (primo) y eliminamos sus múltiplos (9, 15): queda (5 7 11 13 17 19). Todos los que sobreviven a partir de aquí son ya primos, porque cualquier número compuesto menor o igual a 20 tiene que tener un factor menor o igual a $\sqrt{20} \approx 4,5$, y ya hemos cribado por 2 y por 3.

La implementación traduce ese proceso directamente: `SINMUL` filtra los múltiplos de un número dado (usando `MOD`, sección 4.9), `RANGO` genera la lista inicial de candidatos, y `CRIBA` aplica el proceso completo, tomando siempre la cabeza de la lista como el siguiente primo confirmado:

```
; Filtra múltiplos de N de una lista usando MOD (sección 4.9)
(DEFUN SINMUL (N L)
  (MY-FILTR
    (LAMBDA (X) (NOT (= 0 (MOD X N))))
    L))

; Genera lista de enteros de A hasta B
(DEFUN RANGO (A B)
  (IF (> A B)
    NIL
    (CONS A (RANGO (+ A 1) B))))

; Criba de Eratóstenes
(DEFUN CRIBA (L)
  (COND
    ((NULL L) NIL)
    (T (CONS (CAR L)
              (CRIBA (SINMUL (CAR L) (CDR L)))))))

(DEFUN PRIMOS (N)
  (CRIBA (RANGO 2 N)))
```

```
MYLISP> (PRIMOS 30)
(2 3 5 7 11 13 17 19 23 29)
MYLISP> (PRIMOS 50)
(2 3 5 7 11 13 17 19 23 29 31 37 41 43 47)
```

Continúa en la Parte II: Diseño y Estructura de un CAS

Parte II — Cálculo simbólico: Construyendo un CAS

Capítulo 10 — Representación y abstracción de datos

En una frase: vamos a representar expresiones matemáticas como listas de Lisp sin evaluar, y a construir una interfaz de acceso a esas listas (predicados, selectores, constructores) para que ninguna función futura del CAS tenga que manipular la estructura de la lista a mano.

10.1 Motivación: qué es un CAS, y por qué en LISP

Seguro que has visto alguna calculadora, una app de deberes de matemáticas, o un buscador al que le escribes `derivada de  $x^2$` y te devuelve, sin dudar, `2x`. O le pides que simplifique `(x + 1)^2 - (x^2 + 2x)` y te contesta `1`. No es magia (aunque a mi de joven me lo parecía), ni el programa "sabe" matemáticas como las sabe una persona: es **álgebra simbólica** — manipular expresiones matemáticas como estructuras, aplicando reglas mecánicas, sin necesidad de saber a qué número concreto corresponde `x`.

Ahí está la diferencia con lo que ya sabes hacer en MYLISP desde el capítulo 4: `(+ 2 3)` es **cálculo numérico** — el intérprete sustituye, opera, y devuelve `5`. Pero `x + x → 2x` no involucra ningún número: `x` no vale nada concreto, y aun así sabemos con certeza que la expresión se puede reescribir de forma equivalente y más simple. Lo mismo pasa con derivar: la regla de la potencia dice que la derivada de `xn` es `n · xn-1` — una transformación puramente sintáctica sobre la *forma* de la expresión, no sobre su valor numérico. Un **CAS** (*Computer Algebra System*, sistema de álgebra computacional) es, precisamente, un programa que sabe aplicar esas transformaciones: derivar, simplificar, y —aunque no lo cubre este manual— integrar, factorizar o resolver ecuaciones, todo manteniendo las expresiones en forma simbólica exacta en vez de aproximarlas con números.

Esto no es un ejercicio académico aislado: los CAS existen desde hace décadas y son herramientas de trabajo real en ciencia e ingeniería — Macsyma (Matlab, Mathematica, Maple, Maxima, SymPy son parientes o descendientes suyos, cada uno con su propia historia). Y aquí se cierra un círculo con la Parte I de este manual: **Macsyma**, desarrollado desde 1968 en el MIT, uno de los primeros y más influyentes sistemas de álgebra simbólica de la historia, estaba escrito en **MacLisp** — el mismo dialecto del que MYLISP toma buena parte de su sintaxis (§1.4.2). No es casualidad. LISP fue, desde su origen en 1958, una notación pensada para representar y manipular expresiones simbólicas (§1.1.1) — y eso es exactamente lo que necesita un CAS.

10.1.1 Por qué LISP encaja tan bien con este problema

Un CAS necesita, ante todo, una forma de representar una expresión matemática como una estructura de datos que el programa pueda inspeccionar y transformar — lo que en informática se llama un **árbol de sintaxis abstracta** (AST). En la mayoría de lenguajes, construir eso exige diseñar un tipo de dato nuevo, escribir un analizador sintáctico, y mantener funciones separadas para "código" y para "datos". En LISP no hace falta nada de eso, por una propiedad que ya viste en el capítulo 3: la **homoiconicidad** (§3.6) — el código y los datos comparten la misma sintaxis, la S-expresión. La lista `(+ x 3)` ya es un árbol: `+` es la operación, y `x` y `3` son sus dos ramas. No hay que inventar nada; solo hay que decidir, como haremos en el §10.2, qué convenio seguir para organizar esas listas.

El resto de lo que necesitamos ya lo construimos en la Parte I, y lo usaremos sin descanso a partir de aquí:

- **QUOTE** (§3.5) para poder escribir `'(+ x 3)` como un dato, sin que el intérprete intente evaluarlo como una llamada a `+`.
- **La recursión** (§7.6) para recorrer expresiones anidadas: derivar una suma requiere derivar cada uno de sus sumandos, que a su vez pueden ser sumas o productos — la misma idea de "resolver el caso base, combinar con la solución del resto" que ya usaste para sumar listas o invertirlos.

- **Las listas y sus predicados, selectores y constructores** (capítulo 5) para clasificar, extraer y reconstruir las piezas de una expresión, sin tocar `CAR` / `CDR` a pelo en cada función nueva (una regla que se convertirá en ley formal en el §10.3).
- **COND** (§6.3) para las cascadas de casos que aparecen constantemente al recorrer una expresión: "si es un número, hacer esto; si es una suma, hacer aquello otro; si es un producto, ...".
- **LAMBDA y las clausuras** (§7.2, §7.9) para las funciones de orden superior que aparecerán al recorrer y transformar listas de términos.

En otras palabras: la Parte II no introduce un lenguaje nuevo ni trucos especiales. Construye un CAS pequeño pero real — capaz de derivar y simplificar expresiones algebraicas— combinando, capítulo a capítulo, las mismas piezas de MYLISP que ya conoces. El objetivo final, que iremos alcanzando pieza a pieza a lo largo de los capítulos 10 a 19, se ve así:

```
MYLISP> (DERIVA '(POT X 2) 'X)
(* 2 X)
```

Eso es exactamente el $x^2 \rightarrow 2x$ con el que empezaba esta sección — solo que ahora sabrás, línea a línea, cómo se construye.

10.2 La decisión que lo condiciona todo: cómo representamos una expresión

Vamos a representar una expresión matemática como una lista de MyLISP **sin evaluar**. Es decir, $x + 3$ se representará literalmente como la lista `(+ x 3)`, pero nunca se la pasaremos al evaluador tal cual — siempre irá citada con `QUOTE` (o `'`), porque si no, el propio intérprete la evaluaría como una llamada a la función `+` y perderíamos la expresión simbólica.

```
(DEFINE EXPR1 '(+ X 3)) ; EXPR1 es un DATO, no se evalúa
EXPR1 ; -> (+ X 3)
```

Esto es la base de todo: en MyLISP, código y datos comparten la misma sintaxis (S-expressions), así que un AST algebraico "nos sale gratis" — no necesitamos inventar un formato nuevo, solo *decidir un convenio* sobre cómo se organizan esas listas.

10.2.1 Convenio para el capítulo 10 (binario, sin simplificar todavía)

Expresión matemática	Representación MyLISP
constante	número: <code>3</code> , <code>1/2</code> , <code>2.5</code>
variable	símbolo: <code>x</code> , <code>y</code>
suma <code>a + b</code>	<code>(+ a b)</code>
producto <code>a * b</code>	<code>(* a b)</code>
potencia <code>a^b</code>	<code>(POT a b)</code>

Nota sobre la potencia: no podemos usar `^` como operador porque el lexer de MyLISP solo reconoce como símbolo los caracteres alfanuméricos más `-`, `?`, `!` (y los operadores aritméticos ya predefinidos como `+`, `*`, `/`). `^` produciría un error léxico. Por eso usamos la palabra `POT`.

De momento trabajamos **solo binario** (`(+ a b)`, no `(+ a b c)`), aunque el `+` real de MyLISP sea variádico — es una decisión pedagógica deliberada, igual que en SICP. Ampliar a variádico será un ejercicio natural más adelante, una vez el simplificador exista.

10.3 La regla de oro: nunca tocar CAR/CDR directamente fuera de este capítulo

A partir de aquí, **ninguna otra función del CAS accederá a la estructura de la lista directamente**. Todo pasará por constructores, selectores y predicados. Esta es la "barrera de abstracción": si mañana decidimos cambiar la representación (por ejemplo, `(SUMA a b)` en vez de `(+ a b)`, o pasar a listas etiquetadas), solo hay que reescribir este capítulo — nada más se entera del cambio.

10.4 Nota de plataforma: la trampa de los 8 caracteres

MyLISP trunca los nombres de símbolo a **8 caracteres significativos, en silencio, sin avisar**. Si usáramos la nomenclatura clásica de SICP tal cual (`MULTIPLIER` / `MULTIPLICAND`), ambos se truncarían a `MULTIPLI` y **colisionarían** — pasarían a ser el mismo símbolo, con resultados incorrectos y sin ningún mensaje de error. Por eso este manual usa desde el principio una tabla de nombres cortos pensada para no colisionar:

SICP	MyLISP (≤8 car., sin colisión)
<code>addend</code> / <code>augend</code>	<code>ADDEND</code> / <code>AUGEND</code>
<code>multiplier</code> / <code>multiplicand</code>	<code>FACTOR1</code> / <code>FACTOR2</code>
<code>make-sum</code> / <code>make-product</code>	<code>MAKESUM</code> / <code>MAKEPROD</code>
<code>is-sum?</code> / <code>is-product?</code>	<code>ISSUM?</code> / <code>ISPROD?</code>
<code>base</code> / <code>exponent</code>	<code>BASE</code> / <code>EXPON</code>
<code>make-exponentiation</code>	<code>MAKEPOW</code>

10.5 Predicados de clasificación

```
(DEFUN ISCONST? (E) (NUMBERP E))

(DEFUN ISVAR? (E) (SYMBOLP E))

(DEFUN ISSUM? (E)
  (AND (LISTP E) (EQ (CAR E) '+)))

(DEFUN ISPROD? (E)
  (AND (LISTP E) (EQ (CAR E) '*)))

(DEFUN ISPOW? (E)
  (AND (LISTP E) (EQ (CAR E) 'POT)))
```

`EQ` compara símbolos internados por identidad — es la comparación correcta y más barata para comprobar el operador de la cabeza de la lista (no necesitamos `EQUAL`, que recorrería estructuras).

10.6 Selectores

```

(DEFUN ADDEND (E) (CAR (CDR E)))      ; primer sumando
(DEFUN AUGEND (E) (CAR (CDR (CDR E)))) ; segundo sumando

(DEFUN FACTOR1 (E) (CAR (CDR E)))      ; primer factor
(DEFUN FACTOR2 (E) (CAR (CDR (CDR E)))) ; segundo factor

(DEFUN BASE (E) (CAR (CDR E)))         ; base de la potencia
(DEFUN EXPON (E) (CAR (CDR (CDR E))))  ; exponente de la potencia

```

10.7 Constructores (todavía "tontos" — sin simplificar)

```

(DEFUN MAKESUM (A1 A2) (LIST '+ A1 A2))
(DEFUN MAKEPROD (A1 A2) (LIST '* A1 A2))
(DEFUN MAKEPOW (B E) (LIST 'POT B E))

```

Deliberadamente **no** comprobamos aquí si `A1` es `0` o `A2` es `1`. Eso vendrá en el capítulo del simplificador, cuando ya duela no tenerlo (al ver la explosión de `DERIV`). Por ahora, un constructor solo construye.

Sobre el nombre: podría parecer más corto y directo llamarlas `SUM`, `PROD`, `POW` en vez de `MAKESUM`, `MAKEPROD`, `MAKEPOW`. Ahora mismo, con constructores tan simples, esa diferencia parece solo estética. El Capítulo 13 explica por qué el prefijo `MAKE` deja de ser cosmético en cuanto estas funciones empiecen a hacer más que empaquetar argumentos.

10.8 Comprobación de nombres (sin colisiones a 8 caracteres)

Nombre	Longitud	¿Colisiona?
<code>ISCONST?</code>	8	exacto, sin margen
<code>ISVAR?</code>	6	no
<code>ISSUM?</code>	6	no
<code>ISPROD?</code>	7	no
<code>ISPOW?</code>	6	no
<code>ADDEND</code>	6	no
<code>AUGEND</code>	6	no
<code>FACTOR1</code> / <code>FACTOR2</code>	7	no
<code>BASE</code> / <code>EXPON</code>	4 / 5	no
<code>MAKESUM</code>	8	exacto, sin margen — cuidado si se añade un sufijo en el futuro
<code>MAKEPROD</code>	8	igual
<code>MAKEPOW</code>	7	ok

10.9 Probándolo en el REPL

```

(DEFINE E1 (MAKESUM 'X 3))
E1                ; -> (+ X 3)

(ISSUM? E1)       ; -> T
(ISPROD? E1)      ; -> NIL

```

```

(ADDEND E1)           ; -> X
(AUGEND E1)           ; -> 3

(DEFINE E2 (MAKEPROD (MAKESUM 'X 1) 'Y))
E2                    ; -> (* (+ X 1) Y)
(ISPROD? E2)          ; -> T
(FACTOR1 E2)           ; -> (+ X 1)
(ISSUM? (FACTOR1 E2)) ; -> T

(DEFINE E3 (MAKEPOW 'X 2))
E3                    ; -> (POT X 2)
(ISPOW? E3)           ; -> T
(BASE E3)              ; -> X
(EXPON E3)             ; -> 2

(ISCONST? 3)          ; -> T
(ISVAR? 'X)            ; -> T
(ISCONST? 'X)         ; -> NIL

```

10.10 Deuda técnica consciente

`ISVAR?` tal como está definido diría `T` también para `T` y `NIL`, porque internamente son símbolos. No lo arreglamos todavía — es exactamente el tipo de caso límite que conviene dejar anotado y resolver cuando aparezca un caso real que lo necesite (probablemente en `DERIV`, al derivar respecto a una variable concreta).

10.11 Resumen de lo construido en este capítulo

- **Representación:** expresiones matemáticas como listas MyLISP sin evaluar, con `+`, `*` y `POT` como operadores binarios.
- **Predicados:** `ISCONST?`, `ISVAR?`, `ISSUM?`, `ISPROD?`, `ISPOW?`.
- **Selectores:** `ADDEND` / `AUGEND`, `FACTOR1` / `FACTOR2`, `BASE` / `EXPON`.
- **Constructores:** `MAKESUM`, `MAKEPROD`, `MAKEPOW` (todavía sin simplificar — eso llega en el Capítulo 13).
- **Regla de oro establecida:** ninguna función fuera de este capítulo toca `CAR` / `CDR` directamente sobre una expresión del CAS.

10.12 Siguiendo capítulo

Capítulo 11 — Sustitución (`SUBST`) y evaluación numérica: primera función que recorre el AST construido en este capítulo, usando solo los constructores/selectores/predicados definidos aquí — nunca `CAR` / `CDR` directos.

Capítulo 11 — Sustitución (SUBST) y evaluación numérica

En una frase: vamos a escribir las dos primeras funciones que recorren de verdad el árbol del Capítulo 10 — `SUBST` para sustituir variables, `EVALEXPR` para reducir a un número — y con ellas fijamos el patrón de recorrido que se repetirá en el resto del CAS, además de resolver cómo señalar un fallo de dominio en un Lisp sin excepciones.

11.1 El árbol que ya teníamos sin saberlo

En lenguajes como C o Java, representar un árbol de sintaxis abstracta (AST) obliga a definir clases, nodos y punteros explícitos: una clase `sumaNodo` con dos campos `izquierdo` y `derecho`, otra `variableNodo` con un nombre, un mecanismo de herencia o unión de tipos para que todos los nodos encajen en la misma estructura... En Lisp no hace falta nada de eso.

Una expresión como

```
' ( + ( * 2 X ) 3 )
```

ya es el AST. No es una representación *de* un árbol — es un árbol: una lista cuyo primer elemento es el operador y cuyo resto son los operandos, que a su vez pueden ser listas (subárboles) o átomos (hojas). Esta propiedad — que el código y los datos comparten la misma forma sintáctica, la S-expression — se llama **homoiconicidad**, y es la razón por la que Lisp lleva siendo el lenguaje preferido para construir sistemas simbólicos desde los años 60.

El Capítulo 10 no construyó una estructura de datos nueva. Construyó la **interfaz de acceso** a un árbol que las listas de MYLISP ya nos daban gratis: `ISSUM?`, `ISPROD?`, `ISPOW?` son las preguntas "¿qué tipo de nodo es este?"; `ADDEND`, `AUGEND`, `FACTOR1`, `FACTOR2`, `BASE`, `EXPON` son las preguntas "¿cuáles son sus hijos?"; y `MAKESUM`, `MAKEPROD`, `MAKEPOW` son la forma de construir un nodo nuevo sin tocar la lista a mano.

El objetivo de este capítulo es aprender a **navegar** ese árbol: visitar cada nodo, decidir qué hacer según su tipo, bajar recursivamente a los hijos, y volver a montar el resultado. Es el patrón que vamos a repetir en prácticamente cada función del CAS a partir de ahora.

11.2 1. Sustitución simbólica

11.2.1 La idea

Sustituir es recorrer el árbol y, cada vez que llegamos a una hoja que es la variable que buscamos, cambiarla por otra cosa — un número, otra variable, o una subexpresión entera. El resultado sigue siendo un árbol válido de nuestra representación, así que se puede volver a sustituir, derivar, o lo que sea, sobre él.

11.2.2 La estructura recursiva (el patrón que se repetirá siempre)

Toda función que recorre nuestras expresiones tiene la misma forma:

1. **Caso base — constante.** Una constante no contiene ninguna variable. Se devuelve tal cual.
2. **Caso base — variable.** Si es la variable que buscamos, se devuelve el valor de sustitución. Si es una variable distinta, se devuelve igual.
3. **Caso recursivo — suma / producto / potencia.** No hay hoja que examinar todavía: hay que **bajar** a los hijos, sustituir en cada uno por separado, y **volver a subir** reconstruyendo el nodo con el constructor correspondiente.

El punto (3) es la clave que distingue "manipular una lista" de "manipular un árbol de forma abstracta": en ningún momento usamos `CAR`/`CDR` directamente sobre `E` dentro de `SUBST` — usamos los selectores del Capítulo 10, y reconstruimos con los constructores del Capítulo 10. Si mañana cambiamos la representación interna, `SUBST` no se entera.

11.2.3 El código

```
(DEFUN SUBST (E VBL VAL)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
         (COND ((EQ E VBL) VAL)
               (T E)))
        ((ISSUM? E)
         (MAKESUM (SUBST (ADDEND E) VBL VAL)
                   (SUBST (AUGEND E) VBL VAL)))
        ((ISPROD? E)
         (MAKEPROD (SUBST (FACTOR1 E) VBL VAL)
                   (SUBST (FACTOR2 E) VBL VAL)))
        ((ISPOW? E)
```

```
(MAKEPOW (SUBST (BASE E) VBL VAL)
          (SUBST (EXPON E) VBL VAL) ) ) )
```

Nota sobre el `COND`: no hay una cláusula final `(T ...)` de "tipo desconocido, error". Es una decisión deliberada, no un olvido — si en el futuro añadimos un nuevo tipo de nodo (por ejemplo `SIN / COS`) y nos olvidamos de darle un caso aquí, el intérprete devolverá `NIL` silenciosamente en vez de romperse con un error violento. Es una elección discutible entre "fallar rápido y ruidoso" o "fallar silencioso" — la dejamos anotada como decisión de diseño abierta, revisable si en la práctica nos genera más confusión que protección.

11.2.4 Trazarlo a mano (recomendado antes de programarlo)

Para `(SUBST '(+ X 3) 'X 5)`:

1. `E` es `(+ X 3)` → `ISSUM?` es verdadero.
2. Se descompone: `ADDEND` → `X`, `AUGEND` → `3`.
3. Se sustituye por separado: `(SUBST 'X 'X 5)` → `5` (caso variable, coincide); `(SUBST '3 'X 5)` → `3` (caso constante, no cambia).
4. Se reconstruye: `(MAKESUM 5 3)` → `(+ 5 3)`.

El resultado **no se reduce a 8** — `SUBST` no simplifica, solo sustituye. Eso es intencionado: separar "sustituir" de "simplificar" es lo que en el próximo capítulo nos permitirá ver con claridad el problema que resuelve el simplificador.

11.3 2. Evaluación numérica

11.3.1 La idea

Evaluar numéricamente una expresión en un punto es, en el fondo, aplicar `SUBST` hasta que no quede ninguna variable libre, y entonces reducir cada operador del árbol a su resultado numérico. La primera tentación es delegar todo el árbol al `EVAL` del intérprete anfitrión:

```
(DEFUN EVALEXPR (E) (EVAL E)) ; version ingenua -- ver mas abajo por que no
```

Esto funciona por una razón que conviene desenmascarar: `+` y `*` son símbolos que **casualmente** coinciden con funciones reales de MyLISP con exactamente la misma semántica que les hemos dado en nuestro AST. No es que `EVALEXPR` esté "evaluando el operador suma del CAS" — está delegando ciegamente el árbol entero al intérprete y confiando en que los nombres coincidan. Esa coincidencia se rompe en cuanto aparece `POT`, que nos inventamos nosotros y MyLISP no conoce.

Si el CAS tiene que ser una biblioteca portable sobre cualquier Lisp — sin exigir que el intérprete anfitrión conozca sus operadores — entonces esa exigencia debe aplicarse **por igual** a `+`, `*` y `POT`. La solución es que `EVALEXPR` use el mismo esqueleto recursivo que `SUBST`: clasificar con los predicados del Capítulo 10, bajar a los hijos, reducir cada operador de forma explícita.

Pero hay una segunda pregunta, más profunda, que se esconde detrás de la primera: **¿qué hace `EVALEXPR` cuando llega a una hoja que sigue siendo una variable?** La versión ingenua respondía "se lo pregunto al entorno global de MyLISP con `(EVAL E)`" — y eso mezcla dos sistemas de bindings que nuestro diseño quiere mantener separados: el binding *matemático* que crea `SUBST` (una sustitución explícita dentro del árbol del CAS) y el binding *de programa* que crea `DEFINE / LET` en el entorno del intérprete anfitrión. Si el usuario del CAS intenta evaluar `x + 2` sin haber hecho antes `SUBST`, lo correcto no es "preguntarle a MyLISP si por casualidad tiene una variable global llamada `x`" — es que el propio dominio del CAS diga con claridad: *esta expresión tiene una variable libre, no se puede reducir a un número*.

11.3.2 Por qué no podemos "lanzar un error" — y qué hacemos en su lugar

En Common Lisp o Scheme escribiríamos algo como `(error "variable libre")` y el sistema de condiciones se encargaría de interrumpir la evaluación de forma controlada. **MyLISP no tiene nada de eso**. No existe una función `ERROR` invocable desde Lisp, y no hay ningún mecanismo de excepciones o `CONDITION-CASE`. Los únicos errores que existen son los que el propio intérprete lanza internamente ante un tipo incorrecto (por ejemplo, sumar un símbolo), y esos son irrecuperables desde el código Lisp — abortan la evaluación sin que nuestras funciones puedan interceptarlos.

Esta limitación, lejos de ser un obstáculo, nos obliga a usar la única técnica de gestión de errores que **sí** es expresable en un Lisp tan mínimo como este: en vez de *lanzar* un error, **devolver un valor que significa error**, y hacer que quien llama a la función lo compruebe explícitamente antes de seguir. Es un patrón con nombre propio y con una larga tradición:

- En **C**, es el código de retorno especial (`-1`, `NULL`, `errno`) que toda función debe comprobar a mano después de cada llamada, porque el lenguaje no tiene excepciones.
- En **lenguajes funcionales modernos** (Haskell, Rust, OCaml...), es exactamente la idea detrás de los tipos `Option/Maybe` y `Result/Either`: una función que puede fallar no devuelve "el resultado o una excepción", devuelve "una caja que contiene el resultado válido, o una marca explícita de que no lo hay" — y el tipo del lenguaje obliga a abrir la caja y mirar antes de usar el contenido.

Nuestro `'VARLIBRE` es esa misma idea, implementada a mano porque MyLISP no tiene un sistema de tipos que nos la dé gratis: un **valor centinela** que representa "aquí no hay un número, hay un fallo de dominio", y que cada función que lo recibe debe comprobar explícitamente antes de seguir operando con él. La diferencia con `Option/Result` es solo de maquinaria — allí el compilador te obliga a comprobarlo; aquí la disciplina de comprobarlo es responsabilidad nuestra, cláusula por cláusula del `COND`. Es un buen ejemplo de cómo una limitación real de la plataforma (sin excepciones) nos lleva, por necesidad, a redescubrir a mano un patrón que en otros lenguajes viene incorporado.

11.3.3 El código

```
(DEFUN LIBRE? (X) (EQ X 'VARLIBRE))

(DEFUN POTENCIA (B N)
  (COND ((= N 0) 1)
        (T (* B (POTENCIA B (- N 1))))))

(DEFUN EVALSUMA (E)
  (LET ((A (EVALEXPR (ADDEND E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (AUGEND E))))
                (COND ((LIBRE? B) B)
                      (T (+ A B))))))))

(DEFUN EVALPROD (E)
  (LET ((A (EVALEXPR (FACTOR1 E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (FACTOR2 E))))
                (COND ((LIBRE? B) B)
                      (T (* A B))))))))

(DEFUN EVALPOW (E)
  (LET ((B (EVALEXPR (BASE E))))
    (COND ((LIBRE? B) B)
          (T (LET ((N (EVALEXPR (EXPON E))))
                (COND ((LIBRE? N) N)
                      (T (POTENCIA B N))))))))

(DEFUN EVALEXPR (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
```

```
(PROGN (PRINT "CAS: variable libre, no evaluable:")
  (PRINT E)
  'VARLIBRE))
((ISSUM? E) (EVALSUMA E))
((ISPROD? E) (EVALPROD E))
((ISPOW? E) (EVALPOW E)))
```

Nótese que `EVALEXPR` está partida en cuatro funciones (`EVALSUMA`, `EVALPROD`, `EVALPOW` y el propio `EVALEXPR`) en vez de anidar los tres `COND` + `LET` directamente dentro de un único `DEFUN`. Esto no es solo estilo — durante el desarrollo de este capítulo, la versión con todo anidado en una sola función provocó un **error de dirección** (un fallo a nivel de CPU 68000, no un error controlado del intérprete) al cargarla en el QL/emulador. Se corrigió arreglando el parser de MyLISP en el proyecto del intérprete, pero la lección práctica se queda aquí: **expresiones con muchos niveles de anidamiento dentro de un único `DEFUN` son un punto de riesgo real en este tipo de plataformas**, no solo una cuestión de estilo o legibilidad. Partir en funciones auxiliares más pequeñas reduce ese riesgo además de mejorar la lectura del código.

Tres detalles a entender bien, porque son la parte con más enjundia de todo el capítulo:

1. **'VARLIBRE' cabe justo en 8 caracteres**, sin truncar — y su nombre ya explica el fallo sin necesidad de mensaje adicional. `LIBRE?` es el predicado que lo reconoce, deliberadamente sin el prefijo `IS` que usan `ISSUM?` / `ISPROD?` / etc., porque no pregunta por un tipo de nodo del AST sino por un valor de control — una distinción sutil que merece su propia nota si algún lector se fija.
2. **Ningún operador aritmético real (+, *, POTENCIA) se ejecuta nunca sobre el centinela**. Cada rama recursiva comprueba primero si el resultado de evaluar el hijo es `'VARLIBRE`, y si lo es, lo devuelve inmediatamente **sin** llegar a llamar a `+/*/POTENCIA`. Si no hiciéramos esta comprobación, `(+ 'VARLIBRE 3)` le pasaría un símbolo a la suma real de MyLISP, y volveríamos a caer en el mismo problema que queríamos evitar: un error de bajo nivel del intérprete, fuera de control del CAS.
3. **La propagación es manual, nivel a nivel de la recursión — no hay nada automático**. Esto es la lección de diseño de lenguajes más honesta del capítulo: en un Lisp sin excepciones, "un error en las hojas se entera la raíz" no ocurre solo; hay que hacer que **cada** función intermedia coopere explícitamente comprobando y reenviando el centinela hacia arriba. Es el precio de la simplicidad del intérprete, y es exactamente el mismo trabajo que en Rust hace el operador `?` de forma automática sobre un `Result` — aquí lo escribimos a mano con `LET` + `COND` en cada cláusula.

Nota de plataforma: `POTENCIA` usa recursión simple con exponente entero no negativo. En MyLISP sobre QL (Pro Pascal) no aplica la restricción de `{ $A- }` /no-recursión que sí condiciona al proyecto del CPC en Turbo Pascal 3.0 CP/M-80, así que esta recursión no tiene esa trampa — pero queda anotado como limitación conocida que de momento no cubre exponentes negativos ni fraccionarios.

11.3.4 El incidente `VARLIBRE?` y la regla de auditoría de nombres

Al escribir por primera vez el predicado de control, se llamó `VARLIBRE?` — **9 caracteres**. Truncado a 8 se convierte en `VARLIBRE`, que es exactamente el mismo símbolo que usamos como valor centinela (`'VARLIBRE`). El predicado colisionaba con su propio sujeto: definir `VARLIBRE?` sobrescribía silenciosamente el símbolo de datos `VARLIBRE` con una función, rompiendo la comparación `(EQ X 'VARLIBRE)` de forma muy difícil de diagnosticar a simple vista.

La lección para el resto del manual: **antes de fijar el nombre definitivo de cualquier función nueva, hay que contar caracteres incluyendo cualquier sufijo (?, 1, 2 ...) y comprobar el resultado contra todos los nombres ya definidos**, no solo "parece corto". Es fácil confiarse con palabras ya largas de por sí (`VARLIBRE`) a las que luego se les añade un sufijo sin volver a contar. A partir de aquí, cada capítulo cierra con una tabla de auditoría de nombres antes de darse por verificado.

11.3.5 Encadenando SUBST y EVALEXPR

```
(DEFINE E1 (MAKESUM 'X 3))

(SUBST E1 'X 5)           ; -> (+ 5 3)           (sustitucion pura)
(EVALEXPR (SUBST E1 'X 5)) ; -> 8                 (evaluacion numerica)
```

Con más de una variable, se sustituye una detrás de otra:

```
(DEFINE E2 (MAKEPROD (MAKESUM 'X 1) 'Y))

(SUBST E2 'X 2)           ; -> (* (+ 2 1) Y)      ; Y sigue libre
(SUBST (SUBST E2 'X 2) 'Y 10) ; -> (* (+ 2 1) 10)
(EVALEXPR (SUBST (SUBST E2 'X 2) 'Y 10)) ; -> 30
```

Y si intentamos evaluar numéricamente sin haber sustituido todas las variables, el CAS ya no le pregunta nada al entorno global de MyLISP — detecta el fallo de dominio él mismo:

```
(EVALEXPR (SUBST E2 'X 2))
; CAS: variable libre, no evaluable:
; Y
; -> VARLIBRE
```

El resultado es el símbolo `VARLIBRE`, no un número — y cualquier función que use `EVALEXPR` como pieza de algo más grande puede comprobarlo con `LIBRE?` antes de confiar en el resultado, exactamente como se comprueba un `Result/Option` antes de "desempaquetarlo".

11.3.6 Probándolo con potencias

```
(DEFINE E3 (MAKEPOW 'X 2))
(SUBST E3 'X 4)           ; -> (POT 4 2)
(EVALEXPR (SUBST E3 'X 4)) ; -> 16
```

Con la versión recursiva de `EVALEXPR`, `POT` ya no es un caso especial ni una limitación: es un caso más del `COND`, tratado exactamente igual que `+` y `*`. La expresión `(POT 4 2)` nunca se le pasa al `EVAL` del host — `EVALEXPR` la reconoce ella misma y llama a `POTENCIA`.

11.4 Pendiente de verificar

- ☐ Prueba `(SUBST '(+ X 3) 'X 5)` y confirma que da `(+ 5 3)` **sin reducir** — `SUBST` sustituye, no evalúa.
- ☐ Encadena `EVALEXPR` sobre ese mismo resultado: `(EVALEXPR (SUBST '(+ X 3) 'X 5))` debe dar `8`.
- ☐ Prueba el caso de variable libre anidada, `(EVALEXPR (SUBST E2 'X 2))` con `E2 = (MAKEPROD (MAKESUM 'X 1) 'Y)` (definida más arriba), y confirma que el CAS detecta que `Y` sigue libre y devuelve `VARLIBRE` con su mensaje — sin que se propague ningún error de tipo de bajo nivel del intérprete.

11.5 Tabla de auditoría de nombres del capítulo 11

Nombre	Longitud	¿Colisiona?
<code>SUBST</code>	5	no
<code>LIBRE?</code>	6	no
<code>POTENCIA</code>	8	exacto, sin margen
<code>EVALSUMA</code>	8	exacto, sin margen
<code>EVALPROD</code>	8	exacto, sin margen
<code>EVALPOW</code>	7	no
<code>EVALEXPR</code>	8	exacto, sin margen

Cuatro nombres están exactamente en el límite de 8 caracteres sin margen (`POTENCIA`, `EVALSUMA`, `EVALPROD`, `EVALEXPR`). No colisionan entre sí ni con nada del Capítulo 10, pero cualquier sufijo futuro sobre estos nombres (por ejemplo, un hipotético `EVALEXPR2`) debe revisarse con la misma auditoría antes de darse por bueno.

11.5.1 Una nota que se cumplirá más adelante

`SUBST` no tiene ninguna lógica aritmética propia — cuando reconstruye el árbol tras sustituir, delega por completo en `MAKESUM` / `MAKEPROD` / `MAKEPOW`. Ahora mismo esos constructores son los "tontos" del Capítulo 10, así que `(SUBST E1 'X 5)` da `(+ 5 3)` sin reducir, tal como se ve arriba. Pero en el Capítulo 13 vamos a **redefinir esos mismos constructores en el mismo sitio** para que combinen constantes automáticamente — y como `SUBST` no sabe ni le importa qué versión de `MAKESUM` está vigente, heredaré ese cambio sin que nadie toque una línea suya. A partir de entonces, este mismo ejemplo dará `8` directamente, no `(+ 5 3)`. No será un error ni una regresión: será la barrera de abstracción de este capítulo funcionando exactamente como se diseñó. Si llegas aquí después de haber hecho el Capítulo 13, y `SUBST` te da resultados ya combinados, es justo eso lo que está pasando.

11.6 Resumen de lo construido en este capítulo

- `SUBST`: sustitución recursiva de una variable por un valor, siguiendo el patrón clasificar/bajar/reconstruir que se repetirá en todo el CAS.
- `EVALEXPR` (+ auxiliares `EVALSUMA` / `EVALPROD` / `EVALPOW` y `POTENCIA`): evaluación numérica autocontenida, sin delegar la estructura del árbol al `EVAL` del intérprete anfitrión.
- `LIBRE?` y el símbolo centinela `'VARLIBRE`: la técnica de "valor que significa error" para un Lisp sin excepciones, con propagación manual explícita en cada nivel de la recursión.
- **Dos lecciones de plataforma** que quedan como precedente para el resto del manual: cuidado con los caracteres no-ASCII al copiar/pegar código, y cuidado con anidar demasiados niveles dentro de un único `DEFUN` (el error de dirección).

11.7 Siguiente capítulo

Capítulo 12 — Derivación simbólica "cruda" (`DERIV`): la primera función que generará expresiones con redundancias evidentes, motivando por qué el Capítulo 13 introducirá un simplificador.

Capítulo 12 — Derivación simbólica "cruda" (`DERIV`)

En una frase: vamos a codificar las reglas de derivación del instituto de forma mecánica, sin intentar que el resultado sea bonito — precisamente para poder *ver* con nuestros propios ojos, en la pantalla del REPL, el problema que el Capítulo 13 va a resolver.

12.1 Por qué toca ahora, y no antes

Hasta aquí, `SUBST` y `EVALEXPR` han recorrido el árbol sin **crear** estructura nueva de forma agresiva — como mucho reconstruían un nodo igual de "grande" que el original. `DERIV` es la primera función que **genera** árboles: cada regla de derivación no sustituye, combina — toma dos subexpresiones ya derivadas y las junta con `+`, `*`, derivadas anidadas... El tamaño del árbol resultante puede crecer sin control, y verlo ocurrir en la pantalla del REPL es la mejor motivación posible para el capítulo que viene después. No es un capricho de organización del libro: la necesidad de simplificar solo se entiende una vez que se ha sufrido la explosión.

12.2 La teoría: derivar es aplicar reglas mecánicamente

La derivada de una expresión algebraica respecto a una variable se calcula con un conjunto pequeño y fijo de reglas — las mismas que cualquiera aprende en el instituto, solo que aquí las escribimos como código en vez de aplicarlas a mano:

- **Constante:** la derivada de un número es siempre `0`. No importa qué número sea, no depende de la variable.
- **Variable:** si es exactamente la variable respecto a la que derivamos, la derivada es `1`. Si es cualquier otra variable (una constante simbólica, en la práctica), la derivada es `0`.
- **Suma — regla de la suma:** $d/dx(u + v) = du/dx + dv/dx$. Se deriva cada sumando por separado y se suman los resultados.
- **Producto — regla del producto:** $d/dx(u \cdot v) = (du/dx \cdot v) + (u \cdot dv/dx)$. Esta es la primera regla que **no** es "derivar cada hijo por separado y combinar igual que antes" — mezcla las derivadas con las expresiones originales sin derivar, lo que dispara el tamaño del árbol.
- **Potencia — regla de la potencia (caso simple, exponente constante):** $d/dx(u^n) = n \cdot u^{(n-1)} \cdot du/dx$. Aquí asumimos, para esta primera versión "cruda", que el exponente es una constante y no depende de `x` — derivar exponentes variables (`x^x`) es un caso mucho más avanzado que dejamos fuera deliberadamente.

Fíjate en un patrón que se repite en la regla del producto y de la potencia: la derivada de un nodo compuesto necesita **tanto** la subexpresión original **como** su derivada. Esto es distinto de `SUBST`, donde solo hacía falta el resultado recursivo. Es la primera vez que el "patrón de recorrido" del Capítulo 11 no basta tal cual — hay que adaptarlo.

12.3 Trazarlo a mano antes de programarlo

Vamos a derivar `x · x` respecto a `x`, aplicando la regla del producto literalmente, sin simplificar nada:

$$\begin{aligned} d/dx(x \cdot x) &= (d/dx(x) \cdot x) + (x \cdot d/dx(x)) \\ &= (1 \cdot x) + (x \cdot 1) \end{aligned}$$

El resultado matemático correcto es `2x`, pero la regla mecánica nos da `(1 · x) + (x · 1)` — una expresión válida, pero completamente redundante. Nadie escribiría esto a mano. Vamos a dejar que `DERIV` lo genere tal cual, sin vergüenza, porque **ese es justo el punto pedagógico**: ver la fealdad con los propios ojos es lo que va a justificar el Capítulo 13.

12.4 El código

```
(DEFUN DSUMA (E VBL)
  (MAKESUM (DERIV (ADDEND E) VBL) (DERIV (AUGEND E) VBL)))

(DEFUN DPROD (E VBL)
  (MAKESUM (MAKEPROD (DERIV (FACTOR1 E) VBL) (FACTOR2 E))
    (MAKEPROD (FACTOR1 E) (DERIV (FACTOR2 E) VBL))))

(DEFUN DPOW (E VBL)
  (MAKEPROD (MAKEPROD (EXPON E) (MAKEPOW (BASE E) (- (EXPON E) 1)))
    (DERIV (BASE E) VBL)))
```

```
(DEFUN DERIV (E VBL)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) (COND ((EQ E VBL) 1) (T 0)))
        ((ISSUM? E) (DSUMA E VBL))
        ((ISPROD? E) (DPROD E VBL))
        ((ISPOW? E) (DPOW E VBL))))
```

`DERIV` está partida en tres funciones auxiliares (`DSUMA`, `DPROD`, `DPOW`) en vez de tener las tres reglas anidadas dentro de un único `DEFUN`, igual que hicimos con `EVALEXPR` en el Capítulo 11 — pero aquí el motivo es distinto y merece explicarse aparte.

12.4.1 Un segundo límite de plataforma: el buffer de `LOAD`

La versión "todo en un `DEFUN`" de `DERIV` no daba un error léxico ni un error de dirección — daba un error al cargarla con `LOAD` desde fichero. La razón: `DoLoad` acumula **todas las líneas de continuación de una misma definición** en un único buffer antes de parsearlo, y ese buffer tiene un límite de **500 caracteres**. La definición monolítica de `DERIV`, contando toda la indentación necesaria para que fuera legible, ocupaba **547 caracteres** — por encima del límite.

Este es un límite distinto al de "80 caracteres por línea" que ya conocíamos (ese es el límite del *teclado interactivo* de QDOS, no de `LOAD` leyendo un fichero). Aquí el límite no es por línea, sino por **definición completa acumulada**. La lección para el manual: en MyLISP sobre QL hay que vigilar **dos límites de tamaño distintos y por razones distintas**:

1. Al teclear interactivamente en el REPL: máximo ~80 caracteres acumulados por el buffer de teclado de QDOS.
2. Al cargar con `LOAD` desde fichero: máximo 500 caracteres por definición completa (todas sus líneas de continuación sumadas), acumulados en el buffer de `DoLoad`.

Partir una función grande en piezas más pequeñas no es solo buen estilo aquí — es la técnica que resuelve **ambos** límites a la vez, además del riesgo de anidamiento profundo que ya vimos con el error de dirección del Capítulo 11. Tres razones distintas, una misma solución: funciones pequeñas y auxiliares en vez de una única función gigante.

Nota importante sobre la regla de la potencia, dentro de `DPOW`: `(- (EXPON E) 1)` se calcula con la **resta real de MyLISP**, no simbólicamente — porque en esta primera versión asumimos que el exponente ya es una constante numérica (`2`, `3` ...), nunca una subexpresión. Si en el futuro quisiéramos exponentes simbólicos, esta línea tendría que cambiar por completo; por ahora es una limitación consciente, no un descuido, y vale la pena decirlo explícitamente en el manual.

12.5 Probándolo — y viendo la explosión con tus propios ojos

```
(DERIV '(+ X 3) 'X)           ; -> (+ 1 0)
(DERIV (MAKEPROD 'X 'X) 'X)   ; -> (+ (* 1 X) (* X 1))
(DERIV (MAKEPOW 'X 2) 'X)     ; -> (* (* 2 (POT X 1)) 1)
```

Ninguno de estos tres resultados es matemáticamente incorrecto — pero ninguno es tampoco lo que un humano escribiría. `(+ 1 0)` debería ser `1`. `(+ (* 1 X) (* X 1))` debería ser `2X`. `(* (* 2 (POT X 1)) 1)` debería ser `2X`. Cuanto más grande sea la expresión de partida, peor se pone: prueba a derivar dos veces seguidas `((DERIV (DERIV E 'X) 'X))` y verás cómo la "basura" se acumula sobre basura anterior, sin límite aparente.

Este es el problema real, no inventado, que resolverá el Capítulo 13: un simplificador que, aplicado sobre estos mismos resultados, los reduzca a la forma que esperaríamos ver escrita en un papel.

12.6 Pendiente de verificar

- ☐ Probar `(DERIV '(+ X 3) 'X)` y confirmar `(+ 1 0)`.
- ☐ Probar `(DERIV (MAKEPROD 'X 'X) 'X)` y confirmar `(+ (* 1 X) (* X 1))`.
- ☐ Probar `(DERIV (MAKEPOW 'X 2) 'X)` y confirmar `(* (* 2 (POT X 1)) 1)`.

- ☐ Probar una derivada segunda (`DERIV` sobre el resultado de `DERIV`) y observar el crecimiento del árbol.

12.7 Tabla de auditoría de nombres del capítulo 12

Nombre	Longitud	¿Colisiona?
<code>DSUMA</code>	5	no
<code>DPROD</code>	5	no
<code>DPOW</code>	4	no
<code>DERIV</code>	5	no

Nombres deliberadamente cortos: `DERIVSUM` / `DERIVPROD` / `DERIVPOW` habría sido la elección más "obvia" por paralelismo con `EVALSUMA` / `EVALPROD` / `EVALPOW` del Capítulo 11, pero `DERIVPROD` tiene 9 caracteres — se pasa del límite. Se usa el prefijo corto `D` en vez de `DERIV` completo para los auxiliares, dejando margen de sobra en los cuatro nombres.

12.8 Resumen de lo construido en este capítulo

- `DERIV` (+ auxiliares `DSUMA` / `DPROD` / `DPOW`): derivación simbólica mecánica, deliberadamente sin simplificar.
- **Un segundo límite de plataforma documentado:** el buffer de `LOAD` acumula 500 caracteres por definición completa, distinto del límite de 80 caracteres por línea del teclado interactivo.
- **El problema real que motiva el resto del libro:** expresiones correctas pero ilegibles, que se agravan con cada derivada sucesiva.

12.9 Siguiente capítulo

Capítulo 13 — Simplificación mediante "Constructores Inteligentes": resuelve exactamente el problema que acabamos de generar aquí, pero no con una función nueva que recorra el árbol a posteriori — redefiniendo `MAKESUM`, `MAKEPROD` y `MAKEPOW` del Capítulo 10 para que apliquen identidades como $x+0 = x$, $x \cdot 1 = x$, $x \cdot 0 = 0$ en el momento mismo de construir cada nodo. El lector verá cómo la salida "fea" de `DERIV` se vuelve legible **sin cambiar ni una línea de `DERIV`**, gracias a la barrera de abstracción que separamos en el Capítulo 10.

Capítulo 13 — Simplificación mediante "Constructores Inteligentes"

En una frase: en vez de limpiar el árbol después de construirlo, interceptamos la construcción misma — redefiniendo `MAKESUM`, `MAKEPROD` y `MAKEPOW` del Capítulo 10 para que apliquen identidades algebraicas (elemento neutro, elemento absorbente, evaluación de constantes) antes de crear ningún nodo. El resultado mejora la salida de `DERIV` sin tocar ni una línea de `DERIV`.

13.1 De la pared del Capítulo 12 a la solución

El código de `DERIV` del capítulo anterior es correcto y no hay que tocarlo: aislar cada regla en `DSUMA`, `DPROD` y `DPOW` ya fue la decisión correcta para mantenerlo legible. Y respondiendo a la pregunta que cualquiera se hace al terminar el Capítulo 12 — "¿debería hacer que `DERIV` sea más lista?" — la respuesta es que no, en absoluto. Cualquier intento de que `DERIV` "sepa" simplificar mientras deriva arruinaría el propósito pedagógico de ese capítulo: el lector tenía que estrellarse contra la pared de la explosión combinatoria para entender, con sus propios ojos, la frase que resume todo el problema:

El cálculo simbólico sin simplificación es computacionalmente inútil, porque agota la memoria y la paciencia del usuario.

Un ejemplo pequeño basta para verlo: al derivar x^2 (representado internamente como $x \cdot x$), la regla del producto generaba $(1 \cdot x) + (x \cdot 1)$. Si a ese resultado le aplicas otra derivada, el árbol se llena de ceros y unos esparcidos por todas partes, sin que ninguno aporte información matemática real. Ese es el problema real, no inventado, que resolvemos ahora.

13.2 La idea: simplificar en el momento de construir, no después

Hay dos formas de atacar este problema. La primera —la más intuitiva a primera vista— sería escribir una función `SIMP` que recorra un árbol ya construido y lo limpie a posteriori, con sus propios predicados, selectores y reglas de reescritura. La segunda, más elegante y con más tradición en Lisp (es exactamente la técnica que usa SICP para `make-sum`/`make-product`), es no dejar que el árbol "sucio" llegue nunca a existir: interceptar la construcción en el propio constructor.

13.3 Las matemáticas antes que el Lisp

Antes de escribir una sola línea de código, hay que dejar claro **qué** identidad algebraica justifica cada regla y **en qué orden** hay que comprobarlas — porque el orden no es un detalle de estilo, es parte de la especificación: si se comprueban en el orden equivocado, alguna regla nunca llegaría a activarse. Ninguna de estas reglas depende de Lisp ni de MyLISP; son propiedades de la suma y el producto que ya conoces de las matemáticas del instituto, aquí simplemente les ponemos nombre técnico y las ordenamos con precisión:

Para $A1 + A2$:

Prioridad	Condición	Resultado	Nombre de la propiedad
0	$A1$ es 0	$A2$	elemento neutro de la suma ($0 + a = a$)
1	$A2$ es 0	$A1$	elemento neutro de la suma ($a + 0 = a$)
2	$A1$ y $A2$ son ambos números	su suma real	evaluación de constantes ("constant folding": si ya sabemos los dos valores, no hay razón para dejar la suma sin hacer)
3	cualquier otro caso	$(+ A1 A2)$	no hay identidad aplicable, hay que representarlo simbólicamente

Para $A1 \cdot A2$:

Prioridad	Condición	Resultado	Nombre de la propiedad
0	$A1$ es 0 o $A2$ es 0	0	elemento absorbente del producto ($0 \cdot a = 0$, sin importar a)
1	$A1$ es 1	$A2$	elemento neutro del producto ($1 \cdot a = a$)
2	$A2$ es 1	$A1$	elemento neutro del producto ($a \cdot 1 = a$)
3	$A1$ y $A2$ son ambos números	su producto real	evaluación de constantes
4	cualquier otro caso	$(* A1 A2)$	sin identidad aplicable

Para B^E :

Prioridad	Condición	Resultado	Nombre de la propiedad
0	<code>E</code> es <code>0</code>	<code>1</code>	cualquier base elevada a <code>0</code> es <code>1</code> (por convención, incluso si la base es <code>0</code> , aquí no discutimos ese caso límite)
1	<code>E</code> es <code>1</code>	<code>B</code>	eleva a la primera potencia no cambia nada
2	<code>B</code> es <code>1</code>	<code>1</code>	<code>1</code> elevado a cualquier exponente sigue siendo <code>1</code>
3	<code>B</code> y <code>E</code> son ambos números	la potencia real calculada	evaluación de constantes
4	cualquier otro caso	<code>(POT B E)</code>	sin identidad aplicable

Fíjate en un detalle que distingue la suma/producto de la potencia: en la suma y el producto, **el elemento neutro se comprueba antes que la evaluación de constantes**, mientras que en la potencia el orden entre "exponente 0/1" y "ambos números" tiene menos impacto práctico porque los casos no se solapan de la misma manera. Aun así, seguimos siempre la regla general: **primero los casos triviales que dependen de un solo argumento, después la evaluación de constantes, y al final el caso genérico** — es el mismo principio de "lo más específico primero" que ya usamos al ordenar las cláusulas de cualquier `COND` a lo largo de todo el CAS.

Una vez estas tres tablas están claras **en papel**, traducirlas a Lisp es mecánico: cada fila se convierte en una cláusula de un `COND`, en el mismo orden de prioridad. El código no añade ninguna idea nueva — solo la expresa de forma ejecutable.

13.4 Sustituyendo los constructores del Capítulo 10

Con las tablas ya definidas, vamos a **sustituir** — no añadir al lado, sustituir — las tres funciones `MAKESUM`, `MAKEPROD` y `MAKEPOW` que escribimos en el Capítulo 10. Allí eran deliberadamente "tontas": solo empaquetaban sus argumentos en una lista con `LIST`, sin mirar su contenido. A partir de aquí, esas mismas tres funciones —mismo nombre, misma firma, mismo lugar en el programa— pasan a comprobar las tablas de arriba antes de construir nada. Cualquier otra función que ya las usara (`SUBST`, `DERIV`, `DSUMA`, `DPROD`, `DPOW`) sigue llamándolas exactamente igual y no necesita cambiar ni una línea — es la barrera de abstracción del Capítulo 10 demostrando su valor: el código cliente nunca supo cómo se construía un nodo, así que no le afecta que ahora se construya mejor.

13.5 Por qué se llaman `MAKESUM`, no `SUM`

Antes del código, una decisión de nomenclatura que merece explicarse aquí y no antes: podría parecer más limpio llamar a estas funciones `SUM`, `PROD`, `POW` — más cortas, más directas. No lo hacemos, y la razón tiene más peso ahora que en el Capítulo 10.

El prefijo `MAKE` es una señal deliberada: dice "esto **construye** un nodo del árbol", no "esto **calcula** un resultado". En el Capítulo 10 esa distinción apenas se notaba, porque `MAKESUM` siempre construía una lista sin excepción — no había nada que confundir. Pero a partir de aquí, `MAKESUM` empieza a comportarse de dos formas distintas según sus argumentos: `(MAKESUM 2 3)` devuelve el número `5`, mientras que `(MAKESUM 'x 3)` devuelve la lista `(+ x 3)`. Si la función se llamara simplemente `SUM`, un lector nuevo —o tú mismo, releyendo el código dentro de unos meses— podría razonablemente esperar que **siempre** devuelva un número, como sugiere el nombre "suma". El prefijo `MAKE` es precisamente el que evita esa expectativa equivocada: recuerda en cada llamada que la función pertenece a la familia de los **constructores** de la representación del CAS (junto a `ADDEND` / `AUGEND`, `ISSUM?` ...), no a la familia de los operadores aritméticos reales de MyLISP. Que a veces "ahorre trabajo" devolviendo ya el resultado numérico es una optimización interna del constructor, no un cambio de lo que la función es.

Es una distinción pequeña en apariencia, pero es la misma que separa "una función que sabe cuándo puede atajar" de "una función cuyo comportamiento depende silenciosamente del tipo de sus argumentos sin que el nombre lo avise". El coste de escribir tres letras más al llamar a `MAKESUM` en vez de `SUM` es, comparado con eso, insignificante.

13.6 El código

```
(DEFUN CERO? (X) (AND (ISCONST? X) (= X 0)))
(DEFUN UNO? (X) (AND (ISCONST? X) (= X 1)))

(DEFUN MAKESUM (A1 A2)
  (COND ((CERO? A1) A2)
        ((CERO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (+ A1 A2))
        (T (LIST '+ A1 A2))))

(DEFUN MAKEPROD (A1 A2)
  (COND ((OR (CERO? A1) (CERO? A2)) 0)
        ((UNO? A1) A2)
        ((UNO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (* A1 A2))
        (T (LIST '* A1 A2))))

(DEFUN MAKEPOW (B E)
  (COND ((CERO? E) 1)
        ((UNO? E) B)
        ((UNO? B) 1)
        ((AND (ISCONST? B) (ISCONST? E)) (POTENCIA B E))
        (T (LIST 'POT B E))))
```

Unos apuntes sobre decisiones concretas:

- **CERO? / UNO? comprueban ISCONST? antes que nada.** Es imprescindible: si `A1` es la lista `(+ x y)` o el símbolo `x`, llamar directamente a `(= A1 0)` fallaría con un error de tipo del intérprete — exactamente el mismo problema de "no llamar a un operador real sobre algo que no es un número" que ya vimos con el centinela `VARLIBRE` en el Capítulo 11. La comprobación de tipo siempre va primero.
- **Se usa = y no EQ / EQUAL para comparar con cero/uno.** Recordemos el diseño de MyLISP: `=` compara *valor numérico*, mientras que `EQUAL` distingue tipos exactos (`(EQUAL 0 0.0)` sería `NIL`). Para el simplificador nos interesa el valor, no la representación interna — `0`, `0/1` o `0.0` deben tratarse todos como "cero" a efectos algebraicos.
- **MAKEPOW reutiliza POTENCIA**, la misma función que ya definimos en el Capítulo 11 para `EVALPOW`. No hace falta duplicar la lógica de potenciación — es una función de utilidad numérica, no específica de la evaluación, así que tiene sentido compartirla.
- **El orden de las cláusulas sigue exactamente las tablas de arriba: los casos triviales de un solo argumento (CERO? / UNO?) van antes que la evaluación de constantes.** Esto importa en un caso límite concreto: `MAKESUM(0, 0)`. Con este orden, `CERO? A1` se cumple primero y devuelve `A2` (que también es `0`) sin llegar a ejecutar la suma real — el resultado final es el mismo número, pero el camino que se toma es distinto, y en un sistema con reglas más complejas ese orden sí puede cambiar el resultado. Por eso las tablas matemáticas de la sección anterior no son solo documentación — son la especificación exacta que el código debe seguir al pie de la letra.

13.7 El efecto retroactivo sobre el Capítulo 12

Sin cambiar ni una línea de `DERIV`, `DSUMA`, `DPROD` o `DPOW`, los mismos tres ejemplos del capítulo anterior mejoran así:

Expresión	Capítulo 12 (constructores "tontos")	Capítulo 13 (constructores inteligentes)
$\frac{d}{dx}(x + 3)$	<code>(+ 1 0)</code>	<code>1</code>
$\frac{d}{dx}(x \cdot x)$	<code>(+ (* 1 X) (* X 1))</code>	<code>(+ X X)</code>
$\frac{d}{dx}(x^2)$	<code>(* (* 2 (POT X 1)) 1)</code>	<code>(* 2 X)</code>

El primer y el tercer caso quedan perfectamente limpios — `1` y `2x` son exactamente lo que un humano escribiría a mano. El segundo caso, `(+ x x)`, mejora mucho (ya no hay unos ni productos redundantes) pero **no** colapsa a `2x`. Esto no es un fallo del capítulo — es la frontera exacta de lo que un constructor inteligente puede resolver por sí solo: `x + 0` y `x · 1` son identidades que dependen únicamente de **un** de los dos argumentos, mientras que "combinar términos semejantes" (`x + x = 2x`) requiere **comparar los dos operandos entre sí** y reconocer que son la misma subexpresión. Eso es un problema de una naturaleza distinta — requiere igualdad estructural entre subexpresiones y una noción de orden canónico — y es exactamente lo que abordaremos en el próximo capítulo.

13.8 Una regla que parece obvia pero es matemáticamente falsa: 0^x

Es tentador añadir también "si la base es `0`, el resultado es `0`", simétrica a la regla de `UNO?` que acabamos de añadir. **No lo hacemos, y la razón es importante para el manual:** a diferencia de `1^x = 1`, que es cierto para *todo* `x` real sin excepción, `0^x = 0` **solo** es cierto cuando `x` es estrictamente positivo:

- `00` vale, por convención, `1` en la mayoría de contextos algebraicos — no `0`.
- `0(-2) = 1/02` — división por cero, **indefinido**, no `0`.
- `03 = 0` — aquí sí, pero solo porque el exponente es positivo.

Cuando `E` es una variable simbólica en `(MAKEPOW 0 X)`, no sabemos si `x` es positivo, cero o negativo — así que no podemos garantizar que la identidad se cumpla. Simplificar incondicionalmente a `0` produciría un resultado **matemáticamente incorrecto** para dos de los tres casos posibles de `x`, y lo haría de forma silenciosa: el CAS no se equivocaría "un poco", devolvería un número concreto y erróneo con la misma confianza que si fuera correcto. Eso es mucho peor que no simplificar en absoluto.

Por eso, tal como está `MAKEPOW` ahora, `(MAKEPOW 0 'X)` deja el resultado como `(POT 0 X)`, sin evaluar — y ese es el comportamiento **correcto**, no una limitación pendiente de arreglar. La lección general para todo el CAS, no solo para este caso: **una identidad algebraica solo se puede codificar como regla incondicional si es cierta para todos los valores posibles de la variable** — no basta con que sea cierta en el caso típico o esperado. Cuando la certeza depende del signo, la magnitud o algún otro supuesto sobre la variable que el CAS no puede verificar, lo correcto es no simplificar. Añadir esa capacidad de razonar bajo suposiciones (`assume(x > 0)`, por ejemplo) es una funcionalidad real de los CAS avanzados, y queda fuera del alcance de este manual.

13.9 Probándolo

```
(MAKESUM 2 3)           ; -> 5      (combinacion de constantes)
(MAKESUM 'X 0)           ; -> X
(MAKESUM 0 'X)           ; -> X
(MAKEPROD 'X 0)          ; -> 0
(MAKEPROD 'X 1)          ; -> X
(MAKEPOW 'X 0)           ; -> 1
(MAKEPOW 'X 1)           ; -> X
(MAKEPOW 1 'X)           ; -> 1      (nueva regla: base 1)
(MAKEPOW 0 'X)           ; -> (POT 0 X)  (NO se simplifica, correcto)
(MAKEPOW 2 3)           ; -> 8      (combinacion de constantes)

(DERIV '(+ X 3) 'X)      ; -> 1
```

```
(DERIV (MAKEPROD 'X 'X) 'X) ; -> (+ X X)
(DERIV (MAKEPOW 'X 2) 'X) ; -> (* 2 X)
```

13.10 Pendiente de verificar

- ☐ Probar los ocho casos sueltos de `MAKESUM` / `MAKEPROD` / `MAKEPOW`, más los dos nuevos: `(MAKEPOW 1 'X) → 1` y `(MAKEPOW 0 'X) → (POT 0 X)` **sin simplificar** (comportamiento correcto).
- ☐ Re-probar los tres `DERIV` del capítulo 12 y confirmar la mejora de la tabla anterior.
- ☐ Confirmar que `(DERIV (MAKEPROD 'X 'X) 'X)` da `(+ X X)` y **no** colapsa a `(* 2 X)` — es el comportamiento esperado en este capítulo, no un error.

13.11 Tabla de auditoría de nombres del capítulo 13

Nombre	Longitud	¿Colisiona?
<code>CERO?</code>	5	no
<code>UNO?</code>	4	no

`MAKESUM`, `MAKEPROD` y `MAKEPOW` ya estaban auditados en el Capítulo 10 — se redefinen aquí, no se renombran, así que no hace falta volver a comprobarlos.

13.12 Resumen de lo construido en este capítulo

- `CERO?` / `UNO?`: predicados auxiliares para detectar los elementos neutros de la suma y el producto, comprobando el tipo antes de comparar valores.
- `MAKESUM`, `MAKEPROD`, `MAKEPOW` (**versión inteligente**): mismos nombres, misma firma que en el Capítulo 10, ahora aplicando las tablas de identidades algebraicas antes de construir cualquier nodo.
- Un caso rechazado a propósito** (`0^x → 0` para exponente simbólico): la lección de que una identidad solo se puede codificar incondicionalmente si es cierta para *todos* los valores posibles de la variable.
- Confirmación práctica de la barrera de abstracción del Capítulo 10**: `SUBST`, `DERIV` y todas sus auxiliares mejoran su salida sin haber cambiado ni una línea.

13.13 Siguiente capítulo

Capítulo 14 — Orden canónico y términos semejantes: resuelve el caso que se queda fuera de este capítulo (`x + x → 2x`), mediante comparación estructural entre subexpresiones y una noción de orden que permita detectar cuándo dos términos son "el mismo" salvo por un coeficiente numérico.

Capítulo 14 — Orden canónico y términos semejantes

En una frase: los constructores inteligentes del Capítulo 13 no pueden resolver `x + x → 2x` porque solo ven dos argumentos a la vez; aquí construimos `SIMPSUM`, una función que aplanar temporalmente la cadena de sumas en una lista corriente, agrupa por término semejante y reconstruye el árbol — sin cambiar la representación del AST en ningún momento.

14.1 Qué es una "forma canónica"

Antes de escribir código, hay que entender el concepto que da nombre al capítulo. Dos expresiones pueden ser **matemáticamente equivalentes** sin ser **estructuralmente iguales** como listas de Lisp: `(+ x x)` y `(* 2 x)` representan el mismo valor para cualquier `x`, pero son árboles distintos, y `EQUAL` diría que no son iguales. Una **forma canónica** es una regla fija que dice: "de todas las formas posibles de escribir una expresión equivalente, esta es la única que vamos a considerar la correcta" — de manera que, una vez todo pasa por esa regla, comparar dos expresiones por equivalencia se reduce a comparar sus formas canónicas con `EQUAL`. Sin una forma canónica, `x + x`, `2 · x` y `x · 2` conviven como tres representaciones distintas del mismo hecho matemático, y el CAS no tiene manera de saberlo. Este capítulo construye la regla que colapsa todas esas variantes en una sola.

14.2 El problema exacto que dejamos abierto

En el Capítulo 13 vimos que los constructores inteligentes resuelven `x + 0`, `x · 1`, `x · 0`... pero no resuelven `x + x → 2x`. La razón de fondo, y merece decirse con precisión antes de seguir: **MAKESUM solo ve dos argumentos a la vez**. Cuando se le pasa `x` y `x`, no tiene forma de saber que son "el mismo término" salvo comparándolos — y aunque lo hiciera, el problema real aparece con cadenas más largas: `(+ (+ x y) x)` tiene tres sumandos reales (`x`, `y`, `x`) repartidos en dos llamadas anidadas a `MAKESUM`, cada una viendo solo una pareja. Ninguna simplifica localmente puede ver la lista completa de sumandos a la vez. Este es el límite estructural, no un descuido: **una operación que solo mira dos argumentos no puede resolver un problema que depende de compararlos todos entre sí**.

14.3 Dos caminos posibles — y por qué elegimos el tercero

Hay dos formas "obvias" de resolver esto:

- **Árboles binarios estrictos con rotación:** mantener la representación binaria de siempre y programar un mecanismo que reorganice el árbol (rotarlo hacia una forma normalizada) para poder comparar sumandos que están en ramas distintas. Es viable, pero es el camino más difícil: entrelaza la lógica de "encontrar términos semejantes" con la lógica de "mantener la forma de árbol", cuando en realidad son dos problemas independientes.
- **Operadores n-arios (listas planas) en toda la representación:** rediseñar `+` y `*` para que acepten cualquier número de argumentos desde el Capítulo 10 (`(+ x y x)` en vez de árboles binarios anidados). Es la solución "profesional" — así es como Maxima o SymPy representan internamente sumas y productos — pero tiene un coste real para este manual: obligaría a reescribir `ADDEND` / `AUGEND`, `SUBST`, `DERIV` (`DSUMA` / `DPROD`) y `EVALEXPR` (`EVALSUMA` / `EVALPROD`), es decir, **los cuatro capítulos anteriores, ya verificados en hardware real**. Desde el punto de vista de construir un CAS de producción sería el camino correcto a medio plazo; desde el punto de vista de un manual que enseña paso a paso, es un salto que rompe toda la continuidad ya construida y añade una complejidad de representación grande para resolver un problema local.

El camino que seguimos aquí — **aplanar, agrupar y reconstruir** — no cambia la representación oficial del AST en ningún momento. `ISSUM?`, `MAKESUM`, `ADDEND`, `AUGEND` y todo lo que ya existe en `SUBST`, `DERIV` y `EVALEXPR` siguen exactamente igual. Lo que hacemos es escribir una función *nueva*, autocontenida, que:

1. Convierte temporalmente el árbol de sumas en una **lista Lisp corriente** (no una estructura del CAS, una lista normal del lenguaje anfitrión) donde sí se puede ver "todos los sumandos a la vez".
2. Agrupa esa lista por término semejante.
3. Reconstruye un árbol binario de sumas de nuevo, usando los mismos `MAKESUM` / `MAKEPROD` de siempre.

Es la misma idea de "convertir a una forma de trabajo más cómoda, operar, reconstruir" que ya usamos sin ponerle nombre en capítulos anteriores — aquí la forma de trabajo cómoda es una lista plana en vez de otro árbol del CAS.

14.4 Trazando el pipeline completo a mano, antes del código

Antes de ver las cuatro funciones, conviene ver **qué aspecto tienen los datos** en cada frontera entre una función y la siguiente — porque el salto real de este capítulo no está en la dificultad de cada función por separado (todas son un `COND` con recursión, como en los capítulos anteriores), sino en que cada una habla un "idioma" de datos distinto:

```

(+ (+ X Y) X)                                <- arbol del CAS (S-expression anidada)
  |
  | FLATSUM
  ▼
(X Y X)                                       <- lista Lisp plana y corriente
  |
  | AGRUPA (usa COEF/LITERAL termino a termino)
  ▼
((X . 2) (Y . 1))                           <- lista de asociacion (literal . coeficiente)
  |
  | REBUILD
  ▼
(+ (* 2 X) Y)                               <- arbol del CAS otra vez

```

Cuatro formas de datos, tres transiciones. Merece la pena nombrarlas explícitamente para no perderse al leer el código:

1. **Entrada y salida son árboles del CAS** — las mismas listas `(+ ...)` que llevamos manejando desde el Capítulo 10.
2. **FLATSUM produce una lista Lisp corriente**, sin ningún `+` de por medio: tres símbolos sueltos, `(x y x)`. En este punto ya no estamos "dentro" de la representación del CAS — es una lista normal del lenguaje anfitrión, y por eso a partir de aquí se puede usar `APPEND`, `CONS`, `CAR/CDR` libremente sin violar ninguna barrera de abstracción: la barrera protege el árbol del CAS, no las listas de trabajo internas que nosotros mismos creamos y desechamos.
3. **AGRUPA produce una lista de asociación**: pares `(literal . coeficiente)`, uno por cada término distinto. `x` aparecía dos veces en la lista plana, así que aquí ya está condensado en un único par `(x . 2)`. Esta es la estructura donde de verdad "ocurre" la simplificación — las dos anteriores solo preparan el terreno.
4. **REBUILD vuelve a hablar el idioma del CAS**: por cada par `(literal . coeficiente)` construye `(* coeficiente literal)` con `MAKEPROD`, y encadena todos con `MAKESUM` — devolviendo, otra vez, una S-expression legítima del árbol del CAS.

Con esta traza en mente, cada función que viene a continuación es simplemente "cómo se produce, en Lisp, la flecha correspondiente del diagrama de arriba" — no hace falta sostener las cuatro en la cabeza a la vez.

14.5 Paso 1: extraer coeficiente y parte literal de un término

Para poder agrupar `x`, `2x` y `x·2` como "el mismo término con coeficientes distintos", necesitamos descomponer cada sumando en un par `(coeficiente, parte literal)`:

- `3` (constante pura) → coeficiente `3`, sin parte literal.
- `x` (variable sola) → coeficiente `1`, literal `x`.
- `(* 2 x)` o `(* x 2)` → coeficiente `2`, literal `x` — el orden de los factores no debe importar.
- `(* x y)` (producto sin ninguna constante) → coeficiente `1`, literal el producto entero, sin descomponer más.

```

(DEFUN COEF (E)
  (COND ((ISCONST? E) E)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR1 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR2 E))
               (T 1)))
        (T 1)))

(DEFUN LITERAL (E)
  (COND ((ISCONST? E) 1)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR2 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR1 E))
               (T E)))
        (T E)))

```



```
(T E))
(T E)))
```

El truco para que `(* 2 x)` y `(* x 2)` den el mismo resultado es comprobar **FACTOR1** y **FACTOR2** **por separado**, sin asumir que la constante ocupa siempre la misma posición. Y un detalle deliberado: `LITERAL` de una constante pura devuelve `1`, no `NIL`. Esto permite que, al reconstruir con `(MAKEPROD COEF LITERAL)`, una constante pura vuelva a salir intacta sin ningún caso especial — `(MAKEPROD 3 1)` ya sabe simplificarse a `3` gracias a la regla `UNO?` del Capítulo 13. Diseñar bien los constructores en su momento nos está ahorrando trabajo ahora.

Límite explícito de esta extracción: es de un solo nivel. Reconoce una constante multiplicando a *cualquier otra cosa*, pero no reordena productos de varias variables (`(* x y)` y `(* y x)` seguirán siendo literales distintos a efectos de esta función) ni descompone productos con más de un factor constante anidado. Para el objetivo de este capítulo — colapsar sumas como `x + x` o `2x + 3x` — es suficiente; ir más allá exigiría una forma canónica también *dentro* de los productos, que queda fuera de alcance aquí.

14.6 Paso 2: aplanar la cadena de sumas

```
(DEFUN FLATSUM (E)
  (COND ((ISSUM? E)
    (APPEND (FLATSUM (ADDEND E)) (FLATSUM (AUGEND E))))
    (T (LIST E))))
```

Cada sumando que no es él mismo una suma se envuelve en una lista de un elemento (`(LIST E)`); cada suma se descompone recursivamente y se concatena con `APPEND`. El resultado de `(FLATSUM '(+ (+ x y) x))` es la lista `(x y x)` — tres elementos sueltos, sin ningún `+` de por medio. Nótese que `APPEND` y `LIST` son operaciones del *lenguaje anfitrión* sobre listas corrientes, no `MAKESUM` ni nada del CAS: en este paso salimos deliberadamente del dominio del árbol simbólico.

Esto es exactamente la primera flecha de la traza de arriba: entra el árbol `(+ (+ x y) x)`, sale la lista plana `(x y x)`.

14.7 Paso 3: agrupar por parte literal

Necesitamos recorrer la lista aplanada y acumular, para cada literal distinto, la suma de sus coeficientes. Sin `SETQ`, esto se hace construyendo una nueva lista de asociación en cada paso, de forma puramente funcional:

```
(DEFUN ASSOCADD (LIT CF ALIST)
  (COND ((NULL ALIST) (LIST (CONS LIT CF)))
    ((EQUAL (CAR (CAR ALIST)) LIT)
      (CONS (CONS LIT (+ (CDR (CAR ALIST)) CF)) (CDR ALIST)))
    (T (CONS (CAR ALIST) (ASSOCADD LIT CF (CDR ALIST))))))

(DEFUN AGRUPA (TERMS)
  (COND ((NULL TERMS) NIL)
    (T (ASSOCADD (LITERAL (CAR TERMS))
      (COEF (CAR TERMS))
      (AGRUPA (CDR TERMS))))))
```

`ASSOCADD` recibe un literal, un coeficiente y una lista de asociación `((literal . coeficiente-acumulado) ...)`, y devuelve una lista nueva: si el literal ya existe, suma el coeficiente a la entrada existente; si no, añade una entrada nueva. La comparación se hace con `EQUAL`, no `EQ` — porque un literal puede ser un símbolo (`x`) o una lista entera (`(* x y)`), y `EQ` solo garantiza identidad fiable para símbolos.

`AGRUPA` recorre la lista aplanada término a término, extrayendo `LITERAL`/`COEF` de cada uno y acumulándolos con `ASSOCADD`. El resultado de `(AGRUPA '(x y x))` es `((x . 2) (y . 1))` — la segunda flecha de la traza: de la lista plana a la lista de asociación.

14.8 Paso 4: reconstruir el árbol

```
(DEFUN REBUILD (ALIST)
  (COND ((NULL ALIST) 0)
        (T (MAKESUM (MAKEPROD (CDR (CAR ALIST)) (CAR (CAR ALIST)))
                     (REBUILD (CDR ALIST))))))

(DEFUN SIMPSUM (E) (REBUILD (AGRUPA (FLATSUM E))))
```

Por cada entrada `(literal . coeficiente)` se construye `(MAKEPROD coeficiente literal)` — que ya simplifica solo, gracias al Capítulo 13 — y se suman todas con `MAKESUM` en cascada. El caso base, `(NULL ALIST) → 0`, no es un capricho: es el "elemento neutro" de una suma vacía, y aquí ocurre algo que vale la pena señalar explícitamente porque es una muestra real de que la arquitectura por capas está funcionando: cuando la lista tiene un único término, `REBUILD` termina generando `(MAKESUM termino 0)`, y es la regla `CERO?` del Capítulo 13 — escrita sin pensar en absoluto en este capítulo — la que elimina ese `0` sobrante automáticamente. No hemos tenido que programar ningún caso especial para "el último término no lleva un `+ 0` de más": ya estaba resuelto por el trabajo de dos capítulos atrás.

`SIMPSUM` encadena los tres pasos. Es la función que el lector usará directamente, y sus dos únicas fronteras de entrada/salida son árboles del CAS — la tercera flecha de la traza cierra el ciclo, volviendo al mismo "idioma" con el que empezó `FLATSUM`.

14.9 Probándolo — cerrando el hilo abierto desde el Capítulo 12

```
(SIMPSUM '(+ (+ X Y) X))      ; -> (+ (* 2 X) Y)
(SIMPSUM '(+ X X))           ; -> (* 2 X)
```

Y ahora el ejemplo que llevábamos arrastrando desde que empezamos a derivar: la derivada de $x \cdot x$ respecto a x , que en el Capítulo 13 se quedaba en `(+ x x)` sin poder avanzar más:

```
(SIMPSUM (DERIV (MAKEPROD 'X 'X) 'X)) ; -> (* 2 X)
```

Por fin. Tres capítulos después de plantear el problema, $d/dx(x^2) = 2x$ sale exactamente como se escribiría a mano.

Un caso más, para confirmar que el orden de los factores no importa y que los coeficientes ya existentes se combinan bien:

```
(SIMPSUM '(+ (+ (* 2 X) (* 3 X)) Y)) ; -> (+ Y (* 5 X))
(SIMPSUM '(+ X (* X 2)))             ; -> (* 3 X)
```

Nótese en el primer ejemplo que el resultado sale como `(+ Y (* 5 X))` y no `(+ (* 5 X) Y)` — el orden de los sumandos en la suma reconstruida depende del orden en que `REBUILD` procesa la lista de asociación, no del orden original de aparición. Esto es correcto matemáticamente (la suma es conmutativa), pero es una limitación real de esta primera versión: **no hay todavía un orden canónico entre sumandos distintos**, solo entre apariciones del mismo literal. Fijar un orden estable (alfabético, por ejemplo) para que dos expresiones equivalentes produzcan *siempre* el mismo árbol —no solo el mismo valor— es un refinamiento natural para más adelante, y lo dejamos anotado como tal.

14.10 Tres ideas que merece la pena remarcar antes de cerrar el capítulo

14.10.1 1. Sinergia total con el Capítulo 13

El logro más importante de este capítulo no está en `FLATSUM` ni en `AGRUPA` — está escondido en una sola línea de `REBUILD`:

```
(MAKESUM (MAKEPROD (CDR (CAR ALIST)) (CAR (CAR ALIST))) ...)
```

REBUILD **confía ciegamente** en que **MAKESUM** y **MAKEPROD** van a limpiar cualquier caso trivial que aparezca. Ya vimos que esto elimina el `+ 0` sobrante del último término de la lista. Pero el efecto va más lejos: si al agrupar dos términos semejantes sus coeficientes se cancelan **exactamente a cero** — por ejemplo, `x` y `-1·x` — el término entero desaparece del resultado, sin que **REBUILD** sepa nada de cancelaciones ni tenga ningún caso especial para ello:

```
(SIMPSUM '(+ (+ x (* -1 x)) y)) ; -> y (el termino en x se esfuma)
(SIMPSUM '(+ x (* -1 x))) ; -> 0 (cancelacion total)
```

Esto ocurre porque **MAKEPROD** `(0, x)` ya sabe devolver `0` (regla del elemento absorbente), y **MAKESUM** ya sabe eliminar ese `0` de la suma (regla del elemento neutro) — exactamente las mismas dos reglas del Capítulo 13, aplicándose aquí sin que nadie las haya vuelto a invocar explícitamente. No hemos escrito ni un **IF** para "limpiar ceros de la lista final". Las capas de abstracción están haciendo el trabajo pesado por nosotros — que es la señal de que la arquitectura por capas, con barreras de abstracción bien puestas desde el Capítulo 10, está pagando sus dividendos.

14.10.2 2. Recursividad funcional pura, sin estado mutable

ASSOCADD y **AGRUPA** gestionan lo que en otro lenguaje sería "estado acumulado" (la lista de asociación que va creciendo) sin usar ninguna variable mutable — porque MyLISP no tiene **SETQ**. Actualizar una entrada existente se hace reconstruyendo la lista entera con la entrada modificada:

```
(CONS (CONS LIT (+ (CDR (CAR ALIST)) CF)) (CDR ALIST))
```

Esto no es una limitación que sorteamos — es la forma natural de programar en un Lisp sin asignación destructiva, y es exactamente la misma disciplina que ya aplicamos al centinela **VARLIBRE** en el Capítulo 11: en vez de mutar algo en el sitio, se construye y se devuelve una versión nueva. Es Lisp funcional en su forma más pura, y aquí aparece sin que hiciera falta ninguna decisión de diseño adicional — es simplemente la única forma disponible.

14.10.3 3. Nota de rendimiento algorítmico

FLATSUM usa **APPEND** dentro de la recursión. En la mayoría de implementaciones Lisp —MyLISP incluido, porque **APPEND** tiene que recorrer y reconstruir su primer argumento entero para poder engancharle el segundo— esto le da a esta fase una complejidad de **O(n²)** en el peor caso: una cadena de sumas muy desequilibrada hacia la izquierda `((+ (+ (+ (+ x y) z) w) ...))` obliga a recorrer una y otra vez las mismas sublistas ya construidas. Para las expresiones de un CAS didáctico como el nuestro, con árboles pequeños, este coste es completamente irrelevante en la práctica. Pero es una limitación real y conocida de la técnica "aplanar con **APPEND**", y conviene decirlo así de claro para cualquier lector con más experiencia en programación que se pregunte por qué no se ha optimizado: la respuesta es que, a esta escala, optimizarlo sería complejidad prematura — pero un CAS de producción sobre expresiones grandes necesitaría una estrategia de aplanado con coste lineal (acumulando en orden inverso y revirtiendo al final, por ejemplo), no la que usamos aquí.

14.11 Pendiente de verificar

- ☐ Probar `(SIMPSUM '(+ (+ x y) x))` y confirmar `(+ (* 2 x) y)` (el orden exacto de los sumandos puede variar, ver nota anterior).
- ☐ Probar `(SIMPSUM '(+ x x))` y confirmar `(* 2 x)`.
- ☐ Probar `(SIMPSUM (DERIV (MAKEPROD 'x 'x) 'x))` y confirmar `(* 2 x)` — el cierre del hilo del Capítulo 12.
- ☐ Probar **COEF**/**LITERAL** sueltos sobre `(MAKEPROD 2 'x)` y `(MAKEPROD 'x 2)` y confirmar que dan el mismo par en ambos casos.
- ☐ Probar `(SIMPSUM '(+ (+ x (* -1 x)) y))` y confirmar que da `y` (el término en `x` se cancela y desaparece por completo).

☐ Probar `(SIMPSUM '(+ X (* -1 X)))` a solas y confirmar `0`.

14.12 Tabla de auditoría de nombres del capítulo 14

Nombre	Longitud	¿Colisiona?
COEF	4	no
LITERAL	7	no
FLATSUM	7	no
ASSOCADD	8	exacto, sin margen
AGRUPA	6	no
REBUILD	7	no
SIMPSUM	7	no

Un nombre en el límite exacto (`ASSOCADD`), sin colisión con nada de los capítulos anteriores.

14.13 Resumen de lo construido en este capítulo

- **COEF / LITERAL**: descomposición de un término en coeficiente y parte literal, insensible al orden de los factores en un producto.
- **FLATSUM**: aplanar una cadena de sumas anidadas en una lista Lisp corriente.
- **ASSOCADD / AGRUPA**: acumulación funcional pura (sin `SETQ`) de coeficientes por literal, usando `EQUAL` para comparar literales compuestos.
- **REBUILD / SIMPSUM**: reconstrucción del árbol apoyada por completo en `MAKESUM / MAKEPROD` del Capítulo 13, que resuelve gratis tanto el `+0` sobrante como la cancelación total de términos.
- **Cierre del hilo abierto desde el Capítulo 12**: $\frac{d}{dx}(x^2) = 2x$ sale ya exactamente como se escribiría a mano.
- **Un límite documentado con ejemplo concreto**: `SIMPSUM` no alcanza sumas anidadas dentro de productos o potencias — motivo de partida del Capítulo 15.

14.14 Siguiendo capítulo

`SIMPSUM` resuelve el problema que se planteó al principio del capítulo, pero tiene un límite que conviene dejar anotado con un ejemplo concreto antes de cerrar: solo aplanar la cadena de sumas que cuelgan **directamente** unas de otras a través de `+`. Si una suma queda anidada dentro de un producto o una potencia, `SIMPSUM` nunca llega a verla, porque `FLATSUM` solo desciende mientras el nodo raíz sea `ISSUM?`:

```
(SIMPSUM '(* 2 (+ X X))) ; -> (* 2 (+ X X)) -- NO simplifica el interior
```

Esto importa en la práctica: expresiones que salen de derivar bases compuestas (por ejemplo, la derivada de $(x+y)^2$) pueden producir sumas metidas dentro de productos, y `SIMPSUM` las dejaría intactas. El Capítulo 15 construye `SIMP`: un simplificador que recorre **todo** el árbol de forma recursiva —con el mismo patrón clasificar/bajar/ reconstruir de siempre — aplicando `SIMPSUM` en cada nodo suma que encuentre, estén donde estén anidados, y solo entonces tendrá sentido construir una función de nivel superior que encadene `SUBST`, `DERIV` y `SIMP` para uso directo del usuario final del CAS.

Queda además, anotado para más adelante y sin urgencia: un orden canónico estable entre sumandos *distintos* (no solo entre repeticiones del mismo literal, que ya resolvemos aquí), y extender el mismo patrón aplanar/agrupar a productos anidados y potencias ($x \cdot x \rightarrow x^2$, $(x^2) \cdot x \rightarrow x^3$), donde el "literal" pasaría a ser la base y el "coeficiente" el exponente.

Capítulo 15 — Simplificación de árbol completo (SIMP)

En una frase: `SIMPSUM` solo aplana la cadena de sumas que cuelgan directamente unas de otras; aquí construimos `SIMP`, una función que recorre **todo** el árbol de forma recursiva y aplica `SIMPSUM` en cada nodo suma que encuentre, esté donde esté anidado.

15.1 El límite exacto que dejamos abierto

Al cerrar el Capítulo 14 vimos un caso muy concreto que `SIMPSUM` no resuelve:

```
(SIMPSUM '(* 2 (+ X X))) ; -> (* 2 (+ X X)) -- el interior no cambia
```

La razón es estructural, no un descuido: `FLATSUM` solo desciende mientras el nodo que examina sea `ISSUM?`. En cuanto el nodo raíz es un producto, `FLATSUM` lo trata como un único término opaco y nunca mira dentro. `SIMPSUM` fue diseñada para resolver "una cadena de sumas", no "cualquier expresión que contenga sumas en cualquier posición" — y esa distinción es exactamente el problema de este capítulo.

15.2 La teoría: simplificar de abajo hacia arriba

La solución no es una función nueva y distinta de todo lo anterior — es el mismo patrón de recorrido que llevamos usando desde `SUBST` (clasificar el nodo, bajar recursivamente a los hijos, reconstruir), aplicado con una regla de orden muy concreta: **simplifica primero las partes, combina después el todo**. En términos técnicos, esto se llama recorrido en **postorden**: para simplificar un nodo, primero se simplifican completamente sus hijos, y solo cuando ambos hijos ya están en su forma más simple se combina el nodo actual.

Esto garantiza que, cuando lleguemos a examinar un nodo suma, cualquier suma que estuviera escondida más abajo en el árbol —dentro de un producto, dentro de una potencia, a cualquier profundidad— ya ha sido simplificada antes de que el nivel actual la vea. El árbol se limpia desde las hojas hacia la raíz, nunca al revés.

15.3 El código

```
(DEFUN SIMP (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E) E)
        ((ISSUM? E)
         (SIMPSUM (LIST '+ (SIMP (ADDEND E)) (SIMP (AUGEND E))))))
        ((ISPROD? E)
         (MAKEPROD (SIMP (FACTOR1 E)) (SIMP (FACTOR2 E))))
        ((ISPOW? E)
         (MAKEPOW (SIMP (BASE E)) (SIMP (EXPON E))))))
```

Tres apuntes sobre decisiones concretas:

- **Constantes y variables se devuelven tal cual** — son las hojas del árbol, no hay nada que simplificar en ellas.
- **El caso suma es el único que llama a `SIMPSUM`**, y lo hace *después* de simplificar recursivamente ambos hijos, no antes. Esto es lo que permite que una suma anidada varios niveles más abajo salga ya colapsada en el momento en que su padre —un producto, por ejemplo— la recibe de vuelta.
- **Los casos producto y potencia no llaman a `SIMPSUM`** — llaman a `MAKEPROD` / `MAKEPOW`, los mismos constructores inteligentes del Capítulo 13. Esto es deliberado: `SIMPSUM` sabe agrupar términos de una *suma*; combinar factores repetidos dentro de un *producto* ($x \cdot x \rightarrow x^2$) es un problema análogo pero distinto, que dejamos fuera de este capítulo a propósito.

15.4 Trazándolo a mano: SIMP sobre $(x^2 + x^2)$

1. `E` es $(x^2 + x^2) \rightarrow$ `ISPROD?` es verdadero.
2. Se simplifica `FACTOR1`: $(x^2) \rightarrow 2$ (constante, sin cambios).
3. Se simplifica `FACTOR2`: $(x^2 + x^2) \rightarrow$ aquí `ISSUM?` es verdadero, así que se llama a `(SIMPSUM (LIST '+ (SIMP 'X) (SIMP 'X))) = (SIMPSUM '(+ x x)) = (x^2 x). La suma que estaba escondida dentro del producto ya salió resuelta.`
4. Se reconstruye con `MAKEPROD`: $(x^2 (x^2 x))$. Ninguna regla de `MAKEPROD` se activa —no es cero, no es uno, y $(x^2 x)$ no es una constante— así que el resultado es $(x^2 (x^2 x))$.

15.5 Una victoria y un límite nuevo, ambos honestos

```
(SIMP '(x^2 (+ x x))) ; -> (x^2 (x^2 x))
```

La victoria: la suma interior **sí** se simplificó — ya no queda $(+ x x)$ sin tocar dentro del producto, que era exactamente el problema del Capítulo 14. El límite nuevo: el resultado matemáticamente sería $4x$, pero $(x^2 (x^2 x))$ se queda como un producto de un producto, sin colapsar. `MAKEPROD` sabe combinar dos números cuando los ve directamente como sus dos argumentos, pero aquí el segundo argumento no es un número — es la expresión $(x^2 x)$, que *contiene* un número en su interior. Detectar y combinar constantes anidadas dentro de productos es el mismo tipo de problema que resolvimos para sumas en el Capítulo 14 (aplanar, agrupar, reconstruir), aplicado ahora al producto — y es trabajo pendiente, no un error de este capítulo. Se anota como candidato para más adelante.

15.6 Probándolo — cerrando un caso más exigente que los del Capítulo 14

```
(SIMP '(x^2 (+ x x))) ; -> (x^2 (x^2 x))

; Una potencia con base compuesta: derivar (X+Y)^2 respecto a X
(DEFINE E4 (MAKEPOW (MAKESUM 'X 'Y) 2))
(DERIV E4 'X) ; -> (x^2 (+ x y)) (ya sale limpio, sin sums duplicadas)
(SIMP (DERIV E4 'X)) ; -> (x^2 (+ x y)) (SIMP no cambia nada, ya estaba bien)

; El caso que de verdad exigía SIMP frente a SIMPSUM: suma de dos derivadas
; iguales, cada una generando su propia suma interna
(DEFINE E5 (MAKESUM (MAKEPROD 'X 'X) (MAKEPROD 'X 'X)))
(DERIV E5 'X) ; -> (+ (+ x x) (+ x x))
(SIMP (DERIV E5 'X)) ; -> (x^4 x)
```

El último ejemplo es el que de verdad demuestra el valor de `SIMP` frente a `SIMPSUM` a solas: la derivada cruda tiene sumas anidadas dos niveles, y `SIMP` las colapsa todas hasta $(x^4 x)$ en una sola llamada, gracias a que simplifica de abajo hacia arriba antes de intentar combinar nada en el nivel superior.

15.7 Pendiente de verificar

- ☐ Probar $(x^2 + x^2)$ y confirmar $(x^2 (x^2 x))$ — la suma interior simplificada, el producto exterior sin colapsar (comportamiento esperado, no un error).
- ☐ Probar $(x^2 (x^2 x))$ con $E4 = (x+y)^2$ y confirmar $(x^2 (+ x y))$.
- ☐ Probar $(x^4 x)$ con $E5 = x \cdot x + x \cdot x$ y confirmar $(x^4 x)$ — el caso que distingue `SIMP` de `SIMPSUM` a solas.

15.8 Tabla de auditoría de nombres del capítulo 15

Nombre	Longitud	¿Colisiona?
SIMP	4	no

Un único nombre nuevo, sin conflicto con SIMPSUM ni con nada de los capítulos anteriores.

15.9 Siguiente capítulo

Con SIMP verificado, el Capítulo 16 construye el orquestador de nivel superior que encadena SUBST, DERIV y SIMP para uso directo del usuario final del CAS — el paso que deliberadamente no dimos en este capítulo, para no mezclar "construir el simplificador completo" con "decidir cómo se expone al usuario". Queda también anotado, sin urgencia, el límite descubierto aquí: combinar constantes anidadas dentro de productos ($2 \cdot (2 \cdot x) \rightarrow 4x$), con la misma técnica de aplanar/agrupar/reconstruir que ya usamos para sumas, aplicada esta vez al producto.

Capítulo 16 — Simplificador de productos (SIMPPROD)

En una frase: aplicamos exactamente la misma técnica del Capítulo 14 (aplanar, agrupar, reconstruir) al producto en vez de a la suma — cambiando "coeficiente que se suma" por "exponente que se suma", y "literal que se compara" por "base que se compara" — y con eso cerramos los dos huecos que dejamos anotados: $2 \cdot (2 \cdot x) \rightarrow 4x$ y $x \cdot x \rightarrow x^2$.

16.1 Los dos huecos concretos que dejamos abiertos

Al final del Capítulo 15 vimos que SIMP corrige las sumas anidadas dentro de un producto, pero deja sin resolver dos casos relacionados:

```
(SIMP '(* 2 (+ X X))) ; -> (* 2 (* 2 X)) -- no colapsa a (* 4 X)
(SIMPPROD '(* X X)) ; -- esta funcion ni siquiera existe todavia
```

El primero es "constantes anidadas dentro de un producto sin combinar". El segundo es "la misma variable multiplicada por sí misma no se reconoce como una potencia". Los dos son la misma clase de problema que resolvimos para sumas en el Capítulo 14: MAKEPROD solo ve dos argumentos a la vez, así que no puede ver "todos los factores de una cadena de productos" para agruparlos.

16.2 La idea: el mismo patrón, con los papeles intercambiados

Recordarás el patrón del Capítulo 14: para sumas, cada término se descompone en (coeficiente, literal), se agrupan los literales iguales sumando coeficientes, y se reconstruye. Para productos, el patrón es **estructuralmente idéntico**, pero con los papeles cambiados:

Capítulo 14 (suma)	Capítulo 16 (producto)
descomponer en (coeficiente, literal)	descomponer en (base, exponente)
agrupar literales iguales sumando coeficientes	agrupar bases iguales sumando exponentes
reconstruir con MAKEPROD(coef, literal) y sumar con MAKESUM	reconstruir con MAKEPOW(base, expon) y multiplicar con MAKEPROD

Esto no es una coincidencia de diseño — es la misma relación que existe entre la suma y el producto en las propias matemáticas: sumar la misma cantidad varias veces es multiplicarla ($x + x = 2x$), y multiplicar la misma cantidad varias veces es elevarla a una potencia ($x \cdot x = x^2$). El "coeficiente" de una suma es el "exponente" de un producto — ambos cuentan cuántas veces se repite lo mismo.

16.3 Los constructores auxiliares: **BASEOF** y **EXPOF**

Al igual que **COEF**/**LITERAL** sabían descomponer un término de suma, necesitamos algo que descomponga un factor de producto en su base y su exponente:

```
(DEFUN BASEOF (E) (COND ((ISPOW? E) (BASE E)) (T E)))
(DEFUN EXPOF (E) (COND ((ISPOW? E) (EXPON E)) (T 1)))
```

Si el factor ya es una potencia (x^2), su base es x y su exponente es 2 — nada que calcular. Si el factor es cualquier otra cosa (x , una constante, un producto sin simplificar...), se trata como "elevado a la 1", exactamente el mismo truco que en **LITERAL** devolviendo 1 para las constantes puras: permite que el caso general y el caso trivial se traten con el mismo código más adelante.

16.4 Aplanar el producto

```
(DEFUN FLATPROD (E)
  (COND ((ISPROD? E) (APPEND (FLATPROD (FACTOR1 E)) (FLATPROD (FACTOR2 E))))
  (T (LIST E))))
```

Idéntica a **FLATSUM**, cambiando **ISSUM?**/**ADDEND**/**AUGEND** por **ISPROD?**/**FACTOR1**/**FACTOR2**. (**FLATPROD** '(* 2 (* 2 X))) da la lista (2 2 X).

16.5 Separar constantes de símbolos

Aquí aparece una diferencia real con el capítulo 14: en una suma, todos los términos —constantes incluidas— se agrupan con el mismo mecanismo (una constante pura simplemente tiene literal 1). En un producto no podemos hacer lo mismo, porque el papel de las constantes es distinto: no se agrupan por "base" —tendría que ser algo como "base 2, exponente 2" para combinar $2 \cdot 2$, y eso mezclaría potencias de números con potencias de símbolos de forma confusa—. Es más simple y más claro multiplicar todas las constantes directamente entre sí, aparte:

```
(DEFUN NUMPROD (TERMS)
  (COND ((NULL TERMS) 1)
        ((ISCONST? (CAR TERMS)) (* (CAR TERMS) (NUMPROD (CDR TERMS))))
        (T (NUMPROD (CDR TERMS)))))

(DEFUN SYMTERMS (TERMS)
  (COND ((NULL TERMS) NIL)
        ((ISCONST? (CAR TERMS)) (SYMTERMS (CDR TERMS)))
        (T (CONS (CAR TERMS) (SYMTERMS (CDR TERMS))))))
```

NUMPROD recorre la lista y multiplica solo los elementos que son constantes, ignorando el resto — el caso base 1 es el elemento neutro del producto, igual que 0 lo era para **REBUILD** en la suma. **SYMTERMS** hace el filtrado complementario: se queda solo con los factores no constantes, que son los que sí hay que agrupar por base.

16.6 Agrupar por base, sumando exponentes


```
(DEFUN ADDBASE (B EXP ALIST)
  (COND ((NULL ALIST) (LIST (CONS B EXP)))
        ((EQUAL (CAR (CAR ALIST)) B)
         (CONS (CONS B (+ (CDR (CAR ALIST)) EXP)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ADDBASE B EXP (CDR ALIST))))))

(DEFUN AGRUPAP (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ADDBASE (BASEOF (CAR TERMS))
                      (EXPOF (CAR TERMS))
                      (AGRUPAP (CDR TERMS))))))
```

Línea por línea, es `ASSOCADD/AGRUPA` del Capítulo 14 con los nombres cambiados y `BASEOF/EXPOF` en vez de `LITERAL/COEF`. `(AGRUPAP '(X X))` da `((X . 2))` — la base `x` aparece dos veces, así que su exponente acumulado es 2.

16.7 Reconstruir

```
(DEFUN REBUILDP (ALIST)
  (COND ((NULL ALIST) 1)
        (T (MAKEPROD (MAKEPOW (CAR (CAR ALIST)) (CDR (CAR ALIST)))
                      (REBUILDP (CDR ALIST))))))

(DEFUN SIMPPROD (E)
  (LET ((TERMS (FLATPROD E)))
    (MAKEPROD (NUMPROD TERMS)
              (REBUILDP (AGRUPAP (SYMTERMS TERMS))))))
```

Por cada par `(base . exponente)` se construye `(MAKEPOW base exponente)` — que ya sabe reducir exponente 1 a la base sola, gracias al Capítulo 13 — y se multiplican todos con `MAKEPROD` en cascada. El caso base `(NULL ALIST) → 1` es el elemento neutro del producto, y por la misma razón que vimos en `REBUILD`, hace que el último término no arrastre un `•` 1 sobrante: `MAKEPROD` ya sabe eliminarlo.

`SIMPPROD` ata los tres pasos: aplanar el producto una sola vez (con `LET`, para no recalcarlo dos veces), multiplica aparte la parte numérica, y reconstruye la parte simbólica agrupada por base.

16.8 Integrando SIMPPROD en SIMP

Con `SIMPPROD` ya escrita, el cambio en `SIMP` es mínimo y simétrico al que ya tenía para sumas:

```
(DEFUN SIMP (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E) E)
        ((ISSUM? E)
         (SIMPSUM (MAKESUM (SIMP (ADDEND E)) (SIMP (AUGEND E)))))
        ((ISPROD? E)
         (SIMPPROD (MAKEPROD (SIMP (FACTOR1 E)) (SIMP (FACTOR2 E)))))
        ((ISPOW? E)
         (MAKEPOW (SIMP (BASE E)) (SIMP (EXPON E)))))
```

Antes, el caso `ISPROD?` llamaba directamente a `MAKEPROD`. Ahora llama a `SIMPPROD`, que hace todo lo que `MAKEPROD` ya hacía y además agrupa factores repetidos y combina constantes anidadas. La estructura del `COND` es idéntica a la de siempre — clasificar, bajar recursivamente, reconstruir con la función adecuada — solo cambia cuál es "la función adecuada" para el caso producto.

Un detalle importante sobre este código, corregido tras una revisión: los casos suma y producto reconstruyen el nodo intermedio con `MAKESUM / MAKEPROD` — los constructores del Capítulo 10/13 — y no con `(LIST '+ ...) / (LIST '* ...)` a mano. Usar `LIST` directamente aquí habría violado la barrera de abstracción del Capítulo 10: ninguna función fuera de ese capítulo debería construir un nodo sin pasar por sus constructores. El cambio es seguro porque, en el caso general (cuando los hijos ya simplificados son subexpresiones compuestas, no constantes ni cero), `MAKESUM / MAKEPROD` no activan ninguna de sus reglas especiales y caen en su cláusula final, que es literalmente `(LIST '+ A1 A2)` — idéntico a lo que se escribía a mano. Solo difieren cuando algún hijo ya es `0 / 1` o ambos son constantes, y en esos casos `SIMPSUM / SIMPPROD` habrían llegado al mismo resultado de todas formas a través de su propio aplanado — verificado con una batería de casos límite, incluida la cancelación total de un hijo antes de llegar a este punto.

16.9 Probándolo — cerrando los dos huecos

```
(SIMPPROD '(* 2 (* 2 X)))      ; -> (* 4 X)
(SIMPPROD '(* X X))            ; -> (POT X 2)
(SIMPPROD '(* (* X X) X))      ; -> (POT X 3)

; el caso que quedo abierto en el Capítulo 15, ahora resuelto de verdad
(SIMP '(* 2 (+ X X)))          ; -> (* 4 X)
```

El último ejemplo es el cierre real: `SIMP` ya recorre toda la expresión, simplifica la suma interior a `(* 2 X)` (Capítulo 15), y ahora también combina el `2` exterior con el `2` interior a través de `SIMPPROD`, dando `(* 4 X)` directamente — el resultado que un humano escribiría sin pensarlo dos veces.

16.10 Un límite que hereda de `DPOW`, no nuevo de este capítulo

`ADDBASE` suma exponentes con `+` real de MyLISP, lo que asume que los exponentes son siempre números conocidos. Esto es coherente con una limitación que ya existía desde `DPOW` en el Capítulo 12: nuestro CAS no soporta exponentes simbólicos (`xy` con `y` variable). Si alguna vez se extiende esa parte, `AGRUPAP / ADDBASE` tendrían que revisarse también — pero no es una regresión de este capítulo, es la misma limitación de siempre, ahora visible en un sitio más.

16.11 Pendiente de verificar

- ☐ Probar `(SIMPPROD '(* 2 (* 2 X)))` y confirmar `(* 4 X)`.
- ☐ Probar `(SIMPPROD '(* X X))` y confirmar `(POT X 2)`.
- ☐ Probar `(SIMPPROD '(* (* X X) X))` y confirmar `(POT X 3)`.
- ☐ Probar `(SIMP '(* 2 (+ X X)))` y confirmar `(* 4 X)` — el cierre real del hueco del Capítulo 15.
- ☐ Confirmar que no hay regresión: `(SIMP (DERIV (MAKEPROD 'X 'X) 'X))` sigue dando `(* 2 X)`, y el caso `E5` del Capítulo 15 sigue dando `(* 4 X)`.

16.12 Tabla de auditoría de nombres del capítulo 16

Nombre	Longitud	¿Colisiona?
BASEOF	6	no
EXPOF	5	no
FLATPROD	8	exacto, sin margen
NUMPROD	7	no
SYMTERMS	8	exacto, sin margen
ADDBASE	7	no
AGRUPAP	7	no
REBUILDP	8	exacto, sin margen
SIMPPROD	8	exacto, sin margen

Cuatro nombres en el límite exacto de 8 caracteres, ninguno colisiona con los más de 30 nombres ya definidos en capítulos anteriores.

16.13 Siguiendo capítulo

Con `SIMP` resolviendo ya sumas y productos anidados a cualquier profundidad, el Capítulo 17 puede construir por fin el orquestador de nivel superior que encadena `SUBST`, `DERIV` y `SIMP` para uso directo del usuario final del CAS — el paso que veníamos aplazando desde el Capítulo 15, ahora sobre una base de simplificación mucho más sólida. Queda también anotado, sin urgencia: el orden canónico estable entre sumandos y factores *distintos* (no solo entre repeticiones), que ya vimos en el Capítulo 14 que no está garantizado.

Capítulo 17 — El orquestador: DERIVA y DERIVAEN

En una frase: en vez de una mega-función con parámetros opcionales o modos, escribimos dos funciones pequeñas que componen las piezas ya existentes — `DERIVA` para "deriva y simplifica", `DERIVAEN` para "deriva, simplifica y evalúa en un punto" — siguiendo la misma filosofía de SICP: la composición de funciones ya es la interfaz, no hace falta construir una encima.

17.1 Por qué no una mega-función

Antes de llegar aquí se plantearon dos diseños alternativos y los descartamos los dos, por razones distintas:

- **Una única función con parámetros opcionales y `NIL` como interruptor** para apagar fases. Se rechaza porque `NIL` ya tiene un significado propio y cargado en nuestro CAS —la lista vacía, el caso base de media docena de recursiones— y reutilizarlo además como "esta fase no se ejecuta" es la misma sobrecarga de significado que evitamos deliberadamente al inventar el centinela `VARLIBRE` en el Capítulo 11 en vez de reutilizar algo genérico.
- **Una función con un modo explícito**, al estilo de un pequeño intérprete de comandos. Se rechaza porque sugiere que "derivar" y "sustituir" son alternativas mutuamente excluyentes, cuando en realidad son operaciones independientes que normalmente se **encadenan** (deriva, y luego simplifica, y luego, si hace falta, evalúa en un punto) — es una secuencia, no una elección.

La pista definitiva viene de mirar cómo resuelve esto el propio SICP: no lo resuelve con una función orquestadora. Cuando el texto quiere "derivar y simplificar", simplemente escribe `(simplify (deriv exp var))` en el sitio donde hace falta. La composición de funciones con paréntesis anidados **ya es** la interfaz — no hace falta construir nada por encima.

17.2 El código

```
(DEFUN DERIVA (E VBL) (SIMP (DERIV E VBL)))

(DEFUN DERIVAEN (E VBL VAL) (EVALEXPR (SUBST (DERIVA E VBL) VBL VAL)))
```

Dos funciones, cada una con una responsabilidad única y un nombre que la describe sin ambigüedad:

- **DERIVA** — "dame la derivada simbólica, ya simplificada". Cubre el caso de uso más común: nadie quiere ver $(+ (* 1 X) (* X 1))$, todo el mundo quiere $2x$.
- **DERIVAEN** — "dame el valor numérico de la derivada en un punto concreto". Compone **DERIVA** con **SUBST**/**EVALEXPR**, que ya existen y ya están verificados desde el Capítulo 11 — no se ha escrito ninguna lógica nueva de sustitución o evaluación aquí, solo se han conectado las piezas.

Ninguna de las dos necesita parámetros opcionales, banderas, ni modos.

17.3 Componer manualmente cuando hace falta más

Si una expresión tiene más de una variable, **DERIVAEN** solo sustituye la variable respecto a la que se deriva — el resto quedan libres a propósito. Para fijarlas todas, se compone a mano con **SUBST**, tal como haríamos con cualquier otra pieza del CAS:

```
; E4 ya esta definida desde el Capítulo 15 como (X + Y)^2
(DERIVA E4 'X)                                ; -> (* 2 (+ Y X)) ; Y libre

; Fijar X=3 e Y=4 a mano, encadenando SUBST:
(EVALEXPR (SUBST (SUBST (DERIVA E4 'X) 'X 3) 'Y 4))
; -> 14    (derivada = 2(x+y), en x=3,y=4 -> 2*7 = 14)
```

No hace falta ninguna función nueva para este caso — es exactamente la misma técnica de encadenar **SUBST** que ya usamos en el Capítulo 11, ahora aplicada sobre el resultado de **DERIVA** en vez de sobre una expresión suelta. Esta es la prueba de que el diseño de dos funciones pequeñas es suficiente: los casos que no cubren directamente se resuelven componiendo piezas que ya existían, sin tener que anticipar de antemano cada combinación posible con un parámetro dedicado.

17.4 Derivadas de segundo orden, sin ninguna función nueva

Un caso más exigente confirma que el diseño de dos funciones pequeñas también cubre derivadas de orden superior, sin haberlo planeado explícitamente: si **DFDX** es el resultado de **DERIVA**, ya es una expresión válida del CAS —construida con los mismos **MAKESUM**/**MAKEPROD**/**MAKEPOW** de siempre— así que **DERIVA** se puede volver a aplicar sobre ella sin ningún cambio.

```
; f(X,Y) = X^2 * Y^3
(DEFINE F (MAKEPROD (MAKEPOW 'X 2) (MAKEPOW 'Y 3)))

(DEFINE DFDX (DERIVA F 'X))
DFDX                                ; -> (* 2 (* (POT Y 3) X))

(DEFINE D2F (DERIVA DFDX 'Y))      ; derivada parcial mixta d2f/dXdY
D2F                                ; -> (* 6 (* (POT Y 2) X))

(EVALEXPR (SUBST (SUBST D2F 'X 3) 'Y 5))
; -> 450
```

Nota práctica: si al teclear este ejemplo en el QL de forma interactiva el intérprete parece "fallar" justo antes de `(DEFINE D2F ...)`, no es ningún error de `DERIVA` ni de `SIMP` — es el límite de ~80 caracteres del buffer de teclado de QDOS que ya conocemos desde el Capítulo 12. Ese límite no solo afecta al código en sí, también a cualquier comentario que se teclee en la misma sesión interactiva justo antes de la línea que falla. Al cargar por fichero con `LOAD` en vez de teclear a mano, este riesgo desaparece — otra razón más para preferir siempre esa vía cuando el código no sea trivial.

17.5 Probándolo

```
(DEFINE E7 (MAKEPOW 'X 2))
(DERIVA E7 'X)           ; -> (* 2 X)
(DERIVAEN E7 'X 5)       ; -> 10

(DEFINE E8 (MAKEPROD 'X 'X))
(DERIVA E8 'X)           ; -> (* 2 X)
(DERIVAEN E8 'X 5)       ; -> 10
```

17.6 Pendiente de verificar

- ☐ Probar `(DERIVA E7 'X)` y confirmar `(* 2 X)`.
- ☐ Probar `(DERIVAEN E7 'X 5)` y confirmar `10`.
- ☐ Probar la composición manual con dos variables (`E4`) y confirmar `14`.
- ☐ Probar la derivada parcial mixta de $x^2 \cdot y^3$ y confirmar que `(EVALEXPR (SUBST (SUBST D2F 'X 3) 'Y 5))` da `450`.

17.7 Tabla de auditoría de nombres del capítulo 17

Nombre	Longitud	¿Colisiona?
DERIVA	6	no
DERIVAEN	8	exacto, sin margen

Ambos nombres se distinguen claramente de `DERIV` (5 caracteres, ya existente) y no colisionan con nada de los siete capítulos anteriores.

17.8 Siguiente capítulo

Queda anotado, sin urgencia, el mismo pendiente que se arrastra desde el Capítulo 14: un orden canónico estable entre sumandos y factores *distintos* (no solo entre repeticiones del mismo literal o base), para que dos expresiones matemáticamente equivalentes produzcan siempre el mismo árbol, no solo el mismo valor. Es el candidato natural para retomar cuando se quiera seguir profundizando en el simplificador.

Capítulo 18 — Orden canónico estable (EXPLESS?)

En una frase: necesitamos un orden alfabético genuino entre variables distintas, y tenemos la suerte de que MyLISP ya nos da justo lo necesario — `SYMNAME` y `STR<` (§8.2.4-8.2.5) — así que este capítulo se reduce a usarlas para construir `EXPLESS?`, un comparador general de expresiones, y a ordenar con él los términos de una suma o un producto.

18.1 El problema que arrastrábamos

Desde el Capítulo 14 sabíamos que el orden de los sumandos distintos en el resultado de `SIMPSUM` no estaba garantizado — solo lo estaba el orden entre repeticiones del *mismo* literal. Lo vimos otra vez en el Capítulo 17: `(DERIVA '(+X Y)^2 'x)` daba a veces `(* 2 (+ Y X))` en vez del más intuitivo `(* 2 (+ X Y))`. Matemáticamente son idénticos, pero dos expresiones equivalentes que producen árboles distintos rompen la promesa de una verdadera forma canónica (recuerda la definición del Capítulo 14): si `x+y` a veces se representa como `(+ x y)` y otras veces como `(+ y x)`, comparar dos expresiones con `EQUAL` para saber si son "la misma" deja de funcionar.

La solución es ordenar los términos alfabéticamente antes de reconstruir el árbol, para que da igual en qué orden llegaran: el resultado final siempre coloca las variables en el mismo sitio. Para eso hace falta un comparador que sepa decidir, para cualquier par de nombres de variable, cuál va antes en el alfabeto.

18.2 `SYMNAME` y `STR<`: comparar símbolos como texto

Aquí tenemos la suerte de no tener que resolver nada nuevo en el intérprete: MyLISP ya incluye, desde el Capítulo 8, exactamente las dos primitivas que hacen falta.

- `(SYMNAME SIMBOLO)` (§8.2.4) convierte un símbolo a una cadena de texto (`TSTRING`) con su nombre. Es la puerta entre el mundo de los símbolos y el de las cadenas — y, como todo en MyLISP, el resultado queda truncado a 8 caracteres, la misma limitación que ya conocemos desde el Capítulo 10.
- `(STR< A B)` (§8.2.5) compara dos valores lexicográficamente y devuelve `T` si el primero precede al segundo. Acepta tanto cadenas como símbolos directamente — si se le pasan símbolos, los compara sin necesidad de convertirlos antes con `SYMNAME`.

Con estas dos primitivas, comparar dos variables alfabéticamente es tan simple como `(STR< 'MASA 'TIEMPO)`. De hecho, ya vimos en el Capítulo 8 que basta una línea para definir un comparador de símbolos reutilizable: `(DEFUN SYM< (A B) (STR< A B))`. `EXPRESS?`, el comparador que construimos en este capítulo, hace exactamente ese trabajo para variables, pero además sabe clasificar constantes y expresiones compuestas mediante `TIPORDEN`.

18.3 Por qué no basta con el orden de aparición

Antes de construir el comparador, merece la pena descartar una alternativa más simple que parece razonable a primera vista: ordenar los términos por el orden en que aparecieron en la expresión original, sin comparador alguno. Se puede comprobar fácilmente que no funciona: `(x+y)+x` y `y+(x+x)` son la misma suma matemática, pero un recorrido que respete el orden de aparición deja `x` primero en un caso e `y` primero en el otro — **no es canónico**. El problema exige, por definición, un criterio de orden independiente de cómo se anidó la entrada, y eso es justo lo que da una comparación alfabética real como la de `STR<`.

18.4 El comparador general: `EXPRESS?`

Con `SYMNAME` y `STR<` disponibles, `EXPRESS?` puede compararlo todo:

```
(DEFUN TIPORDEN (E)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) 1)
        (T 2)))

(DEFUN EXPRESS? (E1 E2)
  (COND ((< (TIPORDEN E1) (TIPORDEN E2)) T)
        ((> (TIPORDEN E1) (TIPORDEN E2)) NIL)
        ((ISCONST? E1) (< E1 E2))
        ((ISVAR? E1) (STR< E1 E2))
        (T NIL)))
```

`TIPORDEN` fija la prioridad más gruesa: las constantes van antes que las variables, que van antes que cualquier expresión compuesta (sumas, productos, potencias) — completamente general, basada solo en los predicados de clasificación del Capítulo 10. Dentro del mismo tipo, `EXPLESS?` desempata: dos constantes se comparan con `<` real; dos variables se comparan con `(STR< E1 E2)` — comparación alfabética genuina, para **cualquier** nombre de variable, no solo los que estén en una tabla. Dos expresiones compuestas siguen devolviendo `NIL` en ambas direcciones: ese es el único caso que queda sin resolver, y es un problema de otra naturaleza (compararía estructuras completas, no nombres) que dejamos anotado al final del capítulo.

18.5 Ordenar la lista de términos: inserción recursiva

MyLISP no tiene una función `sort` incorporada, así que escribimos la nuestra — un algoritmo de **ordenación por inserción**, clásico y fácil de razonar en términos recursivos:

```
(DEFUN INSERTA (PAR LISTA)
  (COND ((NULL LISTA) (LIST PAR))
        ((EXPLESS? (CAR PAR) (CAR (CAR LISTA))) (CONS PAR LISTA))
        (T (CONS (CAR LISTA) (INSERTA PAR (CDR LISTA))))))

(DEFUN ORDENA (LISTA)
  (COND ((NULL LISTA) NIL)
        (T (INSERTA (CAR LISTA) (ORDENA (CDR LISTA))))))
```

`ORDENA` recorre la lista hasta el final, ordena recursivamente el resto, y luego usa `INSERTA` para colocar el primer elemento en su sitio dentro de esa cola ya ordenada — el patrón de "ordenación por inserción" tal cual se enseña en cualquier curso de algoritmos, aquí expresado de forma puramente recursiva y sin ninguna variable mutable, como el resto del CAS. Nótese que `ORDENA` trabaja sobre listas de pares `(clave . valor)` — las mismas listas de asociación que ya produce `AGRUPA` / `AGRUPAP` — y compara únicamente las claves con `EXPLESS?`; el valor (`coeficiente` o `exponente`, según el caso) viaja sin tocarse.

18.6 Conectándolo a `SIMPSUM` y `SIMPPROD`

El cambio en las dos funciones ya existentes es mínimo, y es la señal de que la arquitectura por capas sigue pagando dividendos: basta con intercalar `ORDENA` entre `AGRUPA` / `AGRUPAP` y `REBUILD` / `REBUILDP`.

```
(DEFUN SIMPSUM (E) (REBUILD (ORDENA (AGRUPA (FLATSUM E)))))

(DEFUN SIMPPROD (E)
  (LET ((TERMS (FLATPROD E)))
    (MAKEPROD (NUMPROD TERMS)
              (REBUILDP (ORDENA (AGRUPAP (SYMTERMS TERMS)))))))
```

Ninguna otra función del CAS — `DERIV`, `SUBST`, `SIMP`, `DERIVA`, `DERIVAEN` — necesita cambiar ni una línea. Todas construyen sus resultados llamando a `SIMPSUM` / `SIMPPROD`, así que todas se benefician del nuevo orden estable de forma automática, exactamente igual que `DERIV` se benefició sin cambios de los constructores inteligentes en el Capítulo 13.

18.7 Probándolo

```
(SIMPSUM '(+ (+ X Y) X))      ; -> (+ (* 2 X) Y)
(SIMPSUM '(+ Y (+ X X)))      ; -> (+ (* 2 X) Y)    -- mismo resultado
(SIMPSUM '(+ (+ Y X) X))      ; -> (+ (* 2 X) Y)    -- mismo resultado otra vez

; el caso que arrastrabamos desde el Capitulo 17
(DEFINE E4 (MAKEPOW (MAKESUM 'X 'Y) 2))
(DERIVA E4 'X)                  ; -> (* 2 (+ X Y))    -- antes: (* 2 (+ Y X))

; simbolos de mas de una letra, no solo letras sueltas
(SIMPSUM '(+ TIEMPO MASA))      ; -> (+ MASA TIEMPO)    -- alfabetico real
(SIMPSUM '(+ MASA TIEMPO))      ; -> (+ MASA TIEMPO)    -- mismo resultado
```

Las tres primeras líneas parten de la misma suma matemática escrita en tres órdenes distintos, y las tres dan **exactamente el mismo árbol** — no solo el mismo valor. Las dos últimas muestran que `STR<` no se limita a variables de una sola letra: `MASA` y `TIEMPO` se ordenan de forma alfabética real y estable, exactamente igual que cualquier par de letras sueltas.

18.8 El límite que sigue abierto

Con `STR<`, la comparación entre **variables** no tiene ningún límite razonable: cualquier nombre, de cualquier longitud (hasta el truncamiento a 8 caracteres que ya conocemos desde el Capítulo 10, una limitación distinta y ya documentada), se ordena alfabéticamente de verdad. Lo único que sigue sin resolver `EXPRESS?` es comparar **dos expresiones compuestas entre sí** — por ejemplo, decidir si `(* X Y)` debería ir antes o después de `(POT Z 2)` en una suma que mezclara ambos como términos completos. Ese es un problema de otra naturaleza: no se trata de comparar nombres, sino de definir un orden entre estructuras completas (¿por su operador principal? ¿recursivamente por sus partes?), y queda fuera del alcance de este capítulo — anotado como posible ampliación futura, no como una limitación de plataforma como las anteriores.

18.9 Pendiente de verificar

- ☐ Probar las tres variantes de `(SIMPSUM ...)` con `X`, `Y` en distinto orden de entrada y confirmar que las tres dan `(+ (* 2 X) Y)`.
- ☐ Probar `(DERIVA E4 'X)` y confirmar `(* 2 (+ X Y))`.
- ☐ Probar `(SIMPSUM '(+ TIEMPO MASA))` y `(SIMPSUM '(+ MASA TIEMPO))` y confirmar que ambas dan `(+ MASA TIEMPO)` — orden alfabético real entre símbolos de más de una letra.
- ☐ Probar `(STR< 'MASA 'TIEMPO)` y `(SYMNAME 'X)` sueltos, para repasar estas dos primitivas del Capítulo 8 antes de verlas dentro de `EXPRESS?`.

18.10 Tabla de auditoría de nombres del capítulo 18

Nombre	Longitud	¿Colisiona?
<code>TIPORDEN</code>	8	exacto, sin margen
<code>EXPRESS?</code>	8	exacto, sin margen
<code>INSERTA</code>	7	no
<code>ORDENA</code>	6	no

Dos nombres en el límite exacto de 8 caracteres, ninguno colisiona con los más de 35 nombres ya definidos en capítulos anteriores.

`SYMNAME` y `STR<` no aparecen en esta tabla porque son primitivas del propio intérprete MyLISP, introducidas en el Capítulo 8 — no funciones definidas en este capítulo.

18.11 Siguiente capítulo

Con el orden canónico resuelto para el caso que importa, el Capítulo 19 puede abordar la impresión infija (`PRINT-MATH`): convertir el árbol del CAS en notación matemática legible ($2x + y$ en vez de `(+ (* 2 x) y)`), con el reto técnico de decidir cuándo hacen falta paréntesis según la precedencia del operador — el cierre natural de todo el manual.

Capítulo 19 — Impresión infija (PRINTMAT)

En una frase: convertimos el árbol del CAS en notación matemática legible — $2x + y$ en vez de `(+ (* 2 x) y)` — pasando hacia abajo, en cada llamada recursiva, la precedencia mínima que necesita un subárbol para no llevar paréntesis. Es el cierre natural del manual: el usuario deja de leer S-expressions y empieza a leer matemáticas.

19.1 Tres primitivas del Capítulo 8, ahora con un uso nuevo

Construir `PRINTMAT` necesita dos capacidades muy concretas: **concatenar texto** y **mostrarlo sin comillas ni salto de línea automático**. La buena noticia es que MyLISP ya nos las dio en el Capítulo 8 — `STRCAT`, `DISPLAY` y `NEWLINE` — pensadas entonces para construir mensajes y salidas con formato; aquí las reutilizamos para construir, pieza a pieza, la representación en texto de una expresión algebraica. `PRINT`, tal como lo conocíamos desde el Capítulo 11, añade `WRITELN` después de cada llamada y envuelve las cadenas en comillas — perfecto para depuración, inservible para construir una línea de salida pieza a pieza. El reparto de responsabilidades entre las tres es, de hecho, el mismo modelo que usa Scheme:

- `PRINT`** — depuración: muestra con comillas y salto de línea, devuelve el valor (para poder intercalarlo dentro de una expresión sin romper el flujo, como ya hacíamos en el Capítulo 11 con `VARLIBRE`).
- `DISPLAY`** — salida para el usuario: sin comillas, sin salto de línea.
- `NEWLINE`** — control explícito de cuándo terminar la línea.

19.2 La teoría: precedencia como "mínimo necesario para no llevar paréntesis"

La idea central es sencilla y potente: cada operador tiene una precedencia, y un subárbol necesita paréntesis cuando su propia precedencia es **menor** que la que le exige el contexto donde aparece. Formalizado como tabla:

Nodo	Precedencia
Constante o variable	4 (nunca necesita paréntesis)
Potencia (<code>POT</code>)	3
Producto (<code>*</code>)	2
Suma (<code>+</code>)	1

```
(DEFUN PRECOP (E)
  (COND ((ISPOW? E) 3)
        ((ISPROD? E) 2)
        ((ISSUM? E) 1)
        (T 4)))
```

La regla de decisión, con la que se envuelve o no un subárbol ya convertido a texto:

```
(DEFUN ENVOLVER (E S MINIMA)
  (COND (((< (PRECOP E) MINIMA) (STRCAT "(" (STRCAT S ")"))
    (T S))))
```

`ENVOLVER` recibe el nodo original `E` (para consultar su precedencia con `PRECOP`), el texto `S` ya generado para ese nodo, y la precedencia mínima que exige quien lo llamó. Si la precedencia propia es estrictamente menor que la exigida, se envuelve entre paréntesis; si no, se devuelve tal cual.

19.3 El matiz que tu propio ejemplo no cubría: la potencia no es asociativa

La regla "estrictamente menor" es exactamente correcta para `+` y `*` porque son operadores **asociativos**: `(+ x (+ y z))` se puede escribir sin paréntesis internos como `x + y + z` sin ambigüedad, así que un `+` anidado dentro de otro `+` (misma precedencia, no *menor*) no necesita envolverse — y la regla, tal cual, ya lo permite correctamente.

La potencia es distinta: `(x2)3` y `x(23)` **no son la misma expresión** (64 frente a 256 si sustituyes valores). Si aplicáramos la regla sin más matices a `(POT (POT X 2) 3)`, la base es ella misma una potencia con la *misma* precedencia (3) que el nodo padre — y "3 no es menor que 3", así que no se envolvería, dando `x2^3` sin paréntesis: ambiguo y matemáticamente distinto de lo que el árbol representa.

La solución no es una regla nueva — es pedir, **solo para la posición de la base de una potencia**, una precedencia mínima más estricta que la normal: en vez de exigir "al menos 3" (lo que dejaría pasar otra potencia sin paréntesis), exigimos "al menos 4" — es decir, que sea un átomo (constante o variable). Cualquier cosa que no sea un átomo puro como base de una potencia lleva paréntesis, sin excepción: `(x+y)2`, `(xy)2`, `(x2)3` — los tres casos, correctos, con la misma regla:

```
(DEFUN INFBASE (B) (INFIJO B 4))
```

El exponente, en cambio, no necesita este refuerzo: `xy^2` (una potencia como exponente de otra) se lee de forma natural como `x(y2)` —exponenciación asociando por la derecha, la convención matemática estándar—, que es exactamente lo que nuestro árbol `(POT X (POT Y 2))` representa. Ahí la regla normal (mínimo 3, no 4) es correcta tal cual.

19.4 Remates tipográficos que salen gratis de la misma técnica

Antes de escribir `INFIJO` en su forma final, dos detalles que mejoran mucho la lectura y que se resuelven con la misma herramienta (comprobar si un factor es una constante) sin añadir ningún concepto nuevo:

Yuxtaposición del coeficiente. `2 * x` se lee mejor como `2x`, sin el operador explícito — es la convención habitual en álgebra. Pero esto **solo** es seguro cuando uno de los factores es una constante numérica: para dos símbolos `( * x y )`, yuxtaponerlos daría `xy`, que sería ambiguo o directamente incorrecto para símbolos de más de una letra como los del Capítulo 18 (`MASA` y `TIEMPO` yuxtapuestos darían `MASATIEMP`, ilegible). Por eso la yuxtaposición se reserva **exclusivamente** para "constante multiplicando a cualquier otra cosa"; entre dos factores no constantes se usa `*` explícito.

Resta implícita. `x + (-3)` se lee mejor como `x - 3`, no como `x + -3`. Basta con detectar si el segundo sumando de una suma es "negativo" (una constante negativa, o un producto con coeficiente negativo) y, si lo es, imprimir " - " seguido de la versión con el signo invertido del mismo término — reutilizando `MAKEPROD`, que ya sabe simplificar el caso `-1 → nada` (coeficiente 1 se elimina solo, gracias al Capítulo 13).

```
(DEFUN NEGTERM? (E)
  (COND ((ISCONST? E) (< E 0))
    ((ISPROD? E)
      (COND ((ISCONST? (FACTOR1 E)) (< (FACTOR1 E) 0))
        ((ISCONST? (FACTOR2 E)) (< (FACTOR2 E) 0))
        (T NIL)))
    (T NIL)))
```

```
(DEFUN NEGAR (E)
  (COND ((ISCONST? E) (- E))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E))
                 (MAKEPROD (- (FACTOR1 E)) (FACTOR2 E)))
               ((ISCONST? (FACTOR2 E))
                 (MAKEPROD (FACTOR1 E) (- (FACTOR2 E))))
         (T E)))
        (T E)))
```

Un detalle elegante: `(NEGAR '(* -1 X))` da `(MAKEPROD 1 X)`, que `MAKEPROD` ya reduce a `X` solo (regla `UNO?` del Capítulo 13) — así que `- (-1)·X` se imprime como `- X`, no como `- 1x`, sin ningún caso especial adicional. La arquitectura de capas vuelve a pagar dividendos.

19.5 El código completo

```
(DEFUN PRECOP (E)
  (COND ((ISPOW? E) 3)
        ((ISPROD? E) 2)
        ((ISSUM? E) 1)
        (T 4)))

(DEFUN ENVOLVER (E S MINIMA)
  (COND (((< (PRECOP E) MINIMA) (STRCAT "(" (STRCAT S ")"))
        (T S)))

(DEFUN NEGTERM? (E)
  (COND ((ISCONST? E) (< E 0))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (< (FACTOR1 E) 0))
               ((ISCONST? (FACTOR2 E)) (< (FACTOR2 E) 0))
               (T NIL)))
        (T NIL)))

(DEFUN NEGAR (E)
  (COND ((ISCONST? E) (- E))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E))
                 (MAKEPROD (- (FACTOR1 E)) (FACTOR2 E)))
               ((ISCONST? (FACTOR2 E))
                 (MAKEPROD (FACTOR1 E) (- (FACTOR2 E))))
         (T E)))
        (T E)))

(DEFUN INFBASE (B) (INFIJO B 4))

(DEFUN INFPROD (E)
  (COND ((ISCONST? (FACTOR1 E))
        (COND ((= (FACTOR1 E) -1) (STRCAT "-" (INFIJO (FACTOR2 E) 2)))
              (T (STRCAT (FACTOR1 E) (INFIJO (FACTOR2 E) 2)))))
        ((ISCONST? (FACTOR2 E))
        (COND ((= (FACTOR2 E) -1) (STRCAT "-" (INFIJO (FACTOR1 E) 2)))
              (T (STRCAT (FACTOR2 E) (INFIJO (FACTOR1 E) 2)))))
        (T (STRCAT (INFIJO (FACTOR1 E) 2)
                    (STRCAT "*" (INFIJO (FACTOR2 E) 2))))))

(DEFUN INFSUMA (E)
```

```

(COND ((NEGTERM? (AUGEND E))
      (STRCAT (INFIJO (ADDEND E) 1)
              (STRCAT " - " (INFIJO (NEGAR (AUGEND E)) 1))))
(T (STRCAT (INFIJO (ADDEND E) 1)
          (STRCAT " + " (INFIJO (AUGEND E) 1)))))

(DEFUN INFIJO (E MINIMA)
  (COND ((ISCONST? E) (STRCAT E ""))
        ((ISVAR? E) (STRCAT E ""))
        ((ISPOW? E)
         (ENVOLVER E
                   (STRCAT (INFBASE (BASE E))
                           (STRCAT "^" (INFIJO (EXPON E) 3)))
                   MINIMA))
        ((ISPROD? E) (ENVOLVER E (INFPROD E) MINIMA))
        ((ISSUM? E) (ENVOLVER E (INFSUMA E) MINIMA))))

(DEFUN PRINTMAT (E) (PROGN (DISPLAY (INFIJO E 0)) (NEWLINE)))

```

`INFIJO` es el motor recursivo: clasifica el nodo, construye el texto de sus partes (bajando la precedencia mínima adecuada a cada posición — `2` para factores de un producto, `1` para sumandos, `3` para el exponente de una potencia, `4` reforzado para la base), y envuelve el resultado con `ENVOLVER` solo si hace falta. `PRINTMAT` es la función que usará el lector: convierte a texto con `MINIMA=0` (el nivel más bajo posible, así la expresión completa nunca se envuelve en paréntesis innecesarios en la raíz) y lo muestra con `DISPLAY + NEWLINE`.

19.6 Un límite de plataforma que hay que decir en voz alta: `STRCAT` trunca a 36 caracteres

Cada llamada a `STRCAT` que supere los 36 caracteres se trunca — y como `INFIJO` construye el resultado por concatenaciones sucesivas, una expresión cuya representación en texto supere esa longitud perdería información **en silencio**, sin ningún aviso. Para todas las expresiones de este manual —derivadas de una o dos variables, exponentes pequeños— el resultado siempre queda muy por debajo del límite (la derivada mixta del Capítulo 17, $6x*y^2$, ocupa 6 caracteres). Pero es una limitación real de esta implementación, no un descuido: `PRINTMAT`, tal como está, **no está pensado para expresiones muy largas**. Extenderlo correctamente exigiría no acumular el resultado completo en una sola cadena antes de mostrarlo, sino ir llamando a `DISPLAY` pieza a pieza a medida que se genera cada fragmento — un rediseño más profundo que queda fuera del alcance de este capítulo de cierre, y anotado aquí con la misma honestidad que el resto de límites de plataforma del manual.

19.7 Probándolo — el cierre del círculo

```

(PRINTMAT (MAKESUM (MAKEPROD 2 'X) 'Y))           ; 2x + y
(PRINTMAT (MAKEPROD 2 (MAKESUM 'X 'Y)))           ; 2(x + y)
(PRINTMAT (MAKESUM 2 (MAKEPROD 'X 'Y)))           ; 2 + x*y
(PRINTMAT (MAKEPOW (MAKEPOW 'X 2) 3))             ; (x^2)^3
(PRINTMAT (MAKEPOW 'X (MAKEPOW 'Y 2)))            ; x^y^2
(PRINTMAT (MAKESUM 'X (MAKEPROD -1 'Y)))           ; x - y
(PRINTMAT (MAKEPROD -1 (MAKESUM 'X 'Y)))           ; -(x + y)

; el cierre real: la derivada del Capítulo 17, impresa de verdad
(DEFINE E1 (MAKEPOW 'X 2))
(PRINTMAT (DERIVA E1 'X))                          ; 2x

; la derivada parcial mixta del Capítulo 17/18
(DEFINE F (MAKEPROD (MAKEPOW 'X 2) (MAKEPOW 'Y 3)))
(PRINTMAT (DERIVA (DERIVA F 'X) 'Y))               ; 6x*y^2

```

Tres capítulos y medio después de plantear el problema en el Capítulo 12 — derivar, simplificar, ordenar canónicamente, y ahora mostrar en notación legible — el CAS completo, de principio a fin, hace exactamente lo que se le pidió al primer párrafo de este manual: recibir una expresión matemática y devolver otra, en un lenguaje que un humano lee sin esfuerzo.

19.8 Pendiente de verificar

- ☐ Probar los siete casos sueltos de `PRINTMAT` y confirmar cada salida exacta.
- ☐ Probar `(PRINTMAT (DERIVA E1 'X))` y confirmar `2x`.
- ☐ Probar `(PRINTMAT (DERIVA (DERIVA F 'X) 'Y))` y confirmar `6x*y^2`.
- ☐ Confirmar que `(PRINTMAT (MAKEPOW (MAKEPOW 'X 2) 3))` da `(x^2)^3` **con** paréntesis — el caso que motivó el refuerzo de precedencia en la base de la potencia.

19.9 Tabla de auditoría de nombres del capítulo 19

Nombre	Longitud	¿Colisiona?
<code>PRECOP</code>	6	no
<code>ENVOLVER</code>	8	exacto, sin margen
<code>NEGTERM?</code>	8	exacto, sin margen
<code>NEGAR</code>	5	no
<code>INFBASE</code>	7	no
<code>INFPROD</code>	7	no
<code>INFSUMA</code>	7	no
<code>INFIJO</code>	6	no
<code>PRINTMAT</code>	8	exacto, sin margen

Tres nombres en el límite exacto de 8 caracteres, ninguno colisiona con los más de 40 nombres ya definidos a lo largo del manual. Nótese que el capítulo no se llama `PRINT-MATH` en el código —ese nombre tiene 10 caracteres y un guion que, aunque válido en un símbolo, no cabría en el límite de 8— sino `PRINTMAT`, la misma disciplina de nombrado que venimos aplicando desde el Capítulo 10.

19.10 Cierre del manual

Con `PRINTMAT`, el CAS construido a lo largo de estos diez capítulos queda completo de extremo a extremo: representación (Cap. 10), sustitución y evaluación (Cap. 11), derivación cruda (Cap. 12), simplificación local mediante constructores inteligentes (Cap. 13), agrupar términos semejantes (Cap. 14) y simplificación de árbol completo (Cap. 15-16), un orquestador de uso directo (Cap. 17), orden canónico estable (Cap. 18), y ahora una salida legible (Cap. 19) — todo ello verificado en hardware real en cada paso, con cada límite de la plataforma documentado en el momento en que se descubrió, no escondido. Quedan anotados, sin urgencia, los candidatos naturales para quien quiera seguir extendiendo el CAS: orden canónico entre expresiones compuestas, factores repetidos dentro de sumas de potencias, y los dominios explícitamente dejados fuera desde el principio (integración simbólica, resolución de ecuaciones, funciones trascendentes). El capítulo siguiente recoge todo esto en una guía de referencia y en un balance honesto de lo que este CAS sabe hacer y de lo que, deliberadamente, se queda fuera.

Capítulo 20 — Resumen de nuestro CAS: qué construimos y qué no

En una frase: cerramos la Parte II con una chuleta de uso —para cuando vuelvas dentro de unas semanas y solo quieras *usar* el CAS sin releer el código fuente— y con un balance honesto de lo que este pequeño sistema de álgebra simbólica sabe hacer y de lo que, deliberadamente, dejamos fuera.

20.1 Vocabulario básico (las reglas de entrada)

- Toda expresión de entrada se construye o bien con los constructores del Capítulo 10/13 (`MAKESUM`, `MAKEPROD`, `MAKEPOW`), o bien escribiendo directamente una lista citada con los tres operadores que el CAS reconoce: `'(+ ...)`, `'(* ...)`, `'(POT ...)`. Las dos formas producen exactamente la misma estructura interna; la lista citada es más rápida de teclear, los constructores son más cómodos cuando la expresión se arma dentro de código Lisp a partir de variables.
- Las variables de cálculo siempre van citadas: `'x`, no `x` a secas (si no, MyLISP intentaría evaluar `x` como si fuera una variable de programa, no un símbolo matemático).
- **No hay operador de resta ni de división en el árbol.** El CAS solo reconoce `+`, `*` y `POT`. Una resta se expresa como una suma con coeficiente negativo: `x - 3` se escribe `'(+ x -3)` o `(MAKESUM 'x -3)`; `x - y` se escribe `'(+ x (* -1 y))`. `PRINTMAT` (ver más abajo) sí sabe mostrar esto como `x - 3 / x - y` de forma legible, aunque internamente sea una suma.

20.2 Guía rápida de la API

Las funciones de uso directo:

Función	Qué hace
<code>(DERIVA E VBL)</code>	Deriva <code>E</code> respecto a <code>VBL</code> y devuelve el resultado ya simplificado.
<code>(DERIVAEN E VBL VAL)</code>	Deriva <code>E</code> respecto a <code>VBL</code> y evalúa el resultado sustituyendo <code>VBL</code> por <code>VAL</code> . Solo fija esa variable — si <code>E</code> tiene más de una, las demás quedan libres (ver el ejemplo de dos variables más abajo).
<code>(EVALEXPR (SUBST E VBL VAL))</code>	Evalúa numéricamente una expresión ya existente (sin derivar), fijando <code>VBL</code> en <code>VAL</code> .
<code>(PRINTMAT E)</code>	Muestra cualquier expresión del CAS en notación matemática legible.

Piezas para componer tú mismo (la filosofía del Capítulo 17: nada de funciones gigantes con banderas, se combinan estas piezas pequeñas según haga falta):

Función	Qué hace
<code>(SUBST E VBL VAL)</code>	Sustituye <code>VBL</code> por <code>VAL</code> dentro de <code>E</code> , sin evaluar.
<code>(DERIV E VBL)</code>	Derivada simbólica cruda , sin simplificar (la de <code>DERIVA</code> menos el paso de <code>SIMP</code>).
<code>(SIMP E)</code>	Simplifica cualquier expresión ya construida, recursivamente en todo el árbol.
<code>(EVALEXPR E)</code>	Evalúa numéricamente <code>E</code> si no tiene variables libres; si las tiene, devuelve el centinela <code>VARLIBRE</code> (Capítulo 11).

Para más de una variable libre, se encadena `SUBST` a mano tantas veces como haga falta antes de `EVALEXPR` — no existe (ni hace falta) una versión de `DERIVAEN` con más parámetros.

20.3 Tabla de traducción: matemáticas → MyLISP

Expresión matemática	Construcción en MyLISP
x^2	<code>(MAKEPOW 'X 2)</code> — O <code>'(POT X 2)</code>
$2x$	<code>(MAKEPROD 2 'X)</code> — O <code>'(* 2 X)</code>
$x + 3$	<code>(MAKESUM 'X 3)</code> — O <code>'(+ X 3)</code>
$x - 3$	<code>(MAKESUM 'X -3)</code> — O <code>'(+ X -3)</code>
$x \cdot y$	<code>(MAKEPROD 'X 'Y)</code> — O <code>'(* X Y)</code>
$x - y$	<code>'(+ X (* -1 Y))</code>
$x^2 + 3x$	<code>(MAKESUM (MAKEPOW 'X 2) (MAKEPROD 3 'X))</code> — O <code>'(+ (POT X 2) (* 3 X))</code>
$(x + y)^2$	<code>(MAKEPOW (MAKESUM 'X 'Y) 2)</code> — O <code>'(POT (+ X Y) 2)</code>
$x^2 \cdot y^3$	<code>(MAKEPROD (MAKEPOW 'X 2) (MAKEPOW 'Y 3))</code>

20.4 Ejemplo de flujo completo

```
; Definir la funcion: f(x) = x^2 + 3x
(DEFINE F '(+ (POT X 2) (* 3 X)))

; Ver la funcion legible:
(PRINTMAT F)                ; -> x^2 + 3x

; Ver su derivada legible:
(PRINTMAT (DERIVA F 'X))    ; -> 3 + 2x

; Calcular la pendiente en x=5:
(DERIVAEN F 'X 5)          ; -> 13
```

Fíjate en `3 + 2x`, no `2x + 3` — no es un error, es el orden canónico del Capítulo 18 aplicándose (las constantes van siempre antes que las variables). Si esto sorprende semanas después de haber cerrado el libro, este es el motivo exacto.

Con dos variables, `DERIVAEN` solo fija la que se deriva; la otra se sustituye a mano con `SUBST`, encadenando tantas llamadas como variables libres queden:

```
(DEFINE G (MAKEPOW (MAKESUM 'X 'Y) 2))    ; g(x,y) = (x+y)^2

(PRINTMAT (DERIVA G 'X))                  ; -> 2(x + y)
(EVALEXPR (SUBST (SUBST (DERIVA G 'X) 'X 3) 'Y 4)) ; -> 14
```

20.5 Lo que nuestro CAS sabe hacer

Vale la pena decirlo en positivo antes de pasar a las limitaciones, porque la lista es más larga de lo que parece cuando se ha ido construyendo capítulo a capítulo sin dar un paso atrás para mirar el conjunto:

- **Representa expresiones algebraicas arbitrarias** con sumas, productos y potencias anidados a cualquier profundidad, usando listas de MyLISP sin evaluar (Capítulo 10).
- **Sustituye variables por valores o por otras subexpresiones**, y evalúa numéricamente cuando ya no queda ninguna variable libre — detectando el caso contrario con su propio mecanismo de error, sin depender del entorno del intérprete anfitrión (Capítulo 11).
- **Deriva simbólicamente** usando las reglas mecánicas del cálculo (suma, producto, potencia con exponente

constante), incluyendo derivadas de segundo orden y derivadas parciales mixtas de varias variables (Capítulos 12 y 17).

- **Simplifica de verdad, no solo superficialmente:** aplica identidades algebraicas en el momento de construir cada nodo (Capítulo 13), combina términos semejantes dentro de sumas (Capítulo 14) y factores repetidos dentro de productos (Capítulo 16), y recorre el árbol completo para que la simplificación no se quede a medias en el primer nivel (Capítulo 15).
- **Produce un resultado canónico y estable:** la misma expresión matemática, escrita de formas distintas, siempre se reduce al mismo árbol interno — incluyendo un orden alfabético real entre variables de cualquier longitud (Capítulo 18).
- **Muestra el resultado en notación matemática legible,** con la puntuación mínima de paréntesis que necesita cada subexpresión según su precedencia (Capítulo 19) — en vez de obligar a leer S-expressions.
- Todo ello construido con las mismas piezas de MYLISP que se presentaron en la Parte I: listas, recursión, `LAMBDA`, `COND`, homoiconicidad — sin ningún truco ni primitiva mágica añadida a mitad de camino (§10.1).

20.6 Las fronteras conocidas: lo que nuestro CAS no hace

- **No soporta exponentes simbólicos** (`xy` con `y` variable) — `DPOW` / `ADDBASE` asumen que el exponente es siempre una constante numérica conocida (Capítulos 12 y 16). Un CAS real necesitaría reglas de derivación distintas para este caso (la regla general de derivación logarítmica), que quedan fuera del alcance de este manual.
- **No garantiza un orden canónico entre estructuras compuestas** — solo entre constantes y entre variables (comparación alfabética real desde el Capítulo 18). Dos sumas o productos de expresiones compuestas pueden aparecer en distinto orden según cómo se construyeron.
- **PRINTMAT no imprime expresiones muy largas** — cada pieza intermedia que genera pasa por `STRCAT`, que trunca a 36 caracteres (Capítulo 19). Para las expresiones de este manual nunca se llega a ese límite, pero es una limitación real, no cosmética.
- **La tabla de cadenas del intérprete tiene solo 50 entradas,** y el recolector de basura solo se dispara mirando la ocupación del heap principal, no la de `strtab` (hallazgo del Capítulo 19). Si encadenas muchas llamadas a `PRINTMAT` / `STR<` / `SYMNAME` seguidas en una sesión larga, puede agotarse antes de que el heap principal dé motivo a un GC. Si pasa, reinicia el intérprete o divide las pruebas en bloques más pequeños cargados por separado.
- **No incluye integración simbólica, resolución de ecuaciones, ni funciones trascendentes** (`SIN`, `COS`, `LOG` ...) — quedaron fuera de alcance desde el planteamiento inicial del manual, no por limitación de última hora. Cada una de ellas es, en sí misma, un capítulo entero de reglas nuevas: integrar exige reconocer patrones y a veces no tiene solución elemental; resolver ecuaciones exige manipular la expresión hacia el lado contrario de un `=` que este CAS ni siquiera representa; las funciones trascendentes exigirían extender `DERIV`, `SIMP` y `PRINTMAT` con casos nuevos para cada una.

20.7 De este CAS a un CAS real

El §10.1 abría la Parte II recordando que sistemas como Macsyma, Maple, Mathematica, Maxima o SymPy hacen, en esencia, lo mismo que este CAS pequeño: representar expresiones simbólicamente y transformarlas aplicando reglas. La diferencia no es de naturaleza, sino de escala — más reglas de derivación e integración cubriendo más funciones, más estrategias de simplificación compitiendo entre sí para elegir la forma "más simple" (un problema nada trivial: ¿qué es más simple, $(x+1)^2$ o x^2+2x+1 ?), soporte para números complejos y matrices, resolución de ecuaciones y sistemas, y años de trabajo en optimizar todo eso para que sea rápido con expresiones de miles de términos.

Pero la arquitectura de fondo —representar, recorrer recursivamente, simplificar por reescritura de reglas, mantener una forma canónica para poder comparar— es exactamente la que se ha construido aquí, capítulo a capítulo, con las herramientas más simples de LISP. Quien haya seguido el manual hasta aquí tiene ya la base conceptual para leer el código fuente de un CAS real y reconocer, debajo de la complejidad añadida, las mismas piezas: constructores inteligentes, simplificadores recursivos, un orden canónico, y una función que convierte árboles en texto legible.

Apéndices

Apéndice A — Referencia rápida

A.1 Formas especiales

Forma	Sintaxis	Descripción
QUOTE	(QUOTE expr) o 'expr	Devuelve expr sin evaluar
IF	(IF cond then [else])	Bifurcación condicional
COND	(COND (c1 e1) ... (T eN))	Multi-caso
AND	(AND e1 e2 ...)	Conjunción con cortocircuito
OR	(OR e1 e2 ...)	Disyunción con cortocircuito
PROGN	(PROGN e1 e2 ...)	Secuencia; devuelve la última
LET	(LET ((v1 e1) ...) cuerpo)	Variables locales (vinculación paralela)
LAMBDA	(LAMBDA (params) cuerpo)	Función anónima / clausura
DEFINE	(DEFINE nombre valor)	Definición global
DEFUN	(DEFUN nombre (params) cuerpo)	Función con nombre (atajo de DEFINE+LAMBDA)
LOAD	(LOAD "mdvX_nombre")	Cargar y evaluar un fichero

A.2 Primitivas de lista

Función	Sintaxis	Descripción
CAR	(CAR lista)	Primer elemento
CDR	(CDR lista)	Resto (todo excepto el primero)
CONS	(CONS elem lista)	Construir par/lista
LIST	(LIST e1 e2 ...)	Construir lista de elementos
APPEND	(APPEND lista1 lista2)	Concatenar dos listas
NULL	(NULL expr)	¿Es NIL?
ATOM	(ATOM expr)	¿Es un átomo?
LISTP	(LISTP expr)	¿Es una lista?

A.3 Primitivas numéricas

Función	Sintaxis	Descripción
<code>+</code>	<code>(+ n1 n2 ...)</code>	Suma (variádica)
<code>-</code>	<code>(- n1 n2 ...)</code> o <code>(- n)</code>	Resta / negación
<code>*</code>	<code>(* n1 n2 ...)</code>	Multiplicación (variádica)
<code>/</code>	<code>(/ n1 n2 ...)</code> o <code>(/ n)</code>	División exacta / inverso. Produce racional si no hay resto
<code>DIV</code>	<code>(DIV a b)</code>	Cociente entero (trunca hacia cero). Solo TINT
<code>MOD</code>	<code>(MOD a b)</code>	Resto entero (mismo signo que a). Solo TINT
<code>=</code>	<code>(= n1 n2)</code>	Igualdad numérica (vía float)
<code><</code>	<code>(< n1 n2)</code>	Menor que
<code>></code>	<code>(> n1 n2)</code>	Mayor que
<code><=</code>	<code>(<= n1 n2)</code>	Menor o igual
<code>>=</code>	<code>(>= n1 n2)</code>	Mayor o igual
<code>NUMBERP</code>	<code>(NUMBERP expr)</code>	¿Es un número?

A.4 Predicados de igualdad

Función	Compara	Tipos	Uso
<code>EQ</code>	Identidad de símbolo	Sólo símbolos y NIL	Comparar símbolos
<code>=</code>	Valor numérico (via float)	Números de cualquier tipo	Igualdad numérica
<code>EQUAL</code>	Estructura y tipo	Cualquier cosa	Igualdad estructural

A.5 Otras primitivas

Función	Sintaxis	Descripción
<code>NOT</code>	<code>(NOT expr)</code>	Negación lógica
<code>SYMBOLP</code>	<code>(SYMBOLP expr)</code>	¿Es un símbolo?
<code>EVAL</code>	<code>(EVAL expr)</code>	Evaluar una expresión
<code>PRINT</code>	<code>(PRINT expr)</code>	Imprimir con formato de lector + salto de línea; devuelve el argumento
<code>DISPLAY</code>	<code>(DISPLAY expr)</code>	Imprimir sin comillas ni salto de línea; devuelve VOID
<code>NEWLINE</code>	<code>(NEWLINE)</code>	Emitir un salto de línea; devuelve NIL
<code>SYMNAME</code>	<code>(SYMNAME símbolo)</code>	Devolver el nombre del símbolo como cadena (máx. 8 chars)
<code>STR<</code>	<code>(STR< a b)</code>	Comparación alfabética; acepta TSTRING o TSYM
<code>STRCAT</code>	<code>(STRCAT a b)</code>	Concatenar dos valores como cadena; acepta TSTRING, TSYM, TINT, TRAT; trunca a 36 chars

A.6 Funciones definibles en LISP puro

Estas funciones **no son primitivas** de MYLISP; deben definirse antes de usarlas.

```
; Longitud de lista
(DEFUN LONGIT (L)
  (IF (NULL L) 0 (+ 1 (LONGIT (CDR L)))))

; N-ésimo elemento (base 0)
(DEFUN NTHEL (L N)
  (IF (= N 0) (CAR L) (NTHEL (CDR L) (- N 1))))

; Abreviaturas CADR, CADDR
(DEFUN CADR (L) (CAR (CDR L)))
(DEFUN CADDR (L) (CAR (CDR (CDR L))))
(DEFUN CADDR (L) (CAR (CDR (CDR (CDR L)))))

; Valor absoluto
(DEFUN ABSOL (X) (IF (< X 0) (- X) X))

; Máximo y mínimo
(DEFUN MAXI (A B) (IF (> A B) A B))
(DEFUN MINI (A B) (IF (< A B) A B))

; Invertir lista
(DEFUN INVERT (L)
  (DEFUN INVERTA (L ACC)
    (IF (NULL L) ACC (INVERTA (CDR L) (CONS (CAR L) ACC)))))
  (INVERTA L NIL))

; MAP, FILTER, REDUCE
(DEFUN MY-MAP (F L)
  (IF (NULL L) NIL
    (CONS (F (CAR L)) (MY-MAP F (CDR L)))))

(DEFUN MY-FILTR (P L)
  (COND ((NULL L) NIL)
        ((P (CAR L)) (CONS (CAR L) (MY-FILTR P (CDR L))))
        (T (MY-FILTR P (CDR L)))))

(DEFUN MY-REDUC (F I L)
  (IF (NULL L) I
    (MY-REDUC F (F I (CAR L)) (CDR L))))

; MCD (algoritmo de Euclides)
(DEFUN MCD (A B)
  (IF (= B 0) A (MCD B (MOD A B))))

; Raíz cuadrada (Newton-Raphson)
(DEFUN MYSQRT (X) (SQRTITR 1.0 X))
(DEFUN SQRTITR (G X)
  (IF (< (ABSOL (- (* G G) X)) 0.001)
    G (SQRTITR (/ (+ G (/ X G)) 2.0) X)))
```

Apéndice B — Arquitectura y limitaciones

B.1 Las dos implementaciones de MYLISP

MYLISP existe en dos implementaciones que comparten el mismo lenguaje pero difieren en arquitectura interna:

MYLISP/QL está escrito en **Prospero Pro Pascal** y compilado para el Sinclair QL (Motorola 68008). Usa un **evaluador recursivo de descenso**: la función `Eva1` del Pascal llama recursivamente a sí misma para evaluar subexpresiones; cada llamada a función LISP consume un marco de la pila de Pascal.

MYLISP/Next está escrito en **C con Z88DK** para el ZX Spectrum Next (Z80 a 28 MHz). Usa un **evaluador iterativo** con una pila explícita de tareas almacenada en memoria paginada. La recursión de usuario no consume pila de CPU; su límite real es el heap, no la profundidad de llamadas.

B.2 Tabla comparativa de plataformas

Característica	MYLISP/QL	MYLISP/Next
Hardware	Sinclair QL, 68008 a 7,5 MHz	ZX Spectrum Next, Z80 a 28 MHz
Compilador / lenguaje	Prospero Pro Pascal	Z88DK / C
Almacenamiento	Microdrive (mdv1, mdv2)	Tarjeta SD (esxdos)
Evaluador	Recursivo (consume pila Pascal)	Iterativo (pila explícita paginada)
Heap (<code>MAXCELL</code>)	24.000 celdas	32.000 celdas
Tabla de símbolos (<code>MAXSYM</code>)	200	200
Longitud de símbolo (<code>SYMLEN</code>)	8 chars (trunca)	8 chars (trunca)
Tabla de cadenas (<code>MAXSTR</code>)	50	50
Longitud máx. cadena (<code>STRLEN</code>)	36 chars	36 chars
Flotantes	Tabla separada (<code>MAXREAL</code> =100)	Empaquetados en la celda
Buffer REPL (total acumulado)	80 caracteres	500 caracteres
Buffer LOAD	500 caracteres	500 caracteres
GC en REPL	Al 80% del heap	Al 80% del heap
GC durante LOAD	Al 50% del heap	No se dispara
(<code>CLEAN</code>) desde dentro de LOAD	Ejecuta el GC	Ignorado con aviso
Profundidad de recursión	Limitada por pila Pascal	Limitada solo por el heap
Exit	<code>BYE</code>	<code>BYE</code>

B.3 El REPL y sus límites

El REPL acepta expresiones multilinea. La longitud total acumulada de una expresión tiene un límite que varía por plataforma (ver tabla B.2). Para programas de más de unas pocas líneas, siempre es mejor guardar el código en fichero y cargarlo con `LOAD`.

B.4 El espacio de nombres (Lisp-1)

MYLISP es un **Lisp-1**: hay un único entorno que contiene tanto variables como funciones. Definir `(DEFINE SUMA 42)` después de `(DEFUN SUMA (A B) (+ A B))` destruye la función y la reemplaza por el número.

B.5 El recolector de basura

MYLISP utiliza un recolector de basura **mark-and-sweep**:

1. **Marca:** recorre todas las celdas alcanzables desde el entorno global y las marca como "vivas".
2. **Barre:** libera todas las celdas no marcadas, devolviéndolas a la lista libre.

El GC es completamente silencioso: el usuario no recibe ningún aviso cuando se ejecuta automáticamente. Los umbrales de disparo y el comportamiento durante `LOAD` varían entre plataformas (ver tabla B.2).

Garantía importante: el GC nunca se activa en medio de una evaluación, sólo entre expresiones completas. Esto garantiza que no puede corromper estructuras parcialmente construidas.

B.6 Tipos de dato internos

Los tipos son idénticos en ambas plataformas. La única diferencia es la representación interna de `TFLOAT`: en el QL se almacena como índice en una tabla separada; en el Next se empaqueta directamente en los bytes de la celda.

Tipo	Tipo LISP	Descripción
<code>TPAIR</code>	Par/lista	Cons cell: dos referencias (CAR, CDR)
<code>TSYM</code>	Símbolo	Índice en la tabla de símbolos
<code>TINT</code>	Entero	Valor entero de 32 bits con signo ($\pm 2.147.483.647$)
<code>TRAT</code>	Racional	Par (numerador, denominador) enteros
<code>TFLOAT</code>	Real	Número en coma flotante
<code>TNIL</code>	NIL	La lista vacía / valor falso
<code>TCLOSURE</code>	Clausura	Par (LAMBDA, entorno)
<code>TSTRING</code>	Cadena	Texto de hasta 36 caracteres

B.7 Funciones NO implementadas

Las siguientes funciones **no existen** en MYLISP como primitivas. No uses sus nombres esperando que funcionen:

`ABS`, `MAX`, `MIN`, `SQRT`, `EXPT`, `REVERSE`, `LENGTH`, `MAP`, `FILTER`, `REDUCE`, `APPLY`

La mayoría pueden definirse en LISP puro (véase el Apéndice A.6).

B.8 Diferencias con Common LISP y Scheme

Característica	MYLISP	Common LISP	Scheme
Espacio de nombres	Lisp-1	Lisp-2	Lisp-1
Booleanos	NIL / cualquier otra cosa	NIL / T	#f / #t
Truncado de nombres	8 chars	Ilimitado	Ilimitado
TCO	No	No (en general)	Sí (requerido)
Enteros exactos	32 bits, sin promoción	Bignum	Bignum
Racionales	Sí, nativo	Sí	Sí
Exit	BYE	(QUIT)	(exit)

Apéndice C — Mensajes de error

C.1 Errores aritméticos

Mensaje	Causa
ERROR: valor fuera de rango	El resultado de una operación entera excede el rango de 32 bits ($\pm 2.147.483.647$).
Use numeros reales (ej. 1.0) si necesita rangos mayores.	Nota complementaria al desbordamiento de entero.
ERROR: division por cero	Intento de dividir por 0 (entero, racional o real).

C.2 Errores de estructura

Mensaje	Causa
ERROR: CAR requiere una lista no vacia	Se aplicó CAR a NIL.
ERROR: CDR requiere una lista no vacia	Se aplicó CDR a NIL.

C.3 Errores de evaluación

Mensaje	Causa
ERROR: simbolo no definido: XXXX	Se referencia un símbolo que no tiene valor en el entorno. El nombre mostrado es el símbolo truncado a 8 caracteres.

C.4 Diagnóstico de errores comunes

- "símbolo no definido: FACTORIA" — Intentaste llamar a FACTORIAL (9 chars) que MYLISP almacena como FACTORIA. Usa el nombre truncado en tu código.
- "valor fuera de rango" — Resultado fuera del rango de 32 bits ($\pm 2.147.483.647$). Opciones: usar reales (1.0 en lugar de 1), o rediseñar el algoritmo.
- "CAR requiere una lista no vacia" — Intentaste hacer (CAR NIL). Añade un test (NULL lista) antes de llamar a CAR.

El REPL no responde — Probablemente tienes paréntesis sin cerrar. Escribe los paréntesis que faltan y pulsa ENTER. Si no funciona, escribe `BYE` y reinicia.

Apéndice D — Referencia rápida del CAS: todas las funciones

A diferencia de la guía de uso del Capítulo 20 —pensada para quien solo quiere *usar* el CAS ya construido—, este apéndice cataloga **las 59 funciones de `CAS.LSP`**, una por una, tal como quedaron en su versión final tras los diez capítulos de la Parte II. Es la referencia a la que acudir para saber, de un vistazo, qué recibe y qué devuelve cualquier función del fichero.

D.1 Representación y constructores (Capítulo 10)

Función	Entrada	Devuelve
ISCONST?	(ISCONST? E)	T si E es una constante numérica; NIL en caso contrario.
ISVAR?	(ISVAR? E)	T si E es un símbolo (variable); NIL en caso contrario.
ISSUM?	(ISSUM? E)	T si E es una lista cuya cabeza es +; NIL en caso contrario.
ISPROD?	(ISPROD? E)	T si E es una lista cuya cabeza es *; NIL en caso contrario.
ISPOW?	(ISPOW? E)	T si E es una lista cuya cabeza es POT; NIL en caso contrario.
ADDEND	(ADDEND E)	El primer sumando de una suma E.
AUGEND	(AUGEND E)	El segundo sumando de una suma E.
FACTOR1	(FACTOR1 E)	El primer factor de un producto E.
FACTOR2	(FACTOR2 E)	El segundo factor de un producto E.
BASE	(BASE E)	La base de una potencia E.
EXPON	(EXPON E)	El exponente de una potencia E.
CERO?	(CERO? X)	T si X es la constante 0; NIL en caso contrario.
UNO?	(UNO? X)	T si X es la constante 1; NIL en caso contrario.
MAKESUM	(MAKESUM A1 A2)	La suma $A1 + A2$, simplificada en el momento de construirla (elimina ceros, combina constantes); si no aplica ninguna regla, $(+ A1 A2)$.
MAKEPROD	(MAKEPROD A1 A2)	El producto $A1 \cdot A2$, simplificado en el momento de construirlo (elimina ceros/unos, combina constantes); si no aplica ninguna regla, $(* A1 A2)$.
MAKEPOW	(MAKEPOW B E)	La potencia B^E , simplificada en el momento de construirla (exponente 0 o 1, base 1, combinación de constantes); si no aplica ninguna regla, $(POT B E)$.

D.2 Sustitución y evaluación (Capítulo 11)

Función	Entrada	Devuelve
SUBST	(SUBST E VBL VAL)	E con todas las apariciones de la variable VBL sustituidas por VAL, sin evaluar.
LIBRE?	(LIBRE? X)	T si X es el centinela VARLIBRE; NIL en caso contrario.
POTENCIA	(POTENCIA B N)	B elevado a N (entero), calculado por multiplicación recursiva.
EVALSUMA	(EVALSUMA E)	El valor numérico de la suma E, o VARLIBRE si algún sumando tiene una variable libre.
EVALPROD	(EVALPROD E)	El valor numérico del producto E, o VARLIBRE si algún factor tiene una variable libre.
EVALPOW	(EVALPOW E)	El valor numérico de la potencia E, o VARLIBRE si la base o el exponente tienen una variable libre.
EVALEXPR	(EVALEXPR E)	El valor numérico de E si no tiene variables libres; VARLIBRE (con un aviso impreso) en caso contrario.

D.3 Derivación cruda (Capítulo 12)

Función	Entrada	Devuelve
DSUMA	(DSUMA E VBL)	La derivada cruda (sin simplificar) de una suma E respecto a VBL.
DPROD	(DPROD E VBL)	La derivada cruda de un producto E respecto a VBL, por la regla del producto.
DPOW	(DPOW E VBL)	La derivada cruda de una potencia E respecto a VBL (exponente constante).
DERIV	(DERIV E VBL)	La derivada simbólica cruda de E respecto a VBL, sin simplificar.

D.4 Orden canónico y términos semejantes (Capítulo 14)

Función	Entrada	Devuelve
COEF	(COEF E)	El coeficiente numérico de un término E (1 si no tiene ninguno explícito).
LITERAL	(LITERAL E)	La parte literal (no numérica) de un término E.
FLATSUM	(FLATSUM E)	La lista de todos los sumandos de E, aplanando cualquier cadena de sumas anidadas.
ASSOCADD	(ASSOCADD LIT CF ALIST)	ALIST actualizada: suma CF al coeficiente ya acumulado para LIT, o añade una entrada nueva si LIT no estaba.
AGRUPA	(AGRUPA TERMS)	Una lista de asociación (literal . coeficiente), agrupando los términos semejantes de TERMS.
REBUILD	(REBUILD ALIST)	La suma reconstruida a partir de la lista de asociación ALIST.
SIMPSUM	(SIMPSUM E)	La suma E simplificada: términos semejantes agrupados y en orden canónico.

D.5 Simplificación de árbol completo (Capítulo 15)

Función	Entrada	Devuelve
<code>SIMP</code>	<code>(SIMP E)</code>	<code>E</code> completamente simplificado, recorriendo y simplificando cada nodo del árbol.

D.6 Simplificador de productos (Capítulo 16)

Función	Entrada	Devuelve
<code>BASEOF</code>	<code>(BASEOF E)</code>	La base de <code>E</code> si es una potencia; <code>E</code> mismo en caso contrario.
<code>EXPOF</code>	<code>(EXPOF E)</code>	El exponente de <code>E</code> si es una potencia; <code>1</code> en caso contrario.
<code>FLATPROD</code>	<code>(FLATPROD E)</code>	La lista de todos los factores de <code>E</code> , aplanando cualquier cadena de productos anidados.
<code>NUMPROD</code>	<code>(NUMPROD TERMS)</code>	El producto de todas las constantes numéricas presentes en <code>TERMS</code> .
<code>SYMTERMS</code>	<code>(SYMTERMS TERMS)</code>	Los términos de <code>TERMS</code> que no son constantes numéricas.
<code>ADDBASE</code>	<code>(ADDBASE B EXP ALIST)</code>	<code>ALIST</code> actualizada: suma <code>EXP</code> al exponente ya acumulado para la base <code>B</code> , o añade una entrada nueva si <code>B</code> no estaba.
<code>AGRUPAP</code>	<code>(AGRUPAP TERMS)</code>	Una lista de asociación <code>(base . exponente)</code> , agrupando los factores repetidos de <code>TERMS</code> .
<code>REBUILDP</code>	<code>(REBUILDP ALIST)</code>	El producto reconstruido a partir de la lista de asociación <code>ALIST</code> .
<code>SIMPPROD</code>	<code>(SIMPPROD E)</code>	El producto <code>E</code> simplificado: factores repetidos combinados en potencias y en orden canónico.

D.7 El orquestador (Capítulo 17)

Función	Entrada	Devuelve
<code>DERIVA</code>	<code>(DERIVA E VBL)</code>	La derivada de <code>E</code> respecto a <code>VBL</code> , ya simplificada.
<code>DERIVAEN</code>	<code>(DERIVAEN E VBL VAL)</code>	La derivada de <code>E</code> respecto a <code>VBL</code> , evaluada numéricamente sustituyendo <code>VBL</code> por <code>VAL</code> .

D.8 Orden canónico estable (Capítulo 18)

Función	Entrada	Devuelve
<code>TIPORDEN</code>	<code>(TIPORDEN E)</code>	<code>0</code> si <code>E</code> es constante, <code>1</code> si es variable, <code>2</code> si es una expresión compuesta.
<code>EXPLESS?</code>	<code>(EXPLESS? E1 E2)</code>	<code>T</code> si <code>E1</code> debe ir antes que <code>E2</code> en el orden canónico; <code>NIL</code> en caso contrario.
<code>INSERTA</code>	<code>(INSERTA PAR LISTA)</code>	<code>LISTA</code> con el par <code>PAR</code> insertado en su posición ordenada según <code>EXPLESS?</code> .
<code>ORDENA</code>	<code>(ORDENA LISTA)</code>	<code>LISTA</code> de pares <code>(clave . valor)</code> ordenada según <code>EXPLESS?</code> .

D.9 Impresión infija (Capítulo 19)

Función	Entrada	Devuelve
<code>PRECOP</code>	<code>(PRECOP E)</code>	La precedencia del operador principal de <code>E</code> (potencia 3, producto 2, suma 1, átomo 4).
<code>ENVOLVER</code>	<code>(ENVOLVER E S MINIMA)</code>	La cadena <code>s</code> entre paréntesis si la precedencia de <code>E</code> es menor que <code>MINIMA</code> ; <code>s</code> sin cambios en caso contrario.
<code>NEGTERM?</code>	<code>(NEGTERM? E)</code>	<code>T</code> si <code>E</code> es un término negativo (constante negativa o producto con coeficiente negativo); <code>NIL</code> en caso contrario.
<code>NEGAR</code>	<code>(NEGAR E)</code>	<code>E</code> con el signo cambiado.
<code>INFBASE</code>	<code>(INFBASE B)</code>	La representación en texto de la base <code>B</code> de una potencia, con la precedencia mínima de un átomo.
<code>INFPROD</code>	<code>(INFPROD E)</code>	La representación en texto infija de un producto <code>E</code> (sin envolver en paréntesis).
<code>INFSUMA</code>	<code>(INFSUMA E)</code>	La representación en texto infija de una suma <code>E</code> , mostrando una resta cuando el segundo sumando es negativo.
<code>INFIJO</code>	<code>(INFIJO E MINIMA)</code>	La representación en texto infija de <code>E</code> , con los paréntesis mínimos necesarios dada la precedencia <code>MINIMA</code> del contexto.
<code>PRINTMAT</code>	<code>(PRINTMAT E)</code>	Muestra <code>E</code> en notación matemática legible y termina la línea; no devuelve un valor útil (efecto de salida).

Apéndice E — Código fuente completo de `CAS.LSP`

El listado completo, tal como queda tras los diez capítulos de la Parte II, en el mismo orden en que se fue construyendo. Pensado para copiarlo a un fichero `CAS.LSP` y cargarlo de una sola vez con `(LOAD "CAS.LSP")` (o el nombre de fichero de 8 caracteres que use tu plataforma, sección 8.3), sin tener que ir capítulo por capítulo a buscar cada función. Para saber qué recibe y qué devuelve cada una, consulta el Apéndice D.

```
; =====
; Capítulo 10 — Representación y abstracción de datos
; =====
; ** PREDICADOS DE CLASIFICACION **
(DEFUN ISCONST? (E) (NUMBERP E))
(DEFUN ISVAR? (E) (SYMBOLP E))
(DEFUN ISSUM? (E)
  (AND (LISTP E) (EQ (CAR E) '+)))
(DEFUN ISPROD? (E)
  (AND (LISTP E) (EQ (CAR E) '*)))
(DEFUN ISPOW? (E)
  (AND (LISTP E) (EQ (CAR E) 'POT)))
; ** SELECTORES **
(DEFUN ADDEND (E) (CAR (CDR E)))
(DEFUN AUGEND (E) (CAR (CDR (CDR E))))
(DEFUN FACTOR1 (E) (CAR (CDR E)))
(DEFUN FACTOR2 (E) (CAR (CDR (CDR E))))
(DEFUN BASE (E) (CAR (CDR E)))
(DEFUN EXPON (E) (CAR (CDR (CDR E))))
; ** CONSTRUCTORES (version inteligente, actualizados en el Capítulo 13) **
(DEFUN CERO? (X) (AND (ISCONST? X) (= X 0)))
(DEFUN UNO? (X) (AND (ISCONST? X) (= X 1)))
(DEFUN MAKESUM (A1 A2)
```

```

(COND ((CERO? A1) A2)
      ((CERO? A2) A1)
      ((AND (ISCONST? A1) (ISCONST? A2)) (+ A1 A2))
      (T (LIST '+ A1 A2)))
(DEFUN MAKEPROD (A1 A2)
  (COND ((OR (CERO? A1) (CERO? A2)) 0)
        ((UNO? A1) A2)
        ((UNO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (* A1 A2))
        (T (LIST '* A1 A2)))
(DEFUN MAKEPOW (B E)
  (COND ((CERO? E) 1)
        ((UNO? E) B)
        ((UNO? B) 1)
        ((AND (ISCONST? B) (ISCONST? E)) (POTENCIA B E))
        (T (LIST 'POT B E)))
; =====
; Capítulo 11 – Sustitución (SUBST) y evaluación numérica
; =====
(DEFUN SUBST (E VBL VAL)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
         (COND ((EQ E VBL) VAL)
               (T E)))
        ((ISSUM? E)
         (MAKESUM (SUBST (ADDEND E) VBL VAL)
                   (SUBST (AUGEND E) VBL VAL)))
        ((ISPROD? E)
         (MAKEPROD (SUBST (FACTOR1 E) VBL VAL)
                    (SUBST (FACTOR2 E) VBL VAL)))
        ((ISPOW? E)
         (MAKEPOW (SUBST (BASE E) VBL VAL)
                   (SUBST (EXPON E) VBL VAL)))))
(DEFUN LIBRE? (X) (EQ X 'VARLIBRE))
(DEFUN POTENCIA (B N)
  (COND ((= N 0) 1)
        (T (* B (POTENCIA B (- N 1))))))
(DEFUN EVALSUMA (E)
  (LET ((A (EVALEXPR (ADDEND E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (AUGEND E))))
                (COND ((LIBRE? B) B)
                      (T (+ A B)))))))
(DEFUN EVALPROD (E)
  (LET ((A (EVALEXPR (FACTOR1 E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (FACTOR2 E))))
                (COND ((LIBRE? B) B)
                      (T (* A B)))))))
(DEFUN EVALPOW (E)
  (LET ((B (EVALEXPR (BASE E))))
    (COND ((LIBRE? B) B)
          (T (LET ((N (EVALEXPR (EXPON E))))
                (COND ((LIBRE? N) N)
                      (T (POTENCIA B N)))))))
(DEFUN EVALEXPR (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)

```

```

      (PROGN (PRINT "CAS: variable libre, no evaluable:")
              (PRINT E)
              'VARLIBRE))
      ((ISSUM? E) (EVALSUMA E))
      ((ISPROD? E) (EVALPROD E))
      ((ISPOW? E) (EVALPOW E)))
; =====
; Capítulo 12 – Derivación simbólica cruda (DERIV)
; =====
(DEFUN DSUMA (E VBL)
  (MAKESUM (DERIV (ADDEND E) VBL) (DERIV (AUGEND E) VBL)))
(DEFUN DPROD (E VBL)
  (MAKESUM (MAKEPROD (DERIV (FACTOR1 E) VBL) (FACTOR2 E))
            (MAKEPROD (FACTOR1 E) (DERIV (FACTOR2 E) VBL))))
(DEFUN DPOW (E VBL)
  (MAKEPROD (MAKEPROD (EXPON E) (MAKEPOW (BASE E) (- (EXPON E) 1)))
            (DERIV (BASE E) VBL)))
(DEFUN DERIV (E VBL)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) (COND ((EQ E VBL) 1) (T 0)))
        ((ISSUM? E) (DSUMA E VBL))
        ((ISPROD? E) (DPROD E VBL))
        ((ISPOW? E) (DPOW E VBL))))
; =====
; Capítulo 14 – Orden canónico y términos semejantes (SIMPSUM)
; =====
(DEFUN COEF (E)
  (COND ((ISCONST? E) E)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR1 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR2 E))
               (T 1)))
        (T 1)))
(DEFUN LITERAL (E)
  (COND ((ISCONST? E) 1)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR2 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR1 E))
               (T E)))
        (T E)))
(DEFUN FLATSUM (E)
  (COND ((ISSUM? E)
         (APPEND (FLATSUM (ADDEND E)) (FLATSUM (AUGEND E))))
        (T (LIST E))))
(DEFUN ASSOCADD (LIT CF ALIST)
  (COND ((NULL ALIST) (LIST (CONS LIT CF)))
        ((EQUAL (CAR (CAR ALIST)) LIT)
         (CONS (CONS LIT (+ (CDR (CAR ALIST)) CF)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ASSOCADD LIT CF (CDR ALIST))))))
(DEFUN AGRUPA (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ASSOCADD (LITERAL (CAR TERMS))
                      (COEF (CAR TERMS))
                      (AGRUPA (CDR TERMS))))))
(DEFUN REBUILD (ALIST)
  (COND ((NULL ALIST) 0)
        (T (MAKESUM (MAKEPROD (CDR (CAR ALIST)) (CAR (CAR ALIST)))
                      (REBUILD (CDR ALIST))))))

```

```

(DEFUN SIMPSUM (E) (REBUILD (ORDENA (AGRUPA (FLATSUM E))))))
; =====
; Capítulo 15 – Simplificación de árbol completo (SIMP)
; =====
(DEFUN SIMP (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E) E)
        ((ISSUM? E)
         (SIMPSUM (MAKESUM (SIMP (ADDEND E)) (SIMP (AUGEND E))))))
        ((ISPROD? E)
         (SIMPPROD (MAKEPROD (SIMP (FACTOR1 E)) (SIMP (FACTOR2 E))))))
        ((ISPOW? E)
         (MAKEPOW (SIMP (BASE E)) (SIMP (EXPON E))))))
; =====
; Capítulo 16 – Simplificador de productos (SIMPPROD)
; =====
(DEFUN BASEOF (E) (COND ((ISPOW? E) (BASE E)) (T E)))
(DEFUN EXPOF (E) (COND ((ISPOW? E) (EXPON E)) (T 1)))
(DEFUN FLATPROD (E)
  (COND ((ISPROD? E)
         (APPEND (FLATPROD (FACTOR1 E)) (FLATPROD (FACTOR2 E))))
        (T (LIST E))))
(DEFUN NUMPROD (TERMS)
  (COND ((NULL TERMS) 1)
        ((ISCONST? (CAR TERMS)) (* (CAR TERMS) (NUMPROD (CDR TERMS))))
        (T (NUMPROD (CDR TERMS)))))
(DEFUN SYMTERMS (TERMS)
  (COND ((NULL TERMS) NIL)
        ((ISCONST? (CAR TERMS)) (SYMTERMS (CDR TERMS)))
        (T (CONS (CAR TERMS) (SYMTERMS (CDR TERMS))))))
(DEFUN ADDBASE (B EXP ALIST)
  (COND ((NULL ALIST) (LIST (CONS B EXP)))
        ((EQUAL (CAR (CAR ALIST)) B)
         (CONS (CONS B (+ (CDR (CAR ALIST)) EXP)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ADDBASE B EXP (CDR ALIST))))))
(DEFUN AGRUPAP (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ADDBASE (BASEOF (CAR TERMS))
                      (EXPOF (CAR TERMS))
                      (AGRUPAP (CDR TERMS))))))
(DEFUN REBUILDP (ALIST)
  (COND ((NULL ALIST) 1)
        (T (MAKEPROD (MAKEPOW (CAR (CAR ALIST)) (CDR (CAR ALIST)))
                      (REBUILDP (CDR ALIST))))))
(DEFUN SIMPPROD (E)
  (LET ((TERMS (FLATPROD E)))
    (MAKEPROD (NUMPROD TERMS)
              (REBUILDP (ORDENA (AGRUPAP (SYMTERMS TERMS))))))
; =====
; Capítulo 17 – El orquestador (DERIVA y DERIVAEN)
; =====
(DEFUN DERIVA (E VBL) (SIMP (DERIV E VBL)))
(DEFUN DERIVAEN (E VBL VAL) (EVALEXPR (SUBST (DERIVA E VBL) VBL VAL)))
; =====
; Capítulo 18 – Orden canónico estable (EXPLESS?)
; =====
(DEFUN TIPORDEN (E)
  (COND ((ISCONST? E) 0)

```

```

        ((ISVAR? E) 1)
        (T 2)))
(DEFUN EXPLESS? (E1 E2)
  (COND ((< (TIPORDEN E1) (TIPORDEN E2)) T)
        (> (TIPORDEN E1) (TIPORDEN E2)) NIL)
        (ISCONST? E1) (< E1 E2))
        (ISVAR? E1) (STR< E1 E2))
        (T NIL)))
(DEFUN INSERTA (PAR LISTA)
  (COND ((NULL LISTA) (LIST PAR))
        ((EXPLESS? (CAR PAR) (CAR (CAR LISTA))) (CONS PAR LISTA))
        (T (CONS (CAR LISTA) (INSERTA PAR (CDR LISTA))))))
(DEFUN ORDENA (LISTA)
  (COND ((NULL LISTA) NIL)
        (T (INSERTA (CAR LISTA) (ORDENA (CDR LISTA))))))
; =====
; Capítulo 19 – Impresión infija (PRINTMAT)
; =====
(DEFUN PRECOP (E)
  (COND ((ISPOW? E) 3)
        ((ISPROD? E) 2)
        ((ISSUM? E) 1)
        (T 4)))
(DEFUN ENVOLVER (E S MINIMA)
  (COND ((< (PRECOP E) MINIMA) (STRCAT "(" (STRCAT S ")"))
        (T S)))
(DEFUN NEGTERM? (E)
  (COND ((ISCONST? E) (< E 0))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (< (FACTOR1 E) 0))
               ((ISCONST? (FACTOR2 E)) (< (FACTOR2 E) 0))
               (T NIL)))
        (T NIL)))
(DEFUN NEGAR (E)
  (COND ((ISCONST? E) (- E))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E))
                 (MAKEPROD (- (FACTOR1 E)) (FACTOR2 E)))
               ((ISCONST? (FACTOR2 E))
                 (MAKEPROD (FACTOR1 E) (- (FACTOR2 E))))
         (T E)))
        (T E)))
(DEFUN INFBASE (B) (INFIJO B 4))
(DEFUN INFPROD (E)
  (COND ((ISCONST? (FACTOR1 E))
        (COND ((= (FACTOR1 E) -1) (STRCAT "-" (INFIJO (FACTOR2 E) 2)))
              (T (STRCAT (FACTOR1 E) (INFIJO (FACTOR2 E) 2)))))
        ((ISCONST? (FACTOR2 E))
        (COND ((= (FACTOR2 E) -1) (STRCAT "-" (INFIJO (FACTOR1 E) 2)))
              (T (STRCAT (FACTOR2 E) (INFIJO (FACTOR1 E) 2)))))
        (T (STRCAT (INFIJO (FACTOR1 E) 2)
                    (STRCAT "*" (INFIJO (FACTOR2 E) 2))))))
(DEFUN INFSUMA (E)
  (COND ((NEGTERM? (AUGEND E))
        (STRCAT (INFIJO (ADDEND E) 1)
                  (STRCAT " - " (INFIJO (NEGAR (AUGEND E)) 1))))
        (T (STRCAT (INFIJO (ADDEND E) 1)
                    (STRCAT " + " (INFIJO (AUGEND E) 1))))))

```

```
(DEFUN INFIJO (E MINIMA)
  (COND ((ISCONST? E) (STRCAT E " "))
        ((ISVAR? E) (STRCAT E " "))
        ((ISPOW? E)
         (ENVOLVER E
                    (STRCAT (INFBASE (BASE E))
                            (STRCAT "^" (INFIJO (EXPON E) 3)))
                    MINIMA))
        ((ISPROD? E) (ENVOLVER E (INFPROD E) MINIMA))
        ((ISSUM? E) (ENVOLVER E (INFSUMA E) MINIMA))))
(DEFUN PRINTMAT (E) (PROGN (DISPLAY (INFIJO E 0)) (NEWLINE)))
```